Outline

Build System Outline

BM-00 — Purpose

Turn an approved Agent Kit into a complete runnable project repository and package that repository as a portable zip export.

BM-01 — Core Principle

The Agent Kit is the **source package of build instructions**.
Build Mode is the **execution and packaging layer** around that package.

BM-02 — Main Goal

Allow the operator to choose, after Kit completion:

- stop at Agent Kit
- build full project repo
- build full project repo and export zip

BM-03 — What the Build System Consumes

Build Mode consumes:

- approved Agent Kit
- locked stack/profile selection
- locked standards snapshot
- locked templates/specs/contracts
- run metadata
- build/export policy rules

BM-04 — What the Build System Produces

Build Mode produces:

- generated project repo
- build manifest
- verification outputs
- export package

- audit/proof records for the build

BM-05 — System Boundary

The Agent Kit is responsible for:

- build instructions
- implementation guidance
- required files/modules
- standards and constraints
- acceptance expectations

Build Mode is responsible for:

- execution of repo generation
- workspace creation
- file materialization
- verification
- packaging
- export tracking

BM-06 — High-Level Flow

1. Agent Kit completes
2. operator selects Build Project
3. build eligibility is checked
4. build workspace is created
5. repo is generated from Kit instructions
6. verification runs
7. manifests are written
8. repo is zipped
9. export is attached to the run

BM-07 — Minimum Internal Components

Keep it small.

1. Build Request

Records that the operator chose to build.

2. Build Planner

Reads the Agent Kit and turns it into a concrete repo build plan.

3. Repo Generator

Creates the actual files/folders/code/config from the Kit.

4. Build Workspace

Provides the temporary and final build directories.

5. Build Verifier

Runs the required repo checks before export.

6. Export Packager

Creates the final zip.

7. Build Manifest Writer

Records what was built, from which inputs, under which rules.

BM-08 — Build States

Simple state model:

- not_requested
- requested
- approved
- building
- verifying
- failed
- passed
- exported

BM-09 — Entry Conditions

Build Mode can start only if:

- Agent Kit exists
- required gates passed
- required specs/templates are present
- critical unknowns are resolved or explicitly allowed

- build policy allows generation

BM-10 — Exit Conditions

Build Mode is complete only if:

- repo is generated
- required verification passes
- manifests are written
- zip export is created
- output is attached to the run

BM-11 — Operator Controls

The operator can choose:

- Kit only
- Build repo
- Build and export zip

The operator should also be able to:

- view build status
- view failure reason
- download final zip
- view verification result

BM-12 — Build Outputs Contract

A successful build should output at least:

- `/repo`
- `build_manifest.json`
- `repo_manifest.json`
- `verification_report.json`
- `project_repo.zip`

BM-13 — Verification Scope

Before zip export, Build Mode should verify:

- required files exist

- repo structure matches expected layout
- config/package files exist
- unresolved critical placeholders do not remain
- required commands/checks pass
- manifest is complete

BM-14 — Packaging Rule

The zip export is a **release-style output**, not just a convenience file.

That means export should only happen after build verification passes. This aligns with your existing separation of release packaging as its own governed concern.

Response: Axion Updates Planning

BM-15 — Audit / Proof Requirements

Each build should record:

- source run ID
- source Agent Kit ID/version
- template/spec snapshot used
- build start/end timestamps
- verification results
- export artifact reference

BM-16 — Failure Modes

Minimum failure classes:

- missing Kit input
- incomplete specs/templates
- repo generation failure
- verification failure
- export failure

BM-17 — Non-Goals

Build Mode is not responsible for:

- inventing product requirements from scratch
- bypassing missing Kit inputs
- silently fixing unresolved critical gaps

- replacing the control plane
- replacing maintenance packaging modes

BM-18 — Relationship to Maintenance Modes

Build Mode should be a **runtime/core build capability**.

It can later feed or interact with maintenance functions like **MM-14 — Release Packaging**, but it should not be reduced to just a maintenance mode itself.

Response: Axion Updates Planning

Bottom line

So yes, it should be fairly simple **if you treat the Agent Kit as the build instruction package**.

But no, it is not just:
“Kit exists, therefore zip repo comes out.”

You still need a thin internal system that handles:
request → build → verify → package → export

That is the right level of simplicity.

Next step is to turn this into a formal Axion section like:

BM-01 — Internal Build Mode
with:
Purpose, Inputs, Flow, Components, Gates, Outputs, Proofs.

BM-00 — Purpose

BM-00 — Purpose

1. Purpose of this section

Define why the Build Mode exists, what problem it solves, and where it fits inside the Axion system.

BM-00 is the anchor section for the whole Build Mode branch. It should make it clear that this is not a separate product, not a random zip exporter, and not a replacement for the Agent Kit. It is the internal system responsible for turning an approved Agent Kit into a generated project repository and, when allowed, packaging that repository as a portable output.

---

2. Core purpose

The purpose of Build Mode is to let Axion move beyond planning and kit assembly into controlled project generation.

Build Mode exists so that, after an Agent Kit is completed and approved, the operator can choose to have Axion generate the actual application codebase as a full repository rather than stopping at documentation and build instructions.

---

3. Primary outcome

The primary outcome of Build Mode is:

- a generated project repository
- produced from an approved Agent Kit
- under locked build constraints
- verified before release
- optionally exported as a zip package for IDE import and continued development

---

4. Why this system exists

The Agent Kit tells the build agent what to build, but the system still needs an internal layer that governs how that build happens.

Without Build Mode, Axion stops at:

- plans
- templates
- specs
- contracts
- instructions

With Build Mode, Axion can continue into:

- repo generation
- structured code materialization
- build verification
- controlled export packaging

So the purpose of Build Mode is to bridge the gap between:
“the system knows what should be built”
and
“the system has produced the actual project repo.”

---

5. Position in the Axion system

Build Mode is an internal runtime capability of Axion.

It sits after Agent Kit generation and approval, and before final export/download.

It should be understood as:

- downstream of Kit assembly
- governed by control-plane rules

- dependent on locked specs, templates, and standards
- responsible for repo generation and export preparation

Build Mode is not the same as the main control plane, the Agent Kit, or release packaging. It depends on those systems and coordinates with them.

---

6. What Build Mode is

Build Mode is:

- an internal build execution system
- a repo generation pipeline
- a governed code-materialization layer
- a bridge from approved Kit to runnable repository
- a controlled path to portable project output

---

7. What Build Mode is not

Build Mode is not:

- a replacement for the Agent Kit
- a freeform coding session
- a generic code generator with no constraints
- a standalone external builder product
- just a zip export tool
- a system that invents missing requirements on its own

---

8. Core principle

Build Mode must operate from the principle that the Agent Kit provides the build intelligence, while Build Mode provides the execution structure.

In simple terms:

- the Agent Kit defines **what must be built**
- the Build Control Plane defines **how the build is allowed to happen**
- the build agent performs **the controlled materialization of the repo**

That means Build Mode should be designed to reduce guessing, reduce drift, and keep repo generation inside a locked execution path.

---

9. Main problem it solves

Build Mode solves the problem of stopping too early.

Without it, Axion can produce a strong build package but still leaves the operator to manually move that package into an IDE or a coding workflow and get the repo built elsewhere.

Build Mode removes that gap by letting Axion itself carry the process forward into:

- repo scaffold creation
- code generation
- file assembly
- verification
- export

---

10. Operator value

For the operator, Build Mode creates a plug-and-play path.

Instead of only receiving planning artifacts, the operator can choose to receive:

- the Agent Kit only, or
- the completed project repository, or
- the completed project repository packaged as a zip

That makes Axion more directly useful as a production system.

---

11. System value

For the system, Build Mode increases internal continuity.

It keeps the build process connected to:

- the same run
- the same artifacts
- the same standards snapshot
- the same gates
- the same audit trail
- the same packaging controls

That is better than handing off the build to an unrelated external process.

---

12. Success definition

BM-00 should define success at the highest level as:

Build Mode successfully fulfills its purpose when Axion can take an approved Agent Kit and produce a verified, portable, ready-to-open project repository under controlled build rules and without relying on uncontrolled agent guessing.

---

13. Short version for the doc

If you want a tighter version to place at the top of the section, use this:

BM-00 — Purpose

Build Mode exists to transform an approved Agent Kit into a controlled, verified, and portable project repository. It is the internal Axion capability that governs repo generation, build verification, and export packaging so the operator can move from kit completion to plug-and-play project output without relying on freeform agent behavior.

BM-01 — Core Principle

BM-01 — Core Principle

1. Purpose of this section

Define the governing idea behind Build Mode.

BM-01 explains the rule that keeps Build Mode from turning into a loose AI coding workflow. This section should make clear that Build Mode is not supposed to “figure things out however it wants.” Its job is to execute a controlled build path from an approved Agent Kit.

---

2. Core principle

The core principle of Build Mode is:

The Agent Kit defines what must be built, and the Build Control Plane constrains how it is built, so the build agent can materialize the repo with minimal guessing and minimal drift.

That is the foundation of the whole system.

---

3. Why this principle matters

Once Axion starts generating actual code, the workload becomes heavier and the risk of drift increases.

Without a strong principle, the build agent can start:

- inferring things that were never locked
- filling gaps with guesses
- adding files or dependencies outside scope
- solving ambiguity in inconsistent ways
- producing repos that technically exist but do not match the approved run

BM-01 exists to prevent that.

---

4. Division of responsibility

Build Mode should follow a strict split of responsibility.

The Agent Kit is responsible for:

- defining the product and feature scope
- defining requirements, specs, and contracts
- defining standards and constraints
- defining what outputs are expected
- defining the build intent

The Build Control Plane is responsible for:

- locking the execution path
- locking allowed build order
- locking file targets and repo shape
- enforcing no-guess behavior
- enforcing pass/fail rules
- enforcing retry and stop conditions

The Build Agent is responsible for:

- executing the locked build path
- generating code and files inside the defined scope
- wiring the repo together according to the approved plan
- stopping when locked inputs are missing or conflicted

This division is what makes Build Mode controlled instead of freeform.

---

5. Build Mode is not a creative coding mode

Build Mode must not be treated as an open invention step.

Its job is not to creatively design the app from scratch.
Its job is to **materialize an already-defined app**.

That means Build Mode should prefer:

- execution over invention
- compliance over improvisation
- stopping over guessing
- traceable decisions over hidden assumptions

---

6. Build intelligence vs build execution

BM-01 should clearly separate two things:

Build intelligence

This comes from the Agent Kit and related locked artifacts.
It answers:

- what is this app
- what features exist
- what modules are required
- what standards apply
- what contracts must be satisfied

Build execution

This comes from Build Mode and the Build Control Plane.
It answers:

- in what order is the repo built
- what files must be created
- what is allowed during generation
- what checks must pass
- when the system must stop, fail, retry, or export

This separation is the main anti-drift rule.

---

7. Minimal-guess operating rule

Build Mode should operate under a minimal-guess rule:

If the required input is explicit, follow it.
If the required input is missing, unclear, or conflicted, do not invent a replacement.

Instead, the system should:

- stop
- fail the relevant step
- record the reason
- surface the blockage to the operator or governing layer

That rule is one of the most important parts of the principle.

---

8. Locked-path execution

Build Mode should be designed as a locked-path system.

That means the build agent should not choose:

- the repo structure from scratch
- the module order from scratch
- dependencies from scratch
- naming from scratch
- scope expansion from scratch

Those things should already be constrained before repo generation begins.

The more those are locked, the more deterministic the build becomes.

---

9. Controlled reasoning, not unrestricted reasoning

The build agent will still need reasoning.
But the reasoning should be narrow and local.

Allowed reasoning should be things like:

- filling in implementation details inside a locked file target
- wiring known modules together
- satisfying an already-declared contract
- translating approved specs into code structure

Disallowed reasoning should be things like:

- inventing new major features
- changing the approved architecture
- adding undeclared libraries by preference
- redefining entities, routes, or flows outside locked scope

So BM-01 should state that Build Mode uses **controlled reasoning inside fixed boundaries**, not unrestricted autonomous interpretation.

---

10. Principle of traceable materialization

Every major build action should be traceable back to an approved source.

In practice, generated code and repo structure should be attributable to:

- Agent Kit instructions
- locked specs
- locked contracts
- locked template decisions
- locked build policies

That way the build is not a black box.
It is a materialization of approved upstream inputs.

---

11. Principle of governed completion

A repo is not considered complete just because files exist.

Build Mode completion should mean:

- the repo was generated from approved inputs
- the repo stayed inside locked scope
- required checks passed
- export eligibility was satisfied

So BM-01 should establish that **completion is governed**, not just produced.

---

12. Principle of fail-fast over silent drift

When uncertainty appears, the system should prefer failure over hidden improvisation.

That means:

- fail early on missing required artifacts
- fail on unresolved critical contradictions
- fail on blocked file targets
- fail on policy violations
- do not silently patch around core gaps

This keeps the build trustworthy.

---

13. Design outcome

If BM-01 is followed correctly, Build Mode should feel less like:

- “AI builds an app however it wants”

and more like:

- “the system walks through a controlled repo-construction path using a locked instruction package”

That is the actual desired behavior.

---

14. Success condition for this principle

BM-01 is successful when Build Mode behaves as a constrained execution system where:

- the Agent Kit supplies the build intelligence
- the Build Control Plane supplies the rails
- the build agent performs only the necessary materialization work
- guessing and drift are minimized by design

15. Short version for the doc

BM-01 — Core Principle

Build Mode operates on a locked-path principle: the Agent Kit defines what must be built, the Build Control Plane constrains how the build may occur, and the build agent materializes the repository inside those boundaries. The system must prefer controlled execution over freeform interpretation, and must stop on critical ambiguity rather than guess.

BM-02 — Main Goal

BM-02 — Main Goal

1. Purpose of this section

Define the main operational goal of Build Mode in practical terms.

BM-02 should explain what the system is trying to accomplish once it is invoked, what success looks like from the operator side, and what the end result should be for the Axion run.

---

2. Main goal

The main goal of Build Mode is to let the operator take an approved Agent Kit and choose to have Axion generate a full project repository that is ready for direct use in a development environment.

In simple terms, the goal is to move from:

- approved build instructions

to:

- generated project repo
- verified output
- portable export

without requiring a separate manual build handoff.

---

3. Operator-facing goal

From the operator’s point of view, the goal is simple:

After the Agent Kit is complete, the operator should be able to choose one of three outcomes:

- stop at Agent Kit

- generate the full project repo
- generate the full project repo and export it as a zip

That is the practical goal of the system.

---

4. System-facing goal

From the system's point of view, the goal is to take all approved upstream build intelligence and materialize it into a controlled repository output.

That means Build Mode should:

- consume the approved Kit and locked build inputs
- generate the repo in a governed way
- verify the result
- package the output if allowed
- attach the result back to the same run

So BM-02 is not just "write code."
It is "produce a governed build output."

---

5. What Build Mode is trying to eliminate

The main goal also includes removing the gap between planning and usable development output.

Without Build Mode, the workflow stops at:

- specs
- templates
- plans

- instructions
- build guidance

Then the operator still has to move that package into another environment and manually continue from there.

Build Mode exists to eliminate that gap by allowing Axion itself to produce the repo.

---

6. Target outcome

The target outcome of Build Mode is a repository that is:

- generated from the approved Kit
- structurally complete enough to open in an IDE
- aligned with locked scope and standards
- verified against required checks
- optionally packaged as a portable zip artifact

The target is not “some code.”
The target is a **usable project repo output**.

---

7. Goal in relation to plug-and-play use

BM-02 should explicitly support your plug-and-play intention.

The goal is to make the output portable enough that the operator can:

- download the repo
- open it in an IDE
- inspect it
- run the next development steps from a real codebase

- continue without rebuilding the project structure manually

That does not necessarily mean every generated app is production-perfect.
It means the system should aim to produce a repo that is ready to enter normal development immediately.

---

8. Goal in relation to drift reduction

The goal is not only to generate code faster.
It is to generate code with less drift.

So BM-02 should include that Build Mode aims to:

- reduce manual repo setup
- reduce interpretation gaps
- reduce off-scope generation
- reduce inconsistent file creation
- reduce the need for the operator to reconstruct the project from kit documents

This is part of the value of the mode.

---

9. Goal in relation to internal continuity

Another major goal is continuity inside the same Axion run.

Build Mode should keep repo generation connected to:

- the same run
- the same Kit
- the same standards snapshot
- the same templates/specs/contracts

- the same audit trail
- the same verification model

So the goal is not just repo creation.
It is repo creation **without breaking the internal chain of custody**.

---

10. Goal in relation to packaging

Build Mode should aim to produce a zip export only as a final governed result, not as a shortcut.

So the goal is:

- first produce a valid generated repo
- then verify it
- then package it if allowed

The goal is not “always output a zip no matter what.”
The zip is the final portable form of a successful build path.

---

11. Non-goals inside the main goal

BM-02 should clarify what the mode is not trying to do.

The main goal is not:

- replacing product planning
- inventing requirements during build
- improvising architecture outside approved scope
- bypassing missing Kit inputs
- shipping code without verification
- hiding unresolved critical gaps

Those are outside the goal of Build Mode.

---

12. Success definition

BM-02 should define success as:

Build Mode succeeds when an operator can take an approved Agent Kit, trigger a governed build path, and receive a verified project repository—optionally zipped for export—without relying on uncontrolled interpretation or manual reconstruction of the codebase.

---

13. Short version for the doc

BM-02 — Main Goal
The main goal of Build Mode is to let an operator move from an approved Agent Kit to a generated, verified, and portable project repository inside the same Axion run. It exists to remove the gap between build instructions and usable repo output by giving the operator a controlled choice to stop at the Kit, generate the repo, or generate and export the repo as a zip.

BM-03 — What the Build System Consumes

BM-03 — What the Build System Consumes

1. Purpose of this section

Define every input Build Mode depends on before it can generate a repo.

BM-03 should make clear that Build Mode does not build from nothing. It consumes a locked set of upstream inputs from the Axion run. This section exists to define those inputs so the build process is traceable, bounded, and less likely to drift.

---

2. Core rule

Build Mode may only build from approved and recognized inputs.

That means the system should not treat freeform assumptions, undeclared operator intent, or missing upstream details as valid build sources. If an input was not provided, resolved, or allowed by policy, Build Mode should not silently substitute its own version.

---

3. Primary consumed input

The primary consumed input is the **approved Agent Kit**.

The Agent Kit is the main build-intelligence package. It tells Build Mode what the app is, what scope has been approved, what structures and constraints must be respected, and what the generated repo is expected to contain.

Without the Agent Kit, Build Mode has no legitimate build basis.

---

4. Core consumed inputs

At minimum, Build Mode should consume the following categories of input.

4.1 Approved Agent Kit

This is the main source package for build intent and implementation guidance.

It should include or point to:

- approved scope
- feature/module expectations
- architecture direction
- specs/contracts/templates
- acceptance expectations
- build-relevant instructions

4.2 Run metadata

Build Mode should consume the run context that produced the Kit.

This includes things like:

- run ID
- project/workspace ID
- profile or environment selection
- timestamps
- operator/build request linkage
- upstream artifact references

This keeps the build tied to the correct run lineage.

4.3 Locked standards snapshot

Build Mode should consume the standards state that was resolved for the run.

This includes:

- naming standards
- architecture constraints
- coding standards that are relevant to generation

- repo structure expectations
- required operational/verification standards

This prevents the build from drifting into a different standards interpretation midway through generation.

4.4 Locked templates, specs, and contracts

Build Mode should consume the actual implementation-relevant artifacts that the Kit references.

Depending on the run, this may include:

- route specs
- entity specs
- page/screen specs
- schema specs
- component specs
- flow specs
- template selections
- acceptance mappings
- domain-specific contracts

These are the detailed materials that the build agent uses to materialize the repo.

4.5 Build/export policy rules

Build Mode should consume policy constraints that determine:

- whether build is allowed
- whether export is allowed
- what must pass before packaging

- whether unresolved items are tolerable
- what stop conditions apply

This keeps build eligibility governed instead of assumed.

4.6 Build profile / stack profile

Build Mode should consume a locked technical profile for the repo being generated.

This may include:

- framework stack
- runtime assumptions
- language choices
- package manager choice
- folder conventions
- build/test command expectations
- dependency boundaries

This is important because the build agent should not choose the stack on its own during generation.

---

5. Secondary consumed inputs

Beyond the core inputs, Build Mode may also consume additional build-supporting inputs when available.

5.1 Repo blueprint / file tree target

A declared target structure for the repository.

This may define:

- root folders

- domain/module folders
- config file locations
- shared library locations
- test locations
- docs/env/example file locations

If present, this should strongly constrain generation order and output placement.

5.2 Required file inventory

A declared set of files that must exist for the build to be considered structurally complete.

This helps prevent the agent from loosely approximating the repo shape.

5.3 Module or slice ordering

A declared order for how the build should be materialized.

For example:

- repo skeleton first
- contracts/types second
- backend next
- frontend next
- tests/docs last
- verification/export after that

If present, Build Mode should follow it instead of inventing its own order.

5.4 Verification command set

A defined set of commands or checks that must run before export eligibility is granted.

This may include:

- install checks
- type checks
- lint checks
- test checks
- structural checks
- placeholder scans
- manifest consistency checks

5.5 Dependency allowlist / restrictions

A declared set of allowed libraries, package classes, or tooling boundaries.

This is useful for preventing the build agent from solving gaps by adding undeclared dependencies.

---

6. Operator-provided trigger input

Build Mode should also consume the operator's post-Kit choice.

That choice may be one of:

- Kit only
- Build repo
- Build repo and export zip

This matters because Build Mode should not automatically assume the operator wants full repo generation just because the Kit exists.

The trigger is part of the consumed input set because it activates the downstream build path.

---

7. Eligibility inputs

Build Mode should consume proof that required upstream conditions have been satisfied.

This includes build eligibility evidence such as:

- Kit completion status
- gate pass results
- required artifact presence
- critical unknown resolution state
- policy eligibility state

This is different from build content.
It is the evidence that Build Mode is allowed to proceed.

---

8. Consumption boundary

BM-03 should clearly state what Build Mode must not consume as authoritative input.

Build Mode should not treat these as valid primary build sources:

- hidden assumptions
- speculative agent guesses
- undeclared feature ideas
- unstated operator intent
- stale or superseded artifacts
- conflicting inputs that were never resolved upstream

If such things exist, they must be resolved before or during eligibility checks, not quietly absorbed as build truth.

---

9. Input hierarchy

Build Mode should follow a clear input precedence order.

A good hierarchy is:

1. approved run and operator authorization
2. approved Agent Kit
3. locked standards snapshot
4. locked templates/specs/contracts
5. locked build profile / repo blueprint
6. policy and verification rules
7. optional supporting metadata and inventories

This hierarchy helps when multiple inputs overlap. The build should rely on the highest-authority approved source rather than improvising across conflicts.

---

10. Required minimum input set

BM-03 should define the minimum set required to start Build Mode.

At minimum, Build Mode should require:

- approved Agent Kit
- run metadata
- locked standards snapshot or equivalent resolved rules
- locked build profile
- required specs/templates/contracts for the declared scope
- build eligibility proof
- operator build request

If one of those is missing, the build should not begin.

---

11. Why this section matters

This section matters because a lot of build drift starts before code generation even begins.

If the consumed inputs are vague, incomplete, or mixed with unapproved assumptions, the build agent is forced to compensate. That compensation is where guessing enters the system.

So BM-03 protects the build by making the upstream input set explicit and governed.

---

12. Success definition

BM-03 is successful when Build Mode only begins from a clear, approved, and traceable set of upstream inputs, and when every major build action can be tied back to those consumed sources rather than freeform interpretation.

---

13. Short version for the doc

BM-03 — What the Build System Consumes
Build Mode consumes the approved Agent Kit, run metadata, locked standards, locked templates/specs/contracts, build profile constraints, policy rules, and build eligibility evidence. These inputs form the authoritative source package for repo generation. Build Mode must not treat guesses, undeclared scope, or unresolved conflicts as valid build inputs.

BM-04 — What the Build System Produces

BM-04 — What the Build System Produces

1. Purpose of this section

Define the outputs that Build Mode is responsible for creating.

BM-04 should make clear that Build Mode does not just “write some code.” It produces a governed set of repo-level outputs that together represent a completed build result. This section exists to define those outputs so success is measurable, packaging is controlled, and the final result is portable and traceable.

---

2. Core rule

Build Mode must produce outputs that are complete enough to support inspection, verification, and handoff.

That means the system should not treat raw generated code alone as the full output. A valid build result should include the generated repository plus the supporting records that explain what was built, how it was built, and whether it passed the required checks.

---

3. Primary output

The primary output of Build Mode is the **generated project repository**.

This is the materialized codebase produced from the approved Agent Kit and locked upstream inputs. It is the main deliverable of the build process.

The generated repository should be structured according to the locked build profile, repo blueprint, scope definition, and approved implementation artifacts.

---

4. Core produced outputs

At minimum, Build Mode should produce the following output categories.

4.1 Generated project repository

This is the actual repo produced by the system.

It should include, as applicable to the run:

- source code
- folder structure
- config files
- manifests
- shared types/contracts
- backend code
- frontend code
- test files
- environment example files
- supporting documentation files
- other required build-scope artifacts

This is the main build product.

4.2 Build manifest

Build Mode should produce a build manifest that records the build event.

This should capture things like:

- run ID
- build ID
- source Agent Kit reference
- source standards/spec/template snapshot references
- build profile used
- build start and end time

- build status
- produced artifact references

The build manifest is the main run-level record of what happened during generation.

4.3 Repo manifest

Build Mode should produce a repo manifest that describes the generated repository itself.

This may include:

- repo root description
- expected folder structure
- file inventory
- module inventory
- dependency summary
- build commands
- verification command set
- generated output summary

This makes the repo easier to inspect and validate.

4.4 Verification outputs

Build Mode should produce the records from the checks that determine whether the repo is eligible for completion and export.

This may include:

- verification report
- command results
- structural check results

- missing file checks
- placeholder scan results
- consistency check results
- pass/fail summaries

These outputs show whether the generated repo met the required conditions.

4.5 Export package

If export is allowed and requested, Build Mode should produce the final portable package.

This should normally be:

- a zip archive of the generated repository
- with the required manifests and supporting outputs included as defined by policy

The export package is the handoff-ready form of the build result.

---

5. Supporting produced outputs

Beyond the core outputs, Build Mode may also produce additional supporting outputs to improve traceability and recovery.

5.1 Build plan output

If the system derives a concrete repo build plan from the Agent Kit, that plan should be written out as its own artifact.

This can help with:

- debugging build order
- proving what the system intended to generate
- resuming or re-running builds
- comparing generated output to planned output

5.2 Workspace records

If the system uses a staging/build workspace, it may produce records describing:

- staged directories
- intermediate output locations
- phase results
- slice completion status

These records help operationally even if they are not part of the final handoff bundle.

5.3 File index

The system may produce an index of generated files.

This can include:

- relative file paths
- file roles
- source references
- status markers
- size/hash metadata if you choose to track that

This is useful for inspection and integrity checks.

5.4 Failure report

If the build fails, the system should still produce structured failure output.

That report may include:

- failed phase
- failed checks
- blocking reason

- missing inputs
- unresolved conflicts
- partial outputs produced before failure

A failed build should still leave a usable trace.

---

6. Output boundary

BM-04 should clearly state what Build Mode is not required to produce.

Build Mode is not required to produce:

- a production deployment
- a live hosted environment
- a guarantee that every business decision is finalized
- undocumented improvisations outside approved scope
- a successful export when build verification fails

Its job is to produce a governed repository output, not to pretend the build succeeded when it did not.

---

7. Output quality expectation

The produced outputs should be:

- structured
- inspectable
- traceable
- tied to the originating run

- consistent with locked inputs
- sufficient for verification and handoff

This means the output set should support both:

- human operator review
- system-level enforcement and audit

---

8. Output hierarchy

Build Mode should distinguish between output layers.

A clean output hierarchy is:

Layer 1 — Main build artifact

- generated project repository

Layer 2 — Build records

- build manifest
- repo manifest
- build plan
- file index

Layer 3 — Verification evidence

- verification report
- command results
- structural checks
- placeholder scans

Layer 4 — Portable handoff artifact

- export zip

This hierarchy helps you separate “what was built” from “proof that it is valid” and from “portable delivery form.”

---

9. Minimum successful output set

BM-04 should define the minimum successful output set.

At minimum, a successful Build Mode run should produce:

- generated project repository
- build manifest
- verification report
- repo manifest or equivalent repo summary
- export zip if export was requested and permitted

If that minimum set is not produced, the build should not be considered fully complete.

---

10. Output relationship to the run

All outputs should remain attached to the same Axion run that produced the Agent Kit.

That means the produced outputs should be attributable through:

- run ID
- build ID
- source artifact references
- timestamps

- status markers

This preserves continuity and avoids detached repo generation.

---

11. Output relationship to export

The export package should be treated as the final portable expression of the build, not the only meaningful output.

Even when a zip is produced, the system should still preserve:

- the generated repo itself
- the manifests
- the verification outputs
- the build records

The zip should not replace internal traceability.

---

12. Why this section matters

This section matters because Build Mode can look deceptively simple if you describe the output as “a repo zip.”

But the real system output is broader than that.
A trustworthy build needs:

- the repo
- the records
- the verification evidence
- the export artifact when allowed

BM-04 prevents Build Mode from being reduced to a blind generator.

---

13. Success definition

BM-04 is successful when Build Mode produces a complete and governed output set consisting of the generated repository plus the records and verification evidence needed to inspect, validate, and export that result.

---

14. Short version for the doc

BM-04 — What the Build System Produces
Build Mode produces a governed build output set that includes the generated project repository, build and repo manifests, verification outputs, and—when requested and allowed—a portable zip export. The repo is the primary artifact, while manifests and verification records provide the traceability and proof needed for inspection, completion, and handoff.

BM-05 — System Boundary

BM-05 — System Boundary

1. Purpose of this section

Define exactly where Build Mode begins and ends.

BM-05 should make clear what Build Mode owns, what it depends on, and what remains outside its responsibility. This section prevents overlap with the Agent Kit, the main control plane, packaging logic, and any later deployment workflows.

---

2. Core boundary rule

Build Mode is responsible for **controlled repo generation and build-result packaging readiness**, but it is not responsible for inventing the app, replacing upstream planning, or acting as a deployment system.

Its job starts with an approved build basis and ends with a generated, verified, and optionally exported project repository.

---

3. Upstream boundary

Build Mode begins **after** the upstream planning and assembly layers have done their work.

That means Build Mode depends on upstream systems to provide:

- approved scope
- approved Agent Kit
- resolved standards
- selected templates/specs/contracts
- build eligibility state
- operator build request

Build Mode should not be the place where those things are first decided.

---

4. Downstream boundary

Build Mode ends once it has:

- generated the repository
- completed required build verification
- written the required manifests/records
- produced the export package if requested and allowed

Anything after that—such as manual editing in an IDE, deployment, hosting, release operations, or post-build maintenance—is outside the core boundary of Build Mode.

---

5. What Build Mode owns

Build Mode should explicitly own the following responsibilities.

5.1 Build execution

It owns the act of turning approved inputs into a materialized repository.

5.2 Repo generation

It owns creating the repo structure, files, wiring, and implementation outputs inside locked constraints.

5.3 Build workspace handling

It owns the internal workspace/staging path used during repo generation.

5.4 Build verification

It owns the build-phase checks required before the result can be treated as successful or exportable.

5.5 Build records

It owns the build manifests, repo summaries, verification records, and failure records tied to the build event.

5.6 Export packaging readiness

It owns preparing the generated repository for portable export and producing the zip when the run allows it.

---

6. What Build Mode does not own

Build Mode should explicitly not own the following.

6.1 Product definition

It does not define what the product should be. That belongs upstream in intake, canonicalization, planning, templates, specs, and the Agent Kit.

6.2 Scope invention

It does not invent missing features, requirements, entities, flows, or business logic outside the approved build basis.

6.3 Standards resolution

It does not decide or reinterpret standards during the build. It consumes the locked standards state.

6.4 Template/spec selection

It does not choose upstream templates/specs/contracts from scratch. It consumes the locked selections.

6.5 Main run governance

It does not replace the main control plane. It operates under it and may have a build-focused sub-control layer.

6.6 External deployment

It does not deploy the app, host the app, configure production infrastructure, or guarantee release to a live environment.

6.7 Manual development continuation

It does not own what happens after the exported repo is opened in an IDE and further edited by a human or another tool.

---

7. Boundary with the Agent Kit

This is one of the most important separations.

The Agent Kit owns:

- build intelligence
- approved implementation intent
- scope and requirements guidance
- standards/spec/template references
- build-facing instructions

Build Mode owns:

- execution of repo generation from that package
- controlled materialization of code/files
- verification of the build result
- production of the portable repo output

So the Agent Kit defines **what should be built**, while Build Mode governs **how the approved repo gets produced**.

---

8. Boundary with the main control plane

Build Mode should sit under the authority of the main control plane, not replace it.

Main control plane owns:

- full run lifecycle
- stage progression
- cross-run policies

- artifacts at the system level
- approvals and audit
- overall packaging/release governance

Build Mode / build control layer owns:

- build request handling
- build execution path
- build states
- build-phase constraints
- build verification and export readiness

This boundary keeps Build Mode focused while still governed by the larger system.

---

9. Boundary with packaging

Build Mode and packaging are related, but they are not identical.

Build Mode is not merely a zip-export utility.
Its main job is repo generation.

Packaging within Build Mode should be understood as:

- the final handoff form of a successful build
- dependent on verification and policy
- a downstream expression of build success

So the zip is part of the output boundary, but it is not the whole system.

---

10. Boundary with verification

Build Mode owns **build-phase verification**, not all possible verification in the entire Axion ecosystem.

That means it should verify things like:

- repo structure
- required file presence
- placeholder resolution state
- required commands/checks
- output consistency

But broader system-wide assurance, external QA strategy, or downstream release certification can remain outside this mode unless explicitly included elsewhere.

---

11. Boundary with recovery and failure

Build Mode should own failure handling for its own execution path.

It should:

- stop on blocked prerequisites
- produce failure records
- identify failed phases or checks
- allow controlled retry/resume where supported

But it should not silently resolve upstream planning problems by inventing new truths during the build.

---

12. Boundary in time

A simple time-based way to define BM-05 is:

Before Build Mode

- intake
- canonicalization
- standards resolution
- template/spec/contract assembly
- planning
- Agent Kit generation
- approvals/eligibility

During Build Mode

- build request accepted
- repo generation begins
- files are materialized
- verification runs
- manifests are written
- export is produced if allowed

After Build Mode

- repo downloaded/opened elsewhere
- manual edits continue
- deployment/release workflows begin
- maintenance/update cycles continue

This timeline helps make the boundary obvious.

---

13. Why this section matters

Without a clear boundary, Build Mode can become bloated and confusing.

People start expecting it to:

- solve missing planning gaps
- replace the control plane
- do deployment
- make product decisions
- hide unresolved conflicts
- act like a fully autonomous dev shop

BM-05 protects the system from that kind of scope creep.

---

14. Success definition

BM-05 is successful when it is clear that Build Mode starts from approved upstream build inputs, owns controlled repo generation and build verification, and ends at verified repo output and optional export—without absorbing product planning, standards resolution, or deployment responsibilities.

---

15. Short version for the doc

BM-05 — System Boundary
Build Mode begins after the Agent Kit and other upstream build inputs are approved, and it ends once the repository has been generated, verified, and optionally exported. It owns controlled repo materialization, build-phase verification, and build records, but it does not own product definition, standards resolution, upstream scope decisions, or downstream deployment.

BM-06 — High-Level Flow

BM-06 — High-Level Flow

1. Purpose of this section

Define the end-to-end flow of Build Mode at a system level.

BM-06 should show the major steps Build Mode follows from activation to completion. This section is not for low-level implementation detail yet. It is for establishing the standard sequence of the build path so the mode stays predictable, governed, and easy to reason about.

---

2. Core flow rule

Build Mode should follow a fixed high-level sequence.

It should not behave like an open-ended loop where the build agent decides the order on its own. The flow needs to be stable enough that:

- the control plane can govern it
- failure points are clear
- build records can map to known phases
- retry/resume can be handled cleanly
- export only happens after valid completion

---

3. Entry point

Build Mode begins only after the upstream build basis is complete and the operator chooses to continue beyond the Agent Kit.

That means the high-level flow starts from:

- approved Agent Kit
- satisfied build eligibility conditions

- operator request to build the repo or build and export the repo

If those conditions are not met, the flow does not start.

---

4. High-level sequence

The standard Build Mode flow should be:

1. Build request is accepted
2. Build eligibility is confirmed
3. Build plan is derived from approved inputs
4. Build workspace is prepared
5. Repository skeleton and required structure are created
6. Repo contents are generated in controlled slices
7. Build records and manifests are written or updated
8. Build-phase verification runs
9. Export eligibility is evaluated
10. Zip export is created if requested and allowed
11. Final build status is recorded and attached to the run

That is the full system-level path.

---

5. Step 1 — Build request accepted

The first step is acceptance of the operator’s build choice.

The operator should be able to choose:

- Kit only
- Build repo
- Build repo and export zip

Build Mode only activates when one of the build-producing options is chosen.

At this step, the system should record:

- the operator request
- the requested output mode
- the run/build linkage
- request timestamp
- initial build status

This turns the build from an idea into a governed run event.

---

6. Step 2 — Build eligibility confirmed

Before generation begins, Build Mode should confirm that the run is allowed to build.

This step checks that:

- the Agent Kit exists
- required upstream gates passed
- required artifacts are present
- critical unresolved blockers are absent or explicitly tolerated by policy
- build/export policy allows progression

If eligibility fails, the flow stops here and a failure result is recorded.

This step is important because it prevents the build agent from compensating for missing upstream truth.

---

7. Step 3 — Build plan derived

Once eligibility passes, Build Mode should derive a concrete build plan from the approved input package.

This step converts upstream build intelligence into an actionable generation path.

The plan may define:

- repo shape
- required files/modules
- build phases or slices
- generation order
- dependency boundaries
- required checks
- export expectations

This is the point where the system turns “approved build basis” into “approved execution path.”

---

8. Step 4 — Build workspace prepared

Before materializing the repo, the system should prepare the internal build workspace.

This step may include:

- creating working directories
- creating staging areas
- defining output paths

- preparing manifest locations
- preparing verification output locations
- preparing export destinations

The workspace gives the build process an organized execution area instead of writing blindly into final output.

---

9. Step 5 — Repository skeleton created

The system should then create the base repo structure before deeper code generation begins.

This step may include:

- root folder creation
- domain/module folder creation
- config/manifests file stubs
- shared directory setup
- test/docs/env/example locations
- other required scaffolding

This helps anchor the build before implementation details are written.

---

10. Step 6 — Repo contents generated in controlled slices

After the skeleton exists, Build Mode should generate the repo contents in controlled slices rather than as one giant undifferentiated pass.

Typical slices might include:

- config and manifest layer
- shared types/contracts layer

- backend layer
- frontend layer
- test layer
- docs/env/example layer

The exact slices may vary by stack, but the rule should stay the same:
Build progressively and in a declared order.

This is the main materialization phase of the flow.

---

11. Step 7 — Build records and manifests written

As the build progresses or after generation completes, Build Mode should write the required records.

This can include:

- build manifest
- repo manifest
- file inventory
- phase completion state
- source references
- build status updates

This step ensures the generated repo is accompanied by traceable build metadata.

---

12. Step 8 — Build-phase verification runs

Once generation is complete enough, the build should move into verification.

This step checks whether the produced repo satisfies the required structural and policy conditions.

Verification may include:

- required file presence
- repo structure checks
- placeholder scans
- consistency checks
- command execution checks
- manifest completeness checks

If required verification fails, the build should not be marked successful and export should not proceed.

---

13. Step 9 — Export eligibility evaluated

After verification passes, the system should determine whether export is allowed.

This step checks:

- whether export was requested
- whether all required build checks passed
- whether policy allows export
- whether required manifests and records exist

A successful build does not automatically mean export should happen unless that was requested and allowed.

---

14. Step 10 — Zip export created

If export was requested and approved, the system should package the generated repository into its portable output form.

This step should:

- assemble the export bundle
- include required files and records according to policy
- write the zip artifact
- register the export reference

This is the final handoff-forming step of the flow.

---

15. Step 11 — Final build status recorded

At the end of the flow, Build Mode should finalize its status and attach the result to the originating run.

This step should record:

- final build state
- pass/fail/exported result
- output references
- verification summary
- failure reason if unsuccessful
- completion timestamp

This closes the build path in a governed way.

---

16. Failure path behavior

BM-06 should also make clear that the flow can stop early.

If failure occurs at any step, the system should:

- stop at the failing step or phase

- record the failure
- preserve relevant partial records
- avoid pretending the build completed
- block export if completion criteria were not met

So the high-level flow is sequential, but it is also gated.

---

17. Resume/retry implication

Even though BM-06 is high-level, it should leave room for controlled recovery.

Because the flow is broken into named steps, the system can later support:

- rerunning a failed verification step
- restarting from a safe build phase
- resuming after a non-destructive interruption

The high-level flow should therefore be phase-based, not monolithic.

---

18. Time-order summary

A simpler summary of the flow is:

- request
- confirm
- plan
- prepare
- scaffold

- generate
- record
- verify
- evaluate export
- package
- finalize

That is the shortest clean summary of BM-06.

---

19. Why this section matters

This section matters because Build Mode becomes much easier to govern when the sequence is fixed.

Without a defined high-level flow:

- the build agent may improvise order
- verification may happen too late or too early
- packaging may occur before real completion
- recovery becomes harder
- audit becomes less useful

BM-06 gives the system a stable backbone.

---

20. Success definition

BM-06 is successful when Build Mode follows a predictable sequence from operator request to final build result, with clear checkpoints for eligibility, generation, verification, export, and closure.

21. Short version for the doc

BM-06 — High-Level Flow

Build Mode follows a fixed sequence: the build request is accepted, eligibility is confirmed, a concrete build plan is derived, a build workspace is prepared, the repository is scaffolded and generated in controlled slices, build records are written, verification runs, export eligibility is evaluated, the zip is created if requested and allowed, and the final result is attached to the originating run.

BM-07 — Minimum Internal Components

BM-07 — Minimum Internal Components

1. Purpose of this section

Define the smallest internal component set Build Mode needs in order to function as a real governed system.

BM-07 should keep the system lean. The goal is not to create a huge subsystem with unnecessary moving parts. The goal is to identify the minimum components required for Build Mode to reliably turn an approved Agent Kit into a generated, verified, and optionally exported project repository.

---

2. Core rule

Build Mode should have only the components necessary to:

- accept a governed build request
- translate approved inputs into a build path
- materialize the repo
- verify the result
- package the output
- record what happened

If a component does not materially support one of those jobs, it probably does not belong in the minimum set.

---

3. Minimum component set

At minimum, Build Mode should include these internal components:

1. Build Request Handler
2. Build Eligibility Checker

3. Build Planner
4. Build Workspace Manager
5. Repo Generator
6. Build Manifest Writer
7. Build Verifier
8. Export Packager
9. Build Runner / Orchestrator

That is the smallest complete set that still gives you a governed build path.

---

4. Component 1 — Build Request Handler

Purpose

Accept and record the operator's decision to continue from Agent Kit output into Build Mode.

What it does

- receives the operator's selected output mode
- links the request to the correct run
- creates the initial build event/state
- records whether the request is:
    - build repo
    - build repo and export zip

Why it is needed

Build Mode should not start just because a Kit exists.
This component makes build activation explicit and governed.

What it should not do

- it should not decide eligibility
- it should not generate the repo
- it should not bypass policy

It only turns operator intent into an official build request.

---

5. Component 2 — Build Eligibility Checker

Purpose

Determine whether Build Mode is allowed to start.

What it does

- confirms the Agent Kit exists
- confirms required upstream gates passed
- confirms required artifacts are present
- checks for critical unresolved blockers
- checks whether build/export is allowed by policy

Why it is needed

This prevents the generator from trying to compensate for missing or invalid upstream inputs.

What it should not do

- it should not invent missing requirements
- it should not reinterpret upstream truth
- it should not silently downgrade failures into warnings unless policy explicitly allows it

This component is the gatekeeper.

---

6. Component 3 — Build Planner

Purpose

Translate the approved input package into a concrete build path.

What it does

- reads the Agent Kit and locked build inputs
- derives the repo shape
- identifies required files/modules
- determines build slices/phases
- determines generation order
- determines required verification expectations

Why it is needed

The Agent Kit contains build intelligence, but the system still needs a concrete execution plan.

What it should not do

- it should not broaden scope
- it should not create new features
- it should not freestyle architecture choices outside the locked profile

This component turns build intent into build instructions.

---

7. Component 4 — Build Workspace Manager

Purpose

Create and manage the internal directories used during the build.

What it does

- prepares the build workspace
- creates staging/output folders
- establishes paths for repo, manifests, verification, and exports
- keeps intermediate and final outputs organized

Why it is needed

The system needs a safe execution area instead of writing directly into final output with no structure.

What it should not do

- it should not decide what gets generated
- it should not verify correctness
- it should not own build policy

This component manages build space, not build logic.

---

8. Component 5 — Repo Generator

Purpose

Materialize the repository from the approved build plan.

What it does

- creates the repo skeleton
- generates required files and folders
- fills code/config/docs outputs
- wires modules together according to the locked plan
- builds in controlled slices

Why it is needed

This is the component that actually produces the repo.

What it should not do

- it should not guess outside locked scope
- it should not add undeclared major features
- it should not substitute for missing required inputs
- it should not export before verification

This is the heaviest component, but it must remain constrained.

---

9. Component 6 — Build Manifest Writer

Purpose

Record what was built and under what basis.

What it does

- writes the build manifest
- writes the repo manifest or repo summary
- records build status changes
- records source references and output references
- may write file inventories or phase summaries

Why it is needed

A generated repo without records is hard to inspect, audit, resume, or trust.

What it should not do

- it should not determine eligibility

- it should not decide pass/fail on its own
- it should not replace verification outputs

This component preserves traceability.

---

10. Component 7 — Build Verifier

Purpose

Check whether the generated repo satisfies the required build-phase conditions.

What it does

- checks required file presence
- checks repo structure
- scans for unresolved placeholders
- runs required commands/checks
- checks manifest completeness and consistency
- produces pass/fail verification outputs

Why it is needed

A repo should not be treated as successful just because files were generated.

What it should not do

- it should not fix major upstream deficiencies by itself
- it should not allow export when required verification fails
- it should not redefine scope during verification

This component decides whether the build result is valid enough to complete.

---

11. Component 8 — Export Packager

Purpose

Create the final portable zip output when export is requested and allowed.

What it does

- gathers the generated repo and required supporting outputs
- assembles the export bundle according to policy
- writes the zip artifact
- records the export reference

Why it is needed

Portable handoff is one of the main operator-facing outcomes of Build Mode.

What it should not do

- it should not run before verification passes
- it should not replace the repo as the primary build artifact
- it should not hide missing internal records

This component turns a valid build result into a portable package.

---

12. Component 9 — Build Runner / Orchestrator

Purpose

Coordinate the Build Mode flow from start to finish.

What it does

- sequences the components in the correct order
- updates build states across phases

- handles success/failure transitions
- ensures export happens only after eligible completion
- centralizes the high-level execution path

Why it is needed

Without an orchestrator, the components exist but the flow becomes fragile and inconsistent.

What it should not do

- it should not replace the control plane
- it should not become a dumping ground for all business logic
- it should not bypass the other components' boundaries

This component is the conductor, not the whole orchestra.

---

13. Optional components that are not part of the minimum set

These may be useful later, but they are not required for the minimum viable Build Mode:

- diff/comparison engine
- template conflict resolver
- auto-repair engine
- advanced dependency resolver
- multi-target export engine
- deployment adapter
- visual build monitor
- build analytics dashboard

Those can come later if needed.
They are not necessary for the first complete internal version.

---

14. Component relationship summary

A simple relationship model is:

- **Build Request Handler** activates the mode
- **Build Eligibility Checker** decides if it may proceed
- **Build Planner** defines the path
- **Build Workspace Manager** prepares the execution area
- **Repo Generator** produces the repository
- **Build Manifest Writer** records the build artifacts and status
- **Build Verifier** decides if the output is valid
- **Export Packager** creates the zip if permitted
- **Build Runner / Orchestrator** governs the sequence

That is the minimum complete circuit.

---

15. Why this section matters

This section matters because Build Mode can easily become either:

- too thin to be trustworthy, or
- too bloated to stay understandable

BM-07 gives you the smallest component model that is still serious enough to support governed repo generation.

It also helps later when deciding where code should live in the repo, because each component has a clear responsibility.

---

16. Success definition

BM-07 is successful when Build Mode has a minimal but complete internal component set that covers activation, eligibility, planning, workspace handling, repo generation, record writing, verification, export, and orchestration—without unnecessary subsystem bloat.

---

17. Short version for the doc

BM-07 — Minimum Internal Components
Build Mode requires a minimum internal set of nine components: a Build Request Handler, Build Eligibility Checker, Build Planner, Build Workspace Manager, Repo Generator, Build Manifest Writer, Build Verifier, Export Packager, and Build Runner/Orchestrator. Together, these components provide the smallest complete system needed to turn an approved Agent Kit into a generated, verified, and optionally exported project repository.

BM-08 — Build States

BM-08 — Build States

1. Purpose of this section

Define the state model for Build Mode.

BM-08 should describe the official statuses a build can move through from the moment the operator requests it until the build either completes, fails, or is exported. This gives Build Mode a governed lifecycle instead of acting like a loose background process.

---

2. Core rule

Build Mode must have an explicit state model.

A build should never be treated as vaguely “in progress” or “done.”
The system should always be able to answer:

- has the build been requested
- is it allowed to begin
- is it actively generating
- is it being verified
- did it fail
- did it pass
- was it exported

That is the reason the state model exists.

---

3. Minimum state set

At minimum, Build Mode should support these states:

- `not_requested`
- `requested`
- `approved`
- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

This is the smallest clean lifecycle that matches your current design.

---

4. State definitions

4.1 `not_requested`

This is the default state before the operator asks Build Mode to do anything.

Meaning:

- the Agent Kit may exist
- the run may be build-eligible
- but no actual build request has been made

This state is important because Build Mode should not auto-start just because upstream work is complete.

4.2 `requested`

This state means the operator has chosen a build-producing option.

Meaning:

- a build request exists
- the run/build linkage has been recorded
- the system now recognizes an intent to build
- eligibility still needs to be checked or confirmed

This is the first active state in the lifecycle.

4.3 `approved`

This state means the build request has passed its eligibility checks and is allowed to proceed.

Meaning:

- required upstream conditions have been satisfied
- the Agent Kit and required artifacts are present
- build policy allows execution
- the system may now enter active repo generation

This is the “cleared to begin” state.

4.4 `building`

This state means the system is actively generating the repository.

Meaning:

- workspace is prepared or being used
- repo skeleton and contents are being materialized

- manifests may be partially written
- the build is not yet complete or validated

This is the main execution state.

4.5 `verifying`

This state means repo generation has reached the point where checks are now being run.

Meaning:

- required files should largely exist
- repo structure has been created
- the system is evaluating whether the build output satisfies completion rules
- export must remain blocked until this state resolves successfully

This separates code generation from completion judgment.

4.6 `failed`

This state means the build path did not complete successfully.

Meaning:

- the build was blocked or invalid
- a required phase or check failed
- the system stopped rather than silently guessing through the problem
- export is not allowed unless there is some later recovery path and the build re-enters the lifecycle properly

This is the main terminal failure state.

4.7 `passed`

This state means the build successfully completed its required build-phase checks.

Meaning:

- the repo was generated
- required verification passed
- manifests/records are sufficiently complete
- the build is considered successful as a repo-generation event
- export may now occur if it was requested and allowed

This is the successful pre-export completion state.

4.8 `exported`

This state means the build result has been packaged into its final portable output form.

Meaning:

- the build had already passed
- export was requested and permitted
- the zip artifact was successfully produced and recorded

This is the terminal handoff-ready state.

---

5. Recommended transitions

The normal transition path should be:

`not_requested`
→ `requested`

→ `approved`
→ `building`
→ `verifying`
→ `passed`
→ `exported` *(only if export was requested and succeeds)*

That is the happy path.

---

6. Failure transitions

Failure should be possible from multiple points, not only at the end.

For example:

- `requested` → `failed`
  if eligibility fails

- `approved` → `failed`
  if startup/preparation fails

- `building` → `failed`
  if repo generation fails or blocked inputs are encountered

- `verifying` → `failed`
  if required checks do not pass

- `passed` → `failed`
  only if a later export attempt fails and you choose to model export failure as collapsing the overall state instead of storing export failure separately

This makes the lifecycle realistic.

---

7. Happy-path interpretation

A clean reading of the happy path is:

- no one asked for a build yet

- a build was requested
- the request was accepted as valid
- the repo was generated
- the result was checked
- the build passed
- the portable export was created

That is the simplest operator-facing interpretation of the states.

---

8. State boundary rules

Each state should mean exactly one thing and should not be overloaded.

For example:

- `requested` should not also mean “already eligible”
- `building` should not also mean “verification complete”
- `passed` should not automatically mean “exported”

Keeping those boundaries clean prevents confusion later in the control plane and UI.

---

9. Terminal vs non-terminal states

It helps to divide the states into two groups.

Non-terminal states

- `not_requested`
- `requested`

- `approved`
- `building`
- `verifying`

These mean the build is still open or may still progress.

Terminal states

- `failed`
- `passed`
- `exported`

These mean the build path has reached an end condition for that attempt.

This distinction will help later with retry and resume rules.

---

10. State and export relationship

`passed` and `exported` should remain separate.

That matters because:

- a build can pass without an export being requested
- a build can pass before export packaging begins
- export is a downstream handoff step, not proof that the repo itself was valid

So do not collapse these into one state.

---

11. State and retry relationship

The state model should later support controlled retries or resumes, but BM-08 does not need to fully define those mechanics yet.

It only needs to stay compatible with ideas like:

- retry after `failed`
- rerun verification from `verifying`
- export later after a build is already `passed`

That means the states should be stable enough to support recovery without being ambiguous.

---

12. Optional future state refinements

For the first version, the minimum state set is enough.

Later, you may choose to add more granular states like:

- `planning`
- `preparing_workspace`
- `packaging`
- `blocked`
- `cancelled`
- `export_failed`

But those are not required for the minimum outline.
The simpler model is better for now.

---

13. Why this section matters

Without explicit states:

- the control plane cannot govern Build Mode cleanly
- the UI cannot explain what is happening
- retries/resumes become messy
- packaging can happen at the wrong time
- audit records become vague

BM-08 turns Build Mode into a proper lifecycle instead of a one-off job.

---

14. Success definition

BM-08 is successful when Build Mode has a clear lifecycle with distinct states for request, approval, active generation, verification, failure, successful completion, and export, and when those states support traceable transitions without ambiguity.

---

15. Short version for the doc

BM-08 — Build States
Build Mode follows an explicit state lifecycle: `not_requested`, `requested`, `approved`, `building`, `verifying`, `failed`, `passed`, and `exported`. These states distinguish operator intent, eligibility, active repo generation, verification, completion, and portable handoff so the build remains governed, traceable, and compatible with retry or export workflows.

BM-09 — Entry Conditions

BM-09 — Entry Conditions

1. Purpose of this section

Define what must be true before Build Mode is allowed to start.

BM-09 should make clear that Build Mode is not triggered simply because an Agent Kit exists. It only begins when the required upstream build basis is present, the build is eligible under policy, and the operator has explicitly asked the system to continue into repo generation.

---

2. Core rule

Build Mode may only start from an approved and build-ready state.

That means all required prerequisites must be satisfied before repo generation begins. If a required input, approval, or eligibility condition is missing, Build Mode should not attempt to compensate by guessing or silently proceeding.

---

3. Main entry condition

The main entry condition is:

An approved Agent Kit exists, required upstream conditions are satisfied, and the operator has requested a build-producing outcome.

Everything else in this section supports that rule.

---

4. Required entry conditions

At minimum, Build Mode should require all of the following before it can start.

4.1 Approved Agent Kit exists

The run must have a valid approved Agent Kit.

This means:

- the Kit has been generated
- it is the active approved build basis for the run
- it has not been superseded or invalidated
- it is accessible to Build Mode as the authoritative build-intelligence package

Without this, there is no legitimate build source.

4.2 Required upstream gates have passed

The run must have satisfied the upstream gating required for build eligibility.

This means the build is not starting from a half-finished or unapproved upstream state.

Depending on your wider Axion rules, this can include confirmation that:

- required intake/canonical/planning outputs are complete enough
- standards resolution completed
- required templates/specs/contracts are selected and present
- upstream verification or coverage thresholds passed

Build Mode should not be the place where those unfinished steps get patched over.

4.3 Required build inputs are present

The minimum required input set for the declared scope must exist.

This usually includes:

- approved Agent Kit
- run metadata
- locked standards snapshot or equivalent

- locked build profile
- required templates/specs/contracts
- build policy context
- any other declared required input for the chosen build profile

If the inputs required by the build profile are not present, Build Mode should not begin.

4.4 Operator build request exists

The operator must explicitly choose a build-producing outcome.

That choice should be one of:

- Build repo
- Build repo and export zip

This condition is important because Build Mode should not assume the operator wants repo generation simply because it is possible.

4.5 Build policy allows execution

The current run, profile, and policy state must allow Build Mode to proceed.

This may include checks such as:

- build is permitted for this run type
- required approvals exist
- unresolved critical blockers are not present
- export is allowed only if requested and policy permits it
- no hard stop rule currently blocks repo generation

This keeps Build Mode under system governance.

4.6 Critical unresolved blockers are absent or explicitly tolerated

Any critical ambiguity that would force the build agent to guess should either:

- be resolved before build start, or
- be explicitly allowed by policy if the system supports such exceptions

Examples include:

- missing required specs
- direct conflicts between locked inputs
- missing repo blueprint where the profile requires one
- blocked or incomplete file target inventories
- incompatible build profile selections

If critical blockers remain unresolved and are not explicitly tolerated, Build Mode should not enter active generation.

---

5. Recommended entry conditions

Beyond the required minimum, Build Mode should also prefer the following conditions when available.

5.1 Build plan can be derived cleanly

The system should be able to derive a concrete build path from the available inputs.

If the inputs exist but are too vague to yield a stable build plan, that should be treated as a warning or failure depending on policy.

5.2 Repo blueprint or file tree target is available

If your build profile depends on a locked repo structure, that structure should be present before the build starts.

This is not always universally required, but it is highly recommended if your goal is minimal guessing.

5.3 Verification command set is known

The build should know ahead of time what checks it will be expected to pass.

That prevents the system from generating first and trying to invent completion rules afterward.

5.4 Required file inventory is known

If the profile expects explicit file targets, those should be known before generation begins.

This further reduces drift during build execution.

---

6. Entry condition hierarchy

A useful way to think about BM-09 is in layers.

Layer 1 — Authority

- approved run context exists
- approved Agent Kit exists
- operator build request exists

Layer 2 — Eligibility

- required gates passed
- policy allows build
- no hard stop blockers remain

Layer 3 — Build readiness

- required inputs are present

- build profile is locked
- standards/specs/templates/contracts are available
- build plan can be derived

Only when all three layers are satisfied should the build begin.

---

7. What does not count as satisfying entry conditions

Build Mode should not treat the following as valid substitutes for entry readiness:

- agent confidence
- likely assumptions
- partial artifact presence
- implied operator intent
- unapproved draft Kit content
- stale superseded inputs
- “we can probably figure it out during generation”

Those are exactly the kinds of shortcuts that cause drift.

---

8. Entry failure behavior

If entry conditions are not satisfied, Build Mode should fail early.

That means it should:

- not enter `building`
- not partially start repo generation

- record which entry condition failed
- record whether the failure is policy-based, input-based, or blocker-based
- remain traceable in the run history

This fail-fast behavior is part of keeping the build trustworthy.

---

9. Relationship to build states

BM-09 maps closely to the early states in BM-08.

A practical interpretation is:

- `not_requested` means no build request yet
- `requested` means an operator request exists
- entry checks run here
- `approved` is only reached after entry conditions pass

So the entry conditions are what control the transition from:
`requested` → `approved`

That is the key lifecycle connection.

---

10. Why this section matters

This section matters because build failures often start before the build agent writes a single file.

If the system enters Build Mode with:

- missing inputs
- unresolved contradictions
- incomplete scope definition

- weak eligibility checks

then the generator is forced to compensate, and that is where guessing starts.

BM-09 protects Build Mode by making build start conditional on real readiness.

---

11. Success definition

BM-09 is successful when Build Mode only begins after an approved Agent Kit, explicit operator request, required upstream gates, required build inputs, and policy eligibility are all in place, and when the system blocks generation rather than guessing through missing readiness conditions.

---

12. Short version for the doc

BM-09 — Entry Conditions
Build Mode may begin only when an approved Agent Kit exists, required upstream gates have passed, required build inputs are present, the operator has explicitly requested a build-producing outcome, policy allows execution, and no unresolved critical blockers remain. These entry conditions protect the build from starting in a state that would force guessing or silent compensation.

BM-10 — Exit Conditions

BM-10 — Exit Conditions

1. Purpose of this section

Define what must be true before Build Mode can be considered complete.

BM-10 should make clear that a build does not “finish” just because code was generated. Build Mode only exits successfully when the required repository outputs, records, and checks are complete enough to satisfy the mode’s completion standard.

---

2. Core rule

Build Mode may only exit successfully from a valid completed state.

That means the system must not treat partial repo generation, unfinished verification, or incomplete packaging as successful completion. A successful exit requires that the defined completion conditions have been met for the requested build outcome.

---

3. Main exit condition

The main exit condition is:

The requested build outcome has been fully produced, required build verification has passed, required records have been written, and the final result has been attached to the originating run.

Everything else in this section supports that rule.

---

4. Successful exit conditions

At minimum, Build Mode should require all of the following for a successful non-export or export-capable completion.

4.1 Repository generation is complete

The system must have finished materializing the declared repository output for the approved build scope.

This means:

- the required repo structure exists
- the required files for the declared scope have been created
- the build did not stop mid-materialization
- the generated output is no longer in an incomplete active-build state

A build cannot exit successfully if the repo is only partially generated.

4.2 Required build records exist

The required build records must be written.

At minimum this should include:

- build manifest
- repo manifest or equivalent repo summary
- build status record
- references to source inputs and produced outputs

Without these records, the output is harder to audit, verify, and hand off.

4.3 Required build-phase verification has passed

The generated repo must pass the required build-phase checks.

This means the verification stage has completed and the required pass conditions were satisfied.

Depending on the profile, this may include:

- required file presence checks
- repo structure checks

- placeholder scans
- consistency checks
- manifest completeness checks
- defined command/check execution results

If required verification fails, Build Mode must not exit as successful.

4.4 Final build status is recorded

The build must have a finalized state and closure record.

This means the system has recorded:

- final build state
- completion timestamp
- pass/fail result
- relevant output references
- failure reason if unsuccessful

The build should not be left in an ambiguous transitional state.

4.5 Output is attached to the originating run

The final build result must remain connected to the same Axion run that produced the Agent Kit.

This means the run should have references to:

- the generated repo
- the build records
- the verification outputs

- the export artifact if one was created

This preserves continuity and chain of custody.

---

5. Export-specific exit conditions

If the operator requested zip export, successful exit requires additional conditions beyond repo completion.

5.1 Export was requested

The run must have an active request for the export-producing outcome.

A zip should not be treated as required if the operator only requested repo generation.

5.2 Export eligibility passed

The system must confirm that export is allowed.

This means:

- the build already passed required verification
- required records exist
- policy allows export
- no export-blocking rule is active

5.3 Zip artifact was successfully created

The export package must actually exist and be recorded.

That means:

- the archive was written successfully
- the export reference was stored

- the artifact is associated with the correct run/build

If export was requested but the zip was not created successfully, the build should not exit as `exported`.

---

6. Exit conditions by requested outcome

BM-10 should distinguish completion based on what the operator asked for.

Outcome A — Build repo only

Successful exit requires:

- repo generation complete
- required records written
- required verification passed
- final build state recorded
- outputs attached to the run

This should end in `passed`.

Outcome B — Build repo and export zip

Successful exit requires:

- everything required for Build repo only
- plus export eligibility passed
- plus zip artifact successfully created and recorded

This should end in `exported`.

This distinction is important because not every successful build needs a zip.

---

7. Unsuccessful exit conditions

Build Mode should also define when it exits unsuccessfully.

An unsuccessful exit occurs when:

- repo generation fails
- required build inputs become blocked during execution
- required verification fails
- required records cannot be completed
- export was required for the requested outcome but export creation fails

In those cases, the build should exit into `failed`, not a partial success label.

---

8. Partial output does not equal successful exit

BM-10 should explicitly state that partial output is not enough.

For example, these do not count as successful completion by themselves:

- some source files exist
- the repo skeleton exists
- manifests were partially written
- most checks passed
- the zip was attempted
- the generator “basically finished”

Build Mode should only exit successfully when the required completion threshold has actually been met.

---

9. Exit condition hierarchy

A useful way to think about successful exit is in layers.

Layer 1 — Build output completeness

- repo generation complete

Layer 2 — Build proof completeness

- records written
- verification passed

Layer 3 — Run closure completeness

- final status recorded
- outputs attached to run

Layer 4 — Export completeness, if requested

- export eligibility passed
- zip artifact created and recorded

Only when the relevant layers are satisfied should the build exit successfully.

---

10. Relationship to build states

BM-10 maps directly to the end of the BM-08 lifecycle.

A practical interpretation is:

- `verifying` resolves into either `failed` or `passed`
- `passed` resolves into `exported` only if export was requested and completed successfully

So exit conditions determine the final state transition at the end of the build lifecycle.

---

11. Exit failure behavior

If exit conditions are not met, the system should:

- not mark the build successful
- not allow export to imply success if verification failed
- record which exit condition was not satisfied
- preserve relevant outputs and failure records
- keep the final result traceable

This protects the integrity of the build result.

---

12. Why this section matters

This section matters because without explicit exit conditions, Build Mode can be declared “done” too early.

That leads to problems like:

- incomplete repos being treated as valid
- export artifacts being mistaken for successful builds
- missing records breaking auditability
- verification being bypassed in practice

BM-10 prevents that by making completion governed instead of assumed.

---

13. Success definition

BM-10 is successful when Build Mode only exits in a successful state after the generated repository is complete, required build records exist, required verification has passed, the final result is recorded against the originating run, and the zip export exists when that export outcome was requested.

---

14. Short version for the doc

BM-10 — Exit Conditions
Build Mode exits successfully only when the requested build outcome has been fully completed: the repository has been generated, required build records have been written, required verification has passed, the final status has been recorded, and the outputs are attached to the originating run. If export was requested, the zip must also be successfully created and recorded before the build may exit as `exported`.

BM-11 — Operator Controls

BM-11 — Operator Controls

1. Purpose of this section

Define what the operator is allowed to control in Build Mode.

BM-11 should make clear that Build Mode is not a hidden automatic process. The operator needs explicit control over whether the system stops at the Agent Kit, proceeds into repo generation, or continues all the way to export. This section also defines the minimum build-facing controls the operator should have without turning Build Mode into a manual coding workflow.

---

2. Core rule

The operator should control **whether Build Mode starts, what output level is requested, and how the build result is reviewed**, but should not manually override the locked build path in ways that reintroduce drift.

So BM-11 is about controlled operator authority, not freeform operator intervention.

---

3. Main operator control

The main operator control is the post-Kit output choice.

After the Agent Kit is complete, the operator should be able to choose one of these outcomes:

- Kit only
- Build repo
- Build repo and export zip

This is the most important control because it determines whether Build Mode is activated at all.

---

4. Required operator controls

At minimum, Build Mode should provide the following controls.

4.1 Build activation control

The operator should be able to explicitly start Build Mode.

This control should:

- attach the request to the current run
- record the requested output mode
- move the build into its requested lifecycle state

Build Mode should not assume consent just because the Kit exists.

4.2 Output mode selection

The operator should be able to choose the requested output level.

At minimum:

- Build repo
- Build repo and export zip

This is important because export is a stronger downstream action than repo generation alone.

4.3 Build status visibility

The operator should be able to see the current build state.

At minimum, the operator should be able to see whether the build is:

- requested
- approved
- building
- verifying

- failed
- passed
- exported

This keeps the process understandable and traceable from the operator side.

4.4 Failure visibility

If the build fails, the operator should be able to see why.

At minimum, the system should surface:

- failed phase or state
- failure reason
- blocking condition
- whether the failure was due to inputs, policy, generation, verification, or export

The operator should not be left with a vague “build failed” message.

4.5 Verification result visibility

The operator should be able to review whether the repo passed the required checks.

At minimum, the operator should be able to see:

- pass/fail result
- key failed checks if any
- whether export is now allowed
- where the verification outputs are attached in the run

This matters because successful generation alone should not be mistaken for validated completion.

4.6 Export/download access

If export succeeds, the operator should be able to access the final zip artifact.

This is one of the core practical outcomes of the whole mode.

---

5. Recommended operator controls

These are not strictly required for the minimum version, but they fit well later.

5.1 Build summary view

The operator should be able to review a compact summary of:

- build inputs used
- build profile used
- output produced
- final state
- verification result
- export result

5.2 Retry or rerun trigger

If your lifecycle later supports it, the operator may be allowed to:

- retry after failure
- rerun export after a passed build
- rerun a safe verification step

This should still remain governed by policy.

5.3 Output artifact browsing

The operator may be allowed to inspect:

- build manifest
- repo manifest
- verification report
- failure report
- export record

5.4 Build history for the run

The operator may be allowed to see prior build attempts tied to the same run.

This is useful if a build was retried or rerun.

---

6. What the operator should not control directly

BM-11 should clearly limit operator control so the build stays locked.

The operator should not directly control:

- ad hoc file generation order during the build
- freeform architecture changes mid-build
- manual scope expansion during generation
- hidden bypass of required gates
- silent override of no-guess rules
- export before verification passes

Those controls belong to the system design and policy layers, not the live operator surface.

---

7. Operator control boundary

A good way to define the boundary is:

The operator may control:

- whether to enter Build Mode
- whether export is requested
- whether to inspect build outputs
- whether to retry or resume where policy allows

The operator may not control:

- the locked build truth
- the active standards snapshot
- the allowed scope during generation
- pass/fail semantics for required checks
- whether a blocked build silently continues

This keeps the system practical without making it loose.

---

8. Relationship to approvals and policy

Operator controls should still operate under policy.

That means the operator may request a build, but the system still decides whether the request is allowed.

Likewise, the operator may request export, but export still depends on:

- verification passing
- records being complete

- policy allowing export

So BM-11 should make clear that operator controls are **requests within governance**, not unchecked commands.

---

9. Relationship to drift prevention

This section matters for drift because too much operator freedom during generation can create the same problems as too much agent freedom.

If the operator can alter core build rules midstream, the build stops being reproducible and traceable.

So the system should let the operator:

- initiate
- observe
- review
- retrieve

but not casually rewrite the governed path while it is running.

---

10. Minimum operator experience

At the simplest level, the operator experience should allow this flow:

1. Agent Kit completes
2. operator chooses Build repo or Build repo and export zip
3. operator sees the build progress through states
4. operator sees pass/fail outcome
5. operator reviews failure or verification details if needed

6. operator downloads the zip if export succeeds

That is the minimum clean operator-facing loop.

---

11. Why this section matters

Without explicit operator controls:

- Build Mode can feel automatic and opaque
- users may not know when they are triggering code generation
- export may feel detached from the run
- failure handling becomes frustrating
- the line between system governance and user intent gets blurry

BM-11 makes Build Mode usable without making it uncontrolled.

---

12. Success definition

BM-11 is successful when the operator can explicitly trigger Build Mode, choose the requested output level, monitor build status, review failures and verification results, and access the final export artifact when available—without being able to bypass the locked build path or required policy checks.

---

13. Short version for the doc

BM-11 — Operator Controls
Build Mode gives the operator explicit control over whether to stop at the Agent Kit, generate the repo, or generate and export the repo as a zip. The operator must also be able to view build status, review failures and verification results, and access the final export artifact. These controls operate within policy and do not allow the operator to bypass the locked build path or required checks.

BM-12 — Build Outputs Contract

BM-12 — Build Outputs Contract

1. Purpose of this section

Define the minimum required output contract for a successful Build Mode run.

BM-12 should make clear exactly what artifacts must exist when Build Mode finishes successfully. This section is not just describing outputs in general. It is defining the formal output set that the system treats as the required deliverable of the build.

---

2. Core rule

A successful Build Mode run must produce a complete and inspectable output package, not just generated source files.

That means the system should treat success as the production of a governed artifact set with:

- the generated repository
- the records that describe it
- the verification evidence that validates it
- the portable export when that export outcome was requested

If the required output contract is not satisfied, the build should not be considered fully complete.

---

3. Main contract principle

The Build Outputs Contract should answer one question clearly:

What exact artifacts must exist for Build Mode to count as successfully completed for this run?

This makes build completion measurable and enforceable.

---

4. Minimum required output artifacts

At minimum, the Build Outputs Contract should require the following artifacts.

4.1 Generated repository

A materialized project repository must exist.

This is the core output and should include the generated repo structure and files for the declared scope.

Recommended logical path:

- `/repo/`

This is the primary build artifact.

4.2 Build manifest

A build manifest must exist.

This artifact records the build event and ties the output back to the originating run and build basis.

Recommended logical path:

- `build_manifest.json`

At minimum it should identify:

- run/build reference
- build profile
- source Agent Kit reference
- build timestamps
- build status
- output references

4.3 Repo manifest

A repo manifest or equivalent repo summary must exist.

This artifact describes the generated repository as an output object.

Recommended logical path:

- `repo_manifest.json`

At minimum it should summarize:

- repo structure
- module/file coverage
- expected commands/checks
- output composition

4.4 Verification report

A verification report must exist.

This artifact shows whether the generated repo passed the required build-phase checks.

Recommended logical path:

- `verification_report.json`

At minimum it should include:

- pass/fail status
- checks executed
- failed checks if any
- verification timestamp

- references to detailed command/check results where applicable

---

5. Export-required artifact

If the operator requested export and policy allowed it, the Build Outputs Contract should also require the export artifact.

5.1 Portable zip export

A zip archive of the generated repository and required supporting outputs must exist.

Recommended logical path:

- `project_repo.zip`

This artifact is only required when the requested outcome includes export.

A build that requested export should not count as fully complete if the zip artifact does not exist.

---

6. Recommended supporting artifacts

These are not necessarily part of the smallest strict minimum, but they are strongly recommended and may be folded into the contract later.

6.1 Build plan artifact

Recommended path:

- `build_plan.json`

This helps show the execution path derived from the Agent Kit.

6.2 File index

Recommended path:

- `file_index.json`

This helps inspect what was generated and where.

6.3 Command results

Recommended path:

- `command_results.json`

This preserves detailed verification outputs.

6.4 Failure report

Recommended path when unsuccessful:

- `failure_report.json`

This is important for failed runs even though it is not a success artifact.

---

7. Contract by requested outcome

BM-12 should define the output contract differently depending on what the operator requested.

Outcome A — Build repo only

Required artifacts:

- `/repo/`
- `build_manifest.json`
- `repo_manifest.json`
- `verification_report.json`

Completion state:

- `passed`

Outcome B — Build repo and export zip

Required artifacts:

- `/repo/`
- `build_manifest.json`
- `repo_manifest.json`
- `verification_report.json`
- `project_repo.zip`

Completion state:

- `exported`

This distinction matters because export is not required for every successful build.

---

8. Contract completeness rule

The output contract is only satisfied when:

- all required artifacts for the selected outcome exist
- those artifacts are associated with the correct run/build
- verification indicates required pass conditions were met
- the final build state is consistent with the artifact set

So Build Mode should not mark completion if, for example:

- the repo exists but no verification report exists
- the verification report exists but the repo is incomplete
- the zip exists but the build never passed verification

- the manifests are missing or detached from the run

---

9. Artifact integrity expectation

Each required output artifact should be:

- present
- identifiable
- linked to the originating run/build
- structurally valid
- internally consistent with the build result

That means the contract is not only about file presence.
It is also about artifact validity and traceability.

---

10. Output contract boundary

BM-12 should clarify that the output contract does not require:

- production deployment artifacts
- live environment URLs
- external hosting status
- downstream release approvals
- post-build manual edits

The contract is specifically about what Build Mode must produce as its own successful build result.

---

11. Relationship to exit conditions

BM-12 is closely tied to BM-10.

A useful way to think about it is:

- BM-10 defines when the build may exit successfully
- BM-12 defines the artifact set that must exist for that success to be true

So the Build Outputs Contract is one of the concrete enforcement tools behind successful exit.

---

12. Relationship to verification

Verification is not outside the contract. It is part of the contract.

That means a Build Mode output is incomplete if it lacks the artifact that proves the repo was checked. This is important because it prevents the system from treating “generated repo folder exists” as equivalent to “successful build.”

---

13. Why this section matters

Without a formal outputs contract:

- success becomes subjective
- different runs may produce different artifact sets
- export may happen without sufficient records
- audit and review become inconsistent
- retry/resume logic becomes harder to reason about

BM-12 turns the build result into a defined artifact package instead of a loose collection of files.

---

14. Success definition

BM-12 is successful when Build Mode has a clear minimum artifact contract for each requested outcome, and when successful completion requires the presence of the generated repo, required manifests, verification report, and zip export when export was requested.

---

15. Short version for the doc

BM-12 — Build Outputs Contract

A successful Build Mode run must satisfy a formal output contract. At minimum, it must produce the generated repository, a build manifest, a repo manifest, and a verification report. If the requested outcome includes export, it must also produce the final zip archive. Build completion is valid only when the required artifact set for the selected outcome exists, is traceable to the originating run, and is consistent with a successful verification result.

BM-13 — Verification Scope

BM-13 — Verification Scope

1. Purpose of this section

Define what Build Mode verification is responsible for checking before a generated repo can be treated as a valid build result.

BM-13 should make clear that verification is not a vague “looks good” step. It is the formal build-phase check layer that determines whether the generated repository is complete enough, consistent enough, and policy-compliant enough to count as passed and, if requested, to move on to export.

---

2. Core rule

Build Mode verification must check whether the generated output satisfies the required build-phase completion conditions.

That means verification should answer:

- was the required repo actually generated
- does the output match the expected build structure
- are required files and records present
- do unresolved critical placeholders remain
- do required checks/commands pass
- is the build safe to mark as complete

Verification is not there to “help the build feel finished.”
It is there to decide whether the build result is valid.

---

3. Main scope principle

The verification scope should be **strict enough to block incomplete or drifting builds**, but narrow enough to stay focused on Build Mode’s own responsibility.

So BM-13 should focus on:

- build-phase structural correctness
- output completeness
- generated artifact validity
- required command/check results
- export readiness

It should not expand into every possible downstream QA or release-certification concern unless you explicitly decide to include those elsewhere.

---

4. Minimum verification categories

At minimum, Build Mode verification should cover these categories:

1. Required output presence
2. Repo structure correctness
3. Placeholder / unresolved output checks
4. Manifest and record completeness
5. Required command/check execution
6. Output consistency and export readiness

That is the minimum serious scope for Build Mode.

---

5. Verification category 1 — Required output presence

Purpose

Confirm that the required build outputs actually exist.

What it should check

- generated repo exists
- required directories exist
- required files exist
- required manifests exist
- verification artifact targets exist

Why it matters

A build cannot be valid if key outputs were never produced.

This is the most basic but necessary layer of verification.

---

6. Verification category 2 — Repo structure correctness

Purpose

Confirm that the generated repo matches the expected structural pattern for the locked build profile.

What it should check

- root structure matches the target repo shape
- required folder layout exists
- modules/domains are placed correctly
- config/test/docs/shared paths exist where required
- file placement is consistent with the declared blueprint or profile

Why it matters

A repo can contain many files and still be structurally wrong.
This check prevents “it generated something” from being treated as “it generated the right repo shape.”

---

7. Verification category 3 — Placeholder / unresolved output checks

Purpose

Confirm that the generated output does not still contain unresolved critical gaps.

What it should check

- unresolved placeholders
- TODO markers that violate build policy
- missing required values that should have been materialized
- unresolved critical template tokens
- unfinished scaffold markers where completion was required

Why it matters

This is one of the most important anti-drift checks.

A repo that still contains unresolved critical build placeholders should not be treated as passed, because that usually means the system generated around a gap instead of truly satisfying the build basis.

---

8. Verification category 4 — Manifest and record completeness

Purpose

Confirm that the non-code build records exist and are sufficiently complete.

What it should check

- build manifest exists and is populated
- repo manifest exists and is populated
- output references are recorded
- run/build linkage is present

- verification report structure is valid
- required build metadata is not missing

Why it matters

A Build Mode result is not just code.
It is a governed output package.
This category protects traceability and auditability.

---

9. Verification category 5 — Required command/check execution

Purpose

Confirm that the generated repo satisfies the required executable or static checks defined by the build profile.

What it should check

Depending on the stack/profile, this may include:

- install or dependency resolution check
- type check
- lint check
- test check
- compile/build check
- schema validation check
- file integrity or consistency check

Why it matters

Structural presence alone is not enough.
A repo may look complete but still fail basic validity checks.

This category is where the build result proves it is more than a folder of generated files.

---

10. Verification category 6 — Output consistency and export readiness

Purpose

Confirm that the build result is coherent enough to be finalized and, if requested, exported.

What it should check

- final output artifact set matches the selected outcome
- repo/manifests/verification records agree with one another
- export preconditions are satisfied
- no blocking inconsistency exists between generated outputs and recorded outputs
- the build result is eligible to transition to `passed` or `exported`

Why it matters

This final category connects raw verification to lifecycle completion.

---

11. What Build Mode verification does not have to cover

BM-13 should clearly state its limits.

Build Mode verification does not necessarily need to cover:

- full business correctness of every feature
- production security certification
- live deployment health
- performance under real user traffic
- long-term maintainability scoring
- user acceptance testing outside declared build scope

Those may matter elsewhere in Axion, but they are not automatically part of Build Mode unless you explicitly expand the mode later.

---

12. Required vs profile-dependent checks

BM-13 should separate:

- universally required verification checks
  from
- profile-dependent verification checks

Universally required

These should almost always exist:

- required output presence
- repo structure correctness
- manifest completeness
- placeholder/unresolved gap scanning

Profile-dependent

These depend on the stack and build profile:

- lint
- tests
- compile/build
- package install checks
- framework-specific validation
- monorepo workspace checks

This keeps the verification model consistent without making every build identical.

---

13. Verification failure rule

If a required verification category fails, the build should not pass.

That means:

- the build must not move to `passed`
- export must not proceed
- failure must be recorded
- the failed check(s) must be surfaced in the build outputs

Verification should be an actual gate, not a courtesy report.

---

14. Verification evidence expectation

Verification should produce structured evidence, not just a status label.

At minimum, verification evidence should show:

- what was checked
- when it was checked
- what passed
- what failed
- whether the failure is blocking
- what final verification result was reached

This makes the step inspectable and reusable for later recovery or audit.

---

15. Relationship to build states

BM-13 maps directly to the `verifying` state in BM-08.

A practical interpretation is:

- generation finishes enough to enter `verifying`
- verification runs against the generated outputs
- success leads to `passed`
- failure leads to `failed`

So BM-13 defines the actual meaning of the `verifying` phase.

---

16. Relationship to exit and export

BM-13 also supports BM-10 and BM-12.

- BM-10 uses verification to decide whether the build may exit successfully
- BM-12 requires the verification report as part of the build outputs contract
- export should only happen after required verification passes

So verification is one of the main gates between generation and handoff.

---

17. Why this section matters

Without a defined verification scope:

- different builds may be judged by different standards
- export may happen too early
- incomplete repos may look successful
- placeholder-heavy output may slip through

- the build lifecycle loses integrity

BM-13 makes pass/fail real.

---

18. Success definition

BM-13 is successful when Build Mode has a clear and enforceable verification scope that checks output presence, repo structure, unresolved critical gaps, manifest completeness, required command/check results, and export readiness before a generated repo may be treated as passed or exported.

---

19. Short version for the doc

BM-13 — Verification Scope
Build Mode verification checks whether the generated repository satisfies the required build-phase completion conditions. At minimum, it covers required output presence, repo structure correctness, unresolved placeholder scanning, manifest and record completeness, required command/check execution, and final output consistency for completion and export readiness. If a required verification category fails, the build must not pass or export.

BM-14 — Packaging Rule

BM-14 — Packaging Rule

1. Purpose of this section

Define how packaging is treated inside Build Mode and when it is allowed to happen.

BM-14 should make clear that packaging is a governed downstream step of a successful build, not a shortcut around build completion. This section exists to prevent the system from treating “zip created” as equivalent to “build succeeded.”

---

2. Core rule

Packaging may only occur after the build has satisfied its required completion and verification conditions.

That means Build Mode must not package:

- incomplete repos
- unverified repos
- failed builds
- outputs missing required records
- outputs blocked by policy

Packaging is not proof of validity.
It is the portable handoff form of an already valid build result.

---

3. Main packaging principle

The main principle of BM-14 is:

A repository must be eligible before it is packageable.

So packaging should always be downstream of:

- repo generation
- verification
- output contract satisfaction
- policy approval for export

This keeps packaging aligned with the rest of the Build Mode lifecycle.

---

4. What packaging means in Build Mode

Within Build Mode, packaging means taking the successful build result and assembling it into a portable artifact that the operator can retrieve and open elsewhere.

In the initial version, packaging should primarily mean:

- creating a zip archive of the generated project repository
- including required supporting artifacts as defined by policy

Packaging is the handoff step, not the generation step.

---

5. Packaging trigger rule

Packaging should only happen if all of the following are true:

- the operator requested export
- the build reached a successful completion state
- required verification passed
- required build records exist
- export policy allows packaging

If one of those conditions is missing, packaging should not run.

---

6. Packaging eligibility conditions

BM-14 should define explicit packaging eligibility conditions.

At minimum, the build should be packageable only when:

6.1 The build has passed

The repo generation and verification path must have completed successfully.

In practice, this means the build should already be in or eligible for the `passed` state before packaging begins.

6.2 Required output artifacts exist

The required non-export output contract must already be satisfied.

At minimum, that means:

- generated repo exists
- build manifest exists
- repo manifest exists
- verification report exists

Packaging should not be used to cover for missing internal outputs.

6.3 No blocking policy rule is active

Packaging must be allowed under the current policy state.

Examples of packaging blockers may include:

- export disallowed for this run type
- unresolved critical compliance conditions

- missing required approvals
- hard stop rules tied to verification outcomes

6.4 Requested outcome includes export

The system should not create a zip by default just because it can.

Packaging should reflect the operator's selected outcome, not system assumption.

---

7. Packaging contents rule

BM-14 should define what packaging is allowed to include.

At minimum, the package should include:

- the generated repository

Depending on policy, it may also include:

- build manifest
- repo manifest
- verification report
- selected supporting records needed for inspection or handoff

The exact contents can be profile- or policy-dependent, but the rule should remain:
the package contents must be intentional and governed.

---

8. Packaging must not replace internal outputs

Even when the zip is created, the internal build outputs should still remain in the run workspace.

That means the system should preserve:

- the generated repo
- manifests
- verification records
- build records
- export record

The zip is a portable delivery artifact.
It is not the only copy of the truth.

---

9. Packaging failure rule

If packaging fails, the system should not pretend export succeeded.

That means:

- the build should not move to `exported`
- the failure should be recorded
- the packaging error should be surfaced clearly
- the already-passed repo build may still remain valid as a `passed` build result if your lifecycle distinguishes build success from export success

This distinction is important because repo validity and zip creation are related but not identical.

---

10. Packaging and state relationship

BM-14 should stay aligned with the Build Mode state model.

A clean state interpretation is:

- build reaches `passed`

- packaging begins only after `passed`
- successful packaging leads to `exported`

So packaging is the transition layer between:
`passed` → `exported`

That is why packaging must remain downstream.

---

11. Packaging and verification relationship

Packaging must never outrun verification.

That means:

- no zip before required checks complete
- no zip when required checks fail
- no zip when unresolved critical build gaps remain
- no zip when the verification report is missing or invalid

This keeps the handoff artifact trustworthy.

---

12. Packaging and operator trust

The packaging rule matters because the operator is likely to treat the zip as “the finished thing.”

If the system creates zips too early or too loosely, it trains the operator to trust artifacts that were never actually validated.

So BM-14 should make packaging a high-trust action:
when the zip exists, it should mean the repo already cleared the required build bar.

---

13. Minimum packaging policy

A clean minimum packaging policy for the first version is:

- package only on explicit request
- package only after `passed`
- include the generated repo
- include required supporting records if policy says they belong in the export
- record the export artifact
- fail loudly if packaging cannot be completed

That is enough for the initial internal version.

---

14. What packaging is not

BM-14 should also state what packaging is not.

Packaging is not:

- repo generation
- verification
- proof that the app is deployed
- permission to skip required checks
- a substitute for missing build records
- a way to turn a failed build into a deliverable

It is only the final portable form of a valid build result.

---

15. Why this section matters

Without a strong packaging rule:

- export may happen too early
- failed or incomplete builds may still get handed off
- operators may trust invalid repo bundles
- the line between “generated” and “validated” becomes blurry

BM-14 protects the meaning of the export artifact.

---

16. Success definition

BM-14 is successful when packaging only occurs after successful repo generation and required verification, when the zip contents follow defined rules, when packaging does not replace internal traceability, and when export status is only granted after the package is actually created and recorded.

---

17. Short version for the doc

BM-14 — Packaging Rule
Packaging is the portable handoff step of Build Mode, not a shortcut around build completion. A repo may be packaged only when export was requested, the build has passed required verification, the required output artifacts exist, and policy allows export. The zip is a downstream delivery artifact that must not replace internal build records or imply success when packaging itself fails.

BM-15 — Audit / Proof Requirements

BM-15 — Audit / Proof Requirements

1. Purpose of this section

Define what Build Mode must record so the build can be traced, explained, and trusted after it runs.

BM-15 should make clear that repo generation is not enough by itself. Because Build Mode is producing a governed code artifact, the system also needs proof of what inputs were used, what happened during execution, what checks passed or failed, and what outputs were produced.

---

2. Core rule

Every Build Mode run must leave an auditable proof trail.

That means the system should be able to answer, after the fact:

- what run this build belonged to
- what Agent Kit and locked inputs were used
- what was requested
- what states and phases occurred
- what was produced
- what verification happened
- whether export occurred
- why the build failed if it failed

If the system cannot answer those questions, the build is not fully trustworthy.

---

3. Main proof principle

The main principle of BM-15 is:

A generated repo must be traceable back to approved inputs and recorded build actions.

That means the build should not behave like a black box.
It should leave enough evidence that an operator or later system component can inspect how the result came to exist.

---

4. Minimum audit / proof categories

At minimum, Build Mode should record proof in these categories:

1. Build identity and lineage
2. Input provenance
3. Build lifecycle/state transitions
4. Output provenance
5. Verification evidence
6. Export evidence
7. Failure evidence when unsuccessful

That is the minimum meaningful proof set.

---

5. Category 1 — Build identity and lineage

Purpose

Identify the build as a governed event inside the Axion system.

What should be recorded

At minimum:

- run ID
- build ID

- project/workspace reference if applicable
- operator request reference
- timestamps for creation/start/completion
- build profile reference

Why it matters

This is the identity layer that ties the build to the larger run history.

Without this, the build becomes detached from its context.

---

6. Category 2 — Input provenance

Purpose

Record what approved source package the build actually consumed.

What should be recorded

At minimum:

- source Agent Kit ID/version/reference
- locked standards snapshot reference
- locked templates/specs/contracts references
- build profile or repo blueprint reference
- policy context or policy version reference
- any other required build-basis artifact identifiers

Why it matters

This is how you prove the build came from approved truth rather than agent improvisation.

It also helps later if you need to compare two builds from different input versions.

---

7. Category 3 — Build lifecycle / state transitions

Purpose

Record how the build moved through its official lifecycle.

What should be recorded

At minimum:

- initial state
- state transitions
- phase or step progression
- timestamps for key transitions
- final state
- any stop/block/fail transition reasons

Examples:

- `requested` at time X
- `approved` at time Y
- `building` at time Z
- `verifying` at time A
- `passed` or `failed` at time B
- `exported` at time C if applicable

Why it matters

This is what makes the build timeline inspectable and reproducible at the governance level.

---

8. Category 4 — Output provenance

Purpose

Record what build outputs were produced and where they are attached.

What should be recorded

At minimum:

- generated repo reference/location
- build manifest reference
- repo manifest reference
- verification report reference
- export artifact reference if created
- any file index or supporting artifact references if tracked

Why it matters

This creates a chain between build execution and the artifacts it produced.

Without this, outputs can exist but still feel disconnected or ambiguous.

---

9. Category 5 — Verification evidence

Purpose

Record what checks were used to determine whether the build passed.

What should be recorded

At minimum:

- verification run timestamp
- checks executed

- pass/fail result for each required category or check
- final verification result
- references to detailed command/check results where applicable
- any blocking findings

Why it matters

This prevents the build from being declared successful without proof that the required checks actually happened.

---

10. Category 6 — Export evidence

Purpose

Record whether the build result was packaged and exported.

What should be recorded

If export was requested or attempted:

- export requested flag
- export eligibility result
- export timestamp
- export artifact reference
- export success/failure result
- packaging error reason if export failed

Why it matters

This keeps the distinction between:

- build passed
  and

- build exported

clear and auditable.

---

11. Category 7 — Failure evidence

Purpose

Record why the build did not complete successfully when it fails.

What should be recorded

At minimum:

- failed state or phase
- blocking reason
- failed check or missing prerequisite
- whether failure was input-based, policy-based, generation-based, verification-based, or export-based
- timestamp of failure
- partial output references if relevant

Why it matters

A failed build still needs a usable trace.
Otherwise recovery, debugging, and governance become much harder.

---

12. Required audit events

BM-15 should identify the minimum events that should always be recorded.

At minimum, the system should record events for:

- build requested

- build approved or rejected
- build started
- verification started
- build passed or failed
- export started if requested
- export succeeded or failed if attempted

This gives you the smallest useful event trail.

---

13. Proof granularity rule

The proof trail should be structured enough to be useful, but not so noisy that it becomes unusable.

That means BM-15 should focus on:

- state changes
- phase transitions
- key inputs
- key outputs
- key decisions
- key failures

It does not need to record every micro-action unless you decide to add deeper tracing later.

---

14. Relationship to manifests

Not all proof has to live in a single audit ledger.

Some proof will live in:

- build manifest
- repo manifest
- verification report
- export record
- run event history

BM-15 should allow these to work together as a proof set, as long as they remain linked and traceable.

---

15. Relationship to the control plane

The control plane should be able to read and govern Build Mode because the build leaves a proof trail that matches its lifecycle.

That means BM-15 should support:

- state inspection
- failure analysis
- export traceability
- run-level audit review
- future retry/resume decisions

So the audit model is not just for humans.
It is also for system governance.

---

16. Relationship to trust and drift prevention

Proof requirements directly support drift prevention.

Why?

Because if the build must always show:

- what inputs it used
- what path it followed
- what checks it passed
- what outputs it produced

then it becomes much harder for hidden guessing or uncontrolled divergence to go unnoticed.

The proof trail does not eliminate drift by itself, but it makes drift much easier to detect.

---

17. Minimum proof standard for successful builds

A successful build should never be considered complete without proof of:

- lineage
- input provenance
- state progression
- output references
- verification result
- export result if applicable

That is the minimum standard for a governed build.

---

18. Minimum proof standard for failed builds

A failed build should still leave proof of:

- identity
- request and attempted lifecycle state

- failure point
- failure reason
- any produced partial outputs
- whether export was attempted or blocked

A failed build without traceability is an operational blind spot.

---

19. Why this section matters

Without audit/proof requirements:

- successful builds are harder to trust
- failed builds are harder to debug
- the build can feel like a black box
- the control plane has less authority
- export artifacts lose credibility

BM-15 ensures that Build Mode produces not just code, but evidence.

---

20. Success definition

BM-15 is successful when every Build Mode run leaves a structured proof trail covering build identity, input provenance, state transitions, produced outputs, verification evidence, export evidence when applicable, and clear failure evidence when unsuccessful.

---

21. Short version for the doc

BM-15 — Audit / Proof Requirements
Every Build Mode run must leave an auditable proof trail. At minimum, the system must record build identity and lineage, the approved inputs consumed, lifecycle/state transitions, produced

output references, verification evidence, export evidence when applicable, and structured failure evidence when the build does not complete successfully. This proof trail makes repo generation traceable, governable, and trustworthy.

BM-16 — Failure Modes

BM-16 — Failure Modes

1. Purpose of this section

Define the ways Build Mode can fail and how those failures should be understood.

BM-16 should make clear that build failure is not a single generic condition. Different failures happen at different parts of the lifecycle and mean different things for recovery, audit, and operator visibility. This section exists so Build Mode can fail in a structured way instead of collapsing all problems into one vague “build failed” result.

---

2. Core rule

Build Mode must fail explicitly and categorically.

That means the system should not:

- hide where the failure happened
- blur input failures with generation failures
- treat verification failure as the same as export failure
- keep running past critical blockers
- silently compensate for missing required truth

Every meaningful failure should have a recognizable class.

---

3. Main failure principle

The main principle of BM-16 is:

When Build Mode cannot continue without guessing, violating policy, or producing an invalid result, it must stop and record the failure class.

This is one of the key anti-drift protections in the system.

---

4. Minimum failure classes

At minimum, Build Mode should recognize these failure classes:

1. Missing Kit input
2. Incomplete or invalid build inputs
3. Build eligibility failure
4. Build planning failure
5. Workspace preparation failure
6. Repo generation failure
7. Verification failure
8. Packaging / export failure
9. Record / manifest failure

That is the minimum clean set.

---

5. Failure class 1 — Missing Kit input

Meaning

The required Agent Kit does not exist, is not approved, is inaccessible, or is not valid as the active build basis.

Typical causes

- no Agent Kit was generated
- the Kit was superseded
- the Kit reference is broken
- the Kit is incomplete in a way that makes build impossible

- the system cannot load the Kit

Why it matters

This is a foundational failure.
Without the Kit, Build Mode has no valid build-intelligence package.

Lifecycle impact

This should usually fail early, before active build execution begins.

---

6. Failure class 2 — Incomplete or invalid build inputs

Meaning

The broader required input set exists only partially, conflicts internally, or fails the declared build-readiness standard.

Typical causes

- missing required specs/contracts/templates
- missing locked standards snapshot
- missing build profile
- conflicting upstream artifacts
- missing repo blueprint where required
- required file inventory absent for the chosen profile

Why it matters

This is different from a missing Kit.
The Kit may exist, but the surrounding build basis may still be too weak or contradictory to support a controlled build.

Lifecycle impact

This should usually fail during eligibility or planning.

---

7. Failure class 3 — Build eligibility failure

Meaning

The operator requested a build, but policy, gating, or blocker conditions do not allow the build to start.

Typical causes

- upstream gates not passed
- policy disallows build for this run
- critical unresolved blockers remain
- required approvals are missing
- export requested but not allowed under policy

Why it matters

This is a governance failure, not necessarily a content failure.

Lifecycle impact

This usually blocks the transition from `requested` to `approved`.

---

8. Failure class 4 — Build planning failure

Meaning

The system cannot derive a concrete and valid build path from the approved inputs.

Typical causes

- the Kit is too vague to produce a stable plan
- required file targets cannot be inferred from the approved inputs
- build slices/order cannot be resolved safely

- conflicting instructions prevent execution planning
- repo shape cannot be determined for the selected profile

Why it matters

This is where the system must choose failure over guessing.

Lifecycle impact

This typically happens after eligibility but before or at the start of active generation.

---

9. Failure class 5 — Workspace preparation failure

Meaning

The system cannot establish the internal environment needed to execute the build.

Typical causes

- build directories cannot be created
- staging/output paths collide or are invalid
- required workspace setup step fails
- manifest/verification/export directories cannot be initialized

Why it matters

This is an operational preparation failure.
The build may be logically valid but still unable to run.

Lifecycle impact

This usually happens right before entering or at the beginning of `building`.

---

10. Failure class 6 — Repo generation failure

Meaning

The system begins repo materialization but cannot complete it successfully.

Typical causes

- required files cannot be generated
- module wiring breaks during generation
- generation encounters blocked missing details
- a critical no-guess condition is hit mid-build
- the generated output becomes structurally invalid during materialization

Why it matters

This is the main execution failure class.

Lifecycle impact

This happens during `building`.

---

11. Failure class 7 — Verification failure

Meaning

The repo was generated enough to check, but the output did not satisfy required build-phase checks.

Typical causes

- required files missing
- repo structure incorrect
- unresolved critical placeholders remain
- manifests incomplete or inconsistent
- required commands/checks fail
- export readiness conditions are not met

Why it matters

This is one of the most important failure classes because it prevents invalid repos from being treated as successful.

Lifecycle impact

This happens during `verifying` and should usually lead to `failed`.

---

12. Failure class 8 — Packaging / export failure

Meaning

The build itself may have passed, but the requested export artifact could not be created or recorded successfully.

Typical causes

- zip creation fails
- export assembly is incomplete
- required export contents cannot be collected
- artifact writing fails
- export recording fails

Why it matters

This must remain distinct from repo validity.
A build can be valid yet still fail to export.

Lifecycle impact

This happens after `passed` when export is requested.

---

13. Failure class 9 — Record / manifest failure

Meaning

The system cannot produce or finalize the required non-code records that make the build traceable and complete.

Typical causes

- build manifest cannot be written
- repo manifest is missing or invalid
- verification report cannot be finalized
- output references cannot be attached to the run
- final state closure record cannot be completed

Why it matters

Build Mode is a governed system.
If records fail in a required way, completion should not be treated as clean.

Lifecycle impact

This can occur during build, verification, or completion finalization.

---

14. Failure severity concept

Even if you do not fully formalize severity yet, BM-16 should imply at least two severity levels:

Blocking failures

These stop the build immediately and prevent completion.

Examples:

- missing Agent Kit
- failed eligibility
- repo generation blocked
- required verification failure

Downstream failures

These happen after much of the build succeeded but still prevent the requested outcome from fully completing.

Examples:

- export failure
- final record writing failure

This distinction will help later with retry and resume rules.

---

15. Failure timing concept

A simple way to understand the failure classes is by when they occur.

Pre-build failures

- missing Kit input
- incomplete/invalid build inputs
- build eligibility failure
- build planning failure

Build-execution failures

- workspace preparation failure
- repo generation failure

Post-generation failures

- verification failure
- record/manifest failure
- packaging/export failure

This is useful for both UI and audit.

---

16. Failure response rule

When a failure occurs, the system should:

- stop at the relevant point
- record the failure class
- record the failure reason
- record the lifecycle state/phase
- preserve useful partial outputs when appropriate
- block export unless the requested outcome can still be validly satisfied

The system should not:

- silently continue
- silently downgrade required failures
- invent missing truth to get past the failure

---

17. Failure and operator visibility

Each failure class should be visible enough that the operator can tell:

- what failed
- where it failed
- whether it was an input, policy, generation, verification, export, or records issue
- whether retry/resume might be possible later

BM-16 does not need to fully define recovery yet, but it should make failure understandable.

---

18. Failure and state relationship

BM-16 should align with BM-08.

Examples:

- missing Kit / invalid inputs / eligibility failure can move `requested` → `failed`
- planning or workspace problems can move `approved` → `failed`
- generation problems can move `building` → `failed`
- verification failure moves `verifying` → `failed`
- packaging failure may occur after `passed` if export was requested

This keeps failure tied to lifecycle logic.

---

19. Why this section matters

Without structured failure modes:

- every failure looks the same
- operator feedback gets weak
- audit becomes less useful
- retry/resume logic becomes harder
- drift risks increase because the system may be tempted to “just keep going”

BM-16 makes failure an explicit part of the Build Mode design.

---

20. Success definition

BM-16 is successful when Build Mode recognizes and records distinct failure classes for missing inputs, invalid build basis, eligibility, planning, workspace setup, generation, verification, export, and required record production, and when those failures stop the build instead of being silently absorbed.

---

21. Short version for the doc

BM-16 — Failure Modes

Build Mode must fail explicitly and categorically. At minimum, it should distinguish missing Kit input, incomplete or invalid build inputs, build eligibility failure, build planning failure, workspace preparation failure, repo generation failure, verification failure, packaging/export failure, and record/manifest failure. These failure classes make the build lifecycle traceable, keep operators informed, and prevent the system from guessing through blocked conditions.

BM-17 — Non-Goals

BM-17 — Non-Goals

1. Purpose of this section

Define what Build Mode is explicitly **not** trying to do.

BM-17 matters because once a system can generate a full repo, people start mentally expanding it into more than it really is. This section prevents Build Mode from being treated like a fully autonomous product architect, deployment engine, or all-purpose software factory.

---

2. Core rule

Build Mode should be judged by the job it is supposed to do, not by adjacent jobs it could be confused with.

Its job is to take an approved build basis and materialize a governed repository output. Anything outside that boundary should be treated as outside the mode unless another section explicitly adds it.

---

3. Main non-goal principle

The main principle of BM-17 is:

Build Mode is a controlled repo-generation and export system, not a substitute for upstream planning, uncontrolled invention, or downstream deployment.

That one sentence should govern the whole section.

---

4. Non-goal 1 — Build Mode does not define the product

Build Mode is not responsible for deciding what product should exist.

It does not:

- create the original product concept
- decide the business scope from scratch

- choose the core feature set from scratch
- invent the project direction during build execution

Those decisions belong upstream in intake, canonicalization, planning, specs, templates, and the Agent Kit.

---

5. Non-goal 2 — Build Mode does not invent missing requirements

Build Mode is not supposed to fill major gaps in upstream truth by imagination.

It should not:

- invent missing entities because they “probably should exist”
- create undeclared routes or screens because they seem useful
- infer missing contracts without an allowed basis
- create entirely new modules outside locked scope
- silently choose a broader app structure than what was approved

If required truth is missing, the correct behavior is to fail or block, not improvise.

---

6. Non-goal 3 — Build Mode is not a freeform coding session

Build Mode is not meant to behave like an unconstrained AI coding assistant.

It is not supposed to:

- roam across the repo making stylistic choices at will
- add optional libraries by personal preference
- rewrite architecture because a different pattern seems cleaner
- dynamically redesign the stack during generation

- treat the build like an open creative exercise

The whole point of the mode is controlled materialization, not open-ended coding.

---

7. Non-goal 4 — Build Mode does not replace the Agent Kit

Build Mode is not the source of build intelligence.

It should not replace:

- the approved Agent Kit
- the specs/contracts/templates the Kit points to
- the upstream planning and scope assembly process

The Kit remains the authoritative instruction package.
Build Mode only executes within that package and the control rules around it.

---

8. Non-goal 5 — Build Mode does not replace the main control plane

Build Mode may have a build-focused control layer, but it is not the main Axion governance system.

It does not replace:

- run lifecycle governance
- cross-run policy enforcement
- system-wide approvals
- global artifact governance
- broader audit authority

It remains a governed subsystem operating under the larger control-plane model.

---

9. Non-goal 6 — Build Mode does not bypass verification

Build Mode is not allowed to treat repo generation alone as success.

It is not supposed to:

- skip required build checks
- treat a partially verified repo as complete
- package outputs before required verification passes
- hide unresolved critical placeholders
- mark a build successful because it “mostly worked”

If verification fails, the build should fail.

---

10. Non-goal 7 — Build Mode does not convert failed builds into valid deliverables

Build Mode should not turn incomplete or failed outputs into handoff artifacts that look trustworthy.

It should not:

- zip failed repos as if they were valid completions
- hand off placeholder-heavy outputs as successful builds
- package around missing manifests
- quietly downgrade critical failures so export can happen

A portable artifact should represent a successful build path, not a disguised failure.

---

11. Non-goal 8 — Build Mode does not perform deployment

Build Mode ends at verified repo output and optional portable packaging.

It does not:

- deploy to hosting
- provision cloud infrastructure
- manage production secrets
- publish to app stores
- stand up databases in production
- own runtime hosting operations

Those are downstream systems or later workflow stages.

---

12. Non-goal 9 — Build Mode does not guarantee production readiness in every sense

Even if Build Mode generates a strong repo, that does not automatically mean the app is fully ready for real-world production use in every dimension.

Build Mode does not inherently guarantee:

- final product-market correctness
- production-grade security review
- scale/performance under real traffic
- operational maturity
- legal/compliance approval beyond what the build scope explicitly checks
- complete human QA signoff

Its guarantee is narrower: governed repo generation inside declared build rules.

---

13. Non-goal 10 — Build Mode does not silently repair upstream planning problems

If upstream truth is broken, incomplete, or contradictory, Build Mode should not become a hidden resolver of those issues.

It should not:

- reinterpret conflicting specs on its own
- create an unofficial scope merger
- decide which upstream artifact “probably wins”
- cover for missing planning artifacts with generated assumptions

Those issues should be resolved upstream or blocked explicitly by policy.

---

14. Non-goal 11 — Build Mode does not own post-export development

Once the repo has been produced and optionally exported, Build Mode is no longer responsible for everything that happens afterward.

It does not own:

- manual code edits in an IDE
- later refactors by a human dev
- new features added after export
- divergence introduced outside the originating Axion run
- downstream branch/release management in an external workflow

Its job ends at governed build output and portable handoff.

---

15. Non-goal 12 — Build Mode is not an excuse for weak upstream artifacts

A system that can generate code can create pressure to weaken the quality of the Agent Kit or upstream specs because people assume the builder will “figure it out.”

BM-17 should reject that idea.

Build Mode is not supposed to make weak upstream inputs acceptable.
If anything, it increases the need for:

- stronger Kits
- clearer specs
- better locked inputs
- stronger build controls

So one non-goal is:
Build Mode is not a substitute for upstream rigor.

---

16. Why non-goals matter here

This section matters because repo generation makes the system look more powerful than it should be trusted to be.

Without explicit non-goals, people may assume Build Mode should:

- think like a product manager
- decide like an architect
- code like a whole dev team
- verify like a QA org
- deploy like DevOps
- recover every upstream weakness automatically

That is exactly the kind of expectation drift that breaks systems.

---

17. Relationship to trust

Being explicit about non-goals actually increases trust.

Why?

Because it clarifies what the build result means and what it does not mean.
That helps the operator use the output correctly instead of over-trusting it.

A trustworthy system is not one that claims to do everything.
It is one that clearly defines its boundaries.

---

18. Success definition

BM-17 is successful when it is clear that Build Mode is only responsible for governed repo generation, build-phase verification, and optional export packaging, and is not responsible for upstream product definition, requirement invention, uncontrolled coding, main control-plane replacement, deployment, or compensating for weak upstream truth.

---

19. Short version for the doc

BM-17 — Non-Goals
Build Mode is not a product-definition engine, a freeform coding session, a replacement for the Agent Kit, a substitute for the main control plane, or a deployment system. It does not invent missing requirements, bypass verification, package failed outputs as valid deliverables, or compensate for weak upstream planning. Its role is narrower: governed repo generation, build-phase verification, and optional portable export from an approved build basis.

BM-18 — Relationship to Maintenance Modes

BM-18 — Relationship to Maintenance Modes

1. Purpose of this section

Define how Build Mode relates to the Axion Maintenance System and make clear that the two are connected but not the same thing.

BM-18 matters because Build Mode touches packaging, outputs, and generated artifacts, which can make it easy to blur the line between:

- runtime repo generation
  and
- maintenance operations on the system itself

This section keeps that separation clean.

---

2. Core rule

Build Mode is a **runtime build capability**, while maintenance modes are **system upkeep and controlled change capabilities**.

That means Build Mode exists to generate a project repository from an approved build basis during a run.

Maintenance modes exist to:

- maintain system health
- manage controlled updates
- handle packaging/release concerns at the maintenance layer
- preserve quality, continuity, and governance across the Axion system over time

So Build Mode and Maintenance are related, but they do different jobs.

---

3. Main relationship principle

The main principle of BM-18 is:

Build Mode produces governed build outputs during a run; Maintenance Modes govern the upkeep, adjustment, and lifecycle management of the system and its controlled artifacts across runs.

That is the cleanest distinction.

---

4. Build Mode is not a maintenance mode

Build Mode should not be reduced to a maintenance mode.

Why?

Because its main purpose is not:

- system upkeep
- post-build housekeeping
- long-term maintenance actions
- system optimization activity
- maintenance-only packaging activity

Its main purpose is:

- turning an approved Agent Kit into a generated repository
- verifying that repository
- optionally exporting it as a portable bundle

That makes Build Mode a core execution capability, not a maintenance action.

---

5. Why the confusion can happen

The confusion usually happens because Build Mode includes:

- manifests
- verification
- packaging
- export
- output records

Those things can sound similar to maintenance responsibilities.

But similarity of artifacts does not mean identity of purpose.

Build Mode uses those things to complete a governed build run.
Maintenance uses similar mechanisms for a different reason: to keep the broader system controlled over time.

---

6. Relationship to Release Packaging

This is the most important overlap point.

You already have **MM-14 — Release Packaging** in the Maintenance System. That confirms that packaging already exists in your maintenance model as a governed concern rather than a casual utility.

Response: Axion Updates Planning

So BM-18 should define the relationship like this:

Build Mode packaging

Packaging inside Build Mode is:

- run-scoped
- downstream of a successful build
- tied to a specific generated repo output
- part of portable handoff for that run

Maintenance release packaging

Release Packaging in maintenance is:

- system-scoped or lifecycle-scoped
- concerned with controlled packaging rules at the maintenance level
- concerned with broader packaging governance, release handling, and upkeep logic

So Build Mode may **use** packaging, but maintenance still **governs packaging as a larger system concern**.

---

7. Build Mode can feed maintenance, but not replace it

A successful Build Mode run may produce artifacts that later become inputs to maintenance-facing processes.

For example, Build Mode may produce:

- a repo
- manifests
- verification outputs
- an export artifact

Maintenance-related processes may later use those for:

- retention
- release handling
- artifact review
- export governance
- lifecycle tracking
- upgrade comparison
- rebuild or regeneration decisions

So the relationship is:

Build Mode can feed Maintenance, but Build Mode is not Maintenance.

---

8. Maintenance should not own repo generation

The inverse is also important.

Maintenance Modes should not become the place where actual repo generation logic lives.

Maintenance may:

- monitor
- package
- govern
- archive
- compare
- manage lifecycle state

But the act of:

- consuming the Agent Kit
- planning the repo build
- materializing code/files
- verifying the build result

belongs to Build Mode.

That keeps the architecture clean.

---

9. Relationship to system updates

Maintenance Modes may later update or affect the environment around Build Mode, such as:

- packaging rules
- policy rules
- output retention rules
- verification policies
- allowed export behavior
- build-related system health rules

But those are changes to the system's governing environment, not the same thing as the active build run itself.

So BM-18 should make clear that Maintenance can influence Build Mode's operating conditions without becoming Build Mode.

---

10. Relationship to artifact upkeep

Build Mode creates run-specific build artifacts.

Maintenance may later govern how those artifacts are:

- retained
- versioned
- compared
- archived
- reviewed
- updated at the system-policy level

Again, that is a lifecycle relationship, not an identity relationship.

Build Mode creates the artifact set.
Maintenance helps govern its life over time.

---

11. Relationship to failure handling

Build Mode owns failure handling for the build run itself.

Maintenance may later own or assist with broader concerns such as:

- trend analysis across failed builds
- policy updates based on recurring failures
- packaging rule updates
- operational cleanup
- system-level improvements

So:

- Build Mode handles the specific failed build event
- Maintenance handles broader system care and evolution around recurring or structural issues

---

12. Relationship to optimization

This is especially relevant to your Maintenance System vision.

Your maintenance direction is more autonomous in suggesting system improvements, optimizations, and updates, but not executing them without consent. That means maintenance is oriented toward:

- system optimization
- pattern detection
- controlled improvement suggestions
- upkeep decisions

Build Mode is different.

Build Mode is oriented toward:

- one run
- one approved Kit
- one governed repo-generation path
- one concrete output set

So Build Mode executes a specific build.
Maintenance looks across time and system health.

---

13. Clean separation statement

A strong separation statement for BM-18 is:

- **Build Mode** answers: “Can this approved Kit be turned into a governed repo output now?”
- **Maintenance Modes** answer: “How should the Axion system and its governed artifacts be maintained, packaged, updated, and managed over time?”

That is the cleanest distinction in plain language.

---

14. Practical architecture implication

Because of this relationship, the architecture should reflect the split:

Build Mode should own

- build request handling
- build planning
- workspace setup
- repo generation
- build verification
- export creation for the run

Maintenance should own

- higher-level packaging governance
- release-policy stewardship
- retention/cleanup rules
- artifact lifecycle management
- system optimization and update suggestion logic
- broader upkeep decisions

This prevents cross-contamination between runtime building and system maintenance.

---

15. What BM-18 should prevent

This section should prevent two bad outcomes:

Outcome 1

Treating Build Mode like “just another maintenance task,” which would understate its importance and muddy its runtime purpose.

Outcome 2

Treating Maintenance like the build engine, which would overload maintenance with execution responsibilities it should not own.

BM-18 exists to stop both of those.

---

16. Success definition

BM-18 is successful when it is clear that Build Mode is a core runtime repo-generation capability, while Maintenance Modes are system-upkeep and lifecycle-governance capabilities; when Build Mode packaging is understood as run-scoped handoff packaging; and when higher-level release/packaging governance remains properly associated with the maintenance layer rather than replacing the build system.

---

17. Short version for the doc

BM-18 — Relationship to Maintenance Modes
Build Mode is a runtime build capability, not a maintenance mode. It exists to turn an approved Agent Kit into a generated, verified, and optionally exported repository during a run. Maintenance Modes exist to govern system upkeep, packaging policy, artifact lifecycle management, and broader optimization over time. Build Mode may produce outputs that feed maintenance processes, and it may use packaging rules that align with maintenance concerns such as MM-14 — Release Packaging, but it does not replace the maintenance layer or reduce to it.

Response: Axion Updates Planning

What needs to change

What needs to change

1. Split the workload into layers

Do not think of Build Mode as:

Kit in → full app out

Think of it as:

Kit in → plan build → generate repo in parts → verify → package

That matters because the heavy part is not just “writing code.” The heavy part is:

- deciding what files must exist
- generating many files consistently
- wiring files together
- checking that the repo is coherent
- packaging only after it passes

2. Make the control plane carry more of the weight

The more the control plane pre-locks, the less the agent has to reason live.

Before the agent starts building, the system should already know:

- stack/profile
- repo shape
- required domains/modules
- required files
- naming rules
- contracts/specs to satisfy
- verification commands
- export rules

That way the agent is not “figuring out the app” from scratch.
It is **materializing a locked plan**.

That is the main way you prepare for heavier workload without wasting tokens.

3. Build in slices, not as one pass

A full repo should be generated in controlled chunks.

Best order:

- repo skeleton
- configs/manifests
- shared types/contracts
- backend
- frontend
- tests
- docs/env/examples
- verification
- zip export

This reduces drift and makes failures easier to isolate.

4. Add a build workspace

Do not generate straight into the final export.

Use a build workspace with stages:

- staging
- generated repo
- verification outputs
- exports

That gives you a safe place to:

- retry a failed step
- inspect intermediate outputs
- keep proofs
- avoid corrupt final bundles

5. Add stronger build gates

Because this workload is heavier, you need a stricter threshold before export.

Minimum build gates:

- kit complete
- required specs present
- no critical unknowns
- required file set generated
- unresolved placeholders blocked
- verification commands pass
- manifest complete
- only then zip export

This also keeps packaging aligned with your current separation, where packaging is a governed step rather than something implied automatically.

Response: Axion Updates Planning

What the system should be prepared to handle

You should expect these pressure points:

Token pressure

Large repos mean more context and more reasoning unless the control plane narrows the task.

Consistency pressure

The issue is not only making code. It is making code that agrees with:

- routes
- entities
- schemas
- config
- naming
- file tree
- acceptance rules

Failure recovery

A big generation run will fail sometimes.
So Build Mode needs restart points, not just one all-or-nothing run.

Verification load

Once Axion outputs repos, verification becomes more important, not less.

The simplest preparation model

Prepare in four buckets:

A. Planning prep

Make sure the Kit includes enough build-ready detail:

- file targets
- module targets

- stack locks
- dependency expectations
- required commands
- acceptance map

B. Execution prep

Add:

- build planner
- generator
- workspace manager
- verifier
- exporter

C. Governance prep

Add:

- build request state
- build approval state
- build progress state
- failure reason state
- export state

D. Recovery prep

Add:

- resumable steps
- partial rerun support
- artifact logs
- failure reports

The key rule

The heavier the build workload gets, the less freedom the generator should have.

That sounds backward, but it is how you keep quality high.

You do **not** prepare by making the agent “smarter” in a vague way.
You prepare by making the job **more constrained, more staged, and more checkable**.

Bottom line

To prepare for the heavier workload, you need to make Build Mode:

- more staged
- more locked
- more resumable
- more verifiable

So the agent is not carrying the whole burden alone.
The Kit tells it what to build, and the control plane narrows the path enough that the code-generation workload becomes manageable.

Control plane

So the better setup is:

1. Main control plane

This governs the whole Axion run:

- run state
- stages
- gates
- approvals
- artifacts
- policies
- audit
- packaging status

2. Build control plane

This governs only the build execution:

- what build profile is active
- what repo shape is locked
- what file targets must exist
- what order modules are built in
- what dependencies are allowed
- what commands/checks must run
- what counts as pass/fail

- what can be retried
- what cannot be guessed

Agentg Kit

Should the Agent Kit alone be trusted to prevent drift?

My honest answer:

Strong enough to guide heavily? yes.
Strong enough by itself to fully prevent drift and guessing? no.

An Agent Kit can be excellent and still not be enough by itself once the task becomes full repo generation.

Why?

Because the build agent will still hit moments like:

- a file is implied but not explicitly listed
- two specs conflict slightly
- naming is mostly clear but not total
- dependency choices are not fully locked
- scaffold order is not explicit enough
- one module needs shared types that were never declared as a file target

That is where guessing starts.

So if your goal is:

“I want it to basically be a rigged step by step guide”

then the answer is:

the Kit should provide the instructions, but the build control plane must provide the rails.

Best model

Think of it like this:

Agent Kit = build intelligence package

It tells the builder:

- what the app is
- what domains/features exist
- what standards apply
- what contracts/specs must be satisfied
- what outputs are expected

Build control plane = anti-drift enforcement layer

It tells the builder:

- exact order of execution
- exact allowed moves
- exact required files
- exact dependency boundaries
- exact acceptance checks
- exact failure behavior
- exact no-guess rules

That combination is what gives you the “rigged” behavior.

What would make it feel rigged

If you want it to feel like a step-by-step guided build instead of open-ended AI coding, then the build agent should receive all of this in locked form:

1. Repo blueprint

Not just “build a React/Express app.”

It should know:

- root folders
- required subfolders
- required files
- required config files
- naming conventions
- path aliases
- import style expectations

2. File production order

The build should not choose its own order.

The system should say:

1. create repo skeleton
2. create manifests/config
3. create shared contracts/types
4. create backend entities/routes/services
5. create frontend screens/components/state hooks
6. create tests
7. create docs/env examples

8. run verification
9. export zip

3. File-by-file targets

The agent should not infer the whole repo shape.

It should be given a list like:

- create these files
- populate these sections
- wire these imports
- satisfy these contracts

4. No-guess rules

Very important.

The build control plane should say:

- if required input is missing, stop
- if spec conflict exists, stop or mark blocked
- if file target is unclear, stop
- do not invent endpoints/entities/routes/components outside the declared scope
- do not add libraries unless allowed by the build profile
- do not rename locked identifiers

5. Checkpoints between slices

Instead of one giant build, require passes between phases:

- skeleton pass
- contracts pass
- backend pass
- frontend pass
- verification pass
- export pass

That is how you reduce drift during long runs.

So is the current Agent Kit strong enough?

I would say:

It is probably strong enough to be the foundation.
But if your target is **minimal guessing**, then it still needs to be paired with these build-specific things:

Build additions the Kit likely needs

- explicit repo file tree target
- file creation order
- required file inventory
- dependency allowlist
- module wiring map
- import/export expectations

- unresolved-input handling rules
- stop conditions
- retry rules
- verification commands tied to phases

Without those, even a strong Kit can still leave room for interpretation.

The clean architecture

Main control plane

Owns:

- run lifecycle
- approvals
- artifacts
- packaging eligibility
- audit

Build control plane

Owns:

- build plan
- build states
- slice order
- target file matrix

- constraints
- retry/resume
- no-guess enforcement

Agent Kit

Owns:

- content instructions
- product/domain logic
- requirements/specs/contracts
- standards/templates

Build agent

Owns:

- materializing the locked plan into code
- only the necessary reasoning inside the locked boundaries

That is the balance you want.

physical design set

The order should be:

1. Build Mode artifact list

Define every file/spec/artifact the Build Mode system needs.

That means making the physical list for things like:

- build state model
- build request schema
- build manifest schema
- repo manifest schema
- verification report schema
- export record schema
- build runner contract
- build planner contract
- repo generator contract
- verifier contract
- exporter contract
- build control plane contract

2. Build Control Plane outline

You already established that the Build Mode needs its own control layer.

So the next major document branch should be:

- build control plane purpose
- responsibilities

- state ownership
- transition rules
- no-guess rules
- build gating behavior
- retry/resume authority
- export authority

3. Schemas and contracts

After that, define the actual system objects.

Minimum set:

- `build_request`
- `build_state`
- `build_plan`
- `build_manifest`
- `repo_manifest`
- `verification_report`
- `export_record`
- `failure_report`

4. File tree placement

Then map those pieces into the current repo structure.

Meaning:

- what goes in `Axion/src/core/buildMode/`
- what extends `controlPlane/`
- what goes in CLI
- what goes in run artifacts under `.axion/runs/.../build/`

5. Build flow mechanics

After the objects are defined, turn BM-06 into implementation mechanics:

- exact phase order
- inputs/outputs per phase
- pass/fail conditions per phase
- what writes what artifact
- what state transitions happen where

6. Gates and rules

Then formalize:

- entry gate
- verification gate
- export gate
- stop conditions
- fail conditions
- no-guess enforcement rules

7. Physical repo creation

After all that, you can start creating the actual files in the repo.

So the immediate next step is:

Build Mode physical design set
starting with:
artifact list → control plane outline → schemas/contracts

Build Mode artifact list

We start with the **Build Mode artifact list**.

That means the next section should be the full list of physical pieces the system needs before we talk about implementation.

The first artifact set should be:

- `build_request`
- `build_state`
- `build_plan`
- `build_manifest`
- `repo_manifest`
- `verification_report`
- `export_record`
- `failure_report`
- `build_runner_contract`
- `build_planner_contract`
- `repo_generator_contract`
- `build_verifier_contract`
- `export_packager_contract`
- `build_control_plane_contract`

After that, we detail each one one by one.

Build Mode — Artifact Inventory

Build Mode — Artifact Inventory

1. Purpose of this section

Define every physical artifact Build Mode needs so the system can move from outline-level design into real implementation planning.

This is the bridge between:

- Build Mode concept
  and
- Build Mode files, schemas, contracts, and runtime artifacts

The goal here is to identify the full artifact inventory before detailing each piece one by one.

---

2. Artifact inventory structure

The Build Mode artifact set should be grouped into five layers:

1. control artifacts
2. runtime state artifacts
3. build execution artifacts
4. output/proof artifacts
5. component contracts

---

3. Control artifacts

These govern whether and how Build Mode is allowed to run.

BM-A01 — `build_control_plane_contract`

Defines the authority, responsibilities, controls, and rules of the Build Control Plane.

BM-A02 — `build_policy_snapshot`

Defines the effective build/export policy state consumed by a build run.

BM-A03 — `build_gate_result`

Records the result of Build Mode gate checks such as entry, verification, and export eligibility.

BM-A04 — `build_transition_record`

Records controlled state transitions in the Build Mode lifecycle.

---

4. Runtime state artifacts

These track the live state of a build run.

BM-A05 — `build_request`

Records the operator's request to enter Build Mode and the requested output level.

BM-A06 — `build_state`

Defines the current Build Mode lifecycle state for the run.

BM-A07 — `build_context`

Defines the working runtime context for the build, including run linkage and active references.

BM-A08 — `build_session`

Defines one concrete build attempt/execution session under a run.

---

5. Build execution artifacts

These define what the system is supposed to do during repo generation.

BM-A09 — `build_plan`

Defines the actionable build path derived from the approved Agent Kit and locked inputs.

BM-A10 — `repo_blueprint`

Defines the expected repository structure and placement model.

BM-A11 — `required_file_inventory`

Defines the minimum file set expected for the declared build profile/scope.

BM-A12 — `build_phase_record`

Defines each build phase or slice and its status/results.

BM-A13 — `workspace_manifest`

Defines the internal workspace/staging structure used during build execution.

BM-A14 — `dependency_rule_set`

Defines allowed dependency behavior for the generated repo.

BM-A15 — `verification_command_set`

Defines the required commands/checks the generated repo must satisfy.

---

6. Output / proof artifacts

These are the artifacts produced by Build Mode and used for traceability and handoff.

BM-A16 — `build_manifest`

Records what build occurred, from what basis, and with what result.

BM-A17 — `repo_manifest`

Describes the generated repository as an output object.

BM-A18 — `file_index`

Indexes the files produced in the generated repo.

BM-A19 — `verification_report`

Records the build-phase verification result.

BM-A20 — `command_results`

Stores detailed results from executable verification checks.

BM-A21 — `export_record`

Records packaging/export details for the build.

BM-A22 — `failure_report`

Records structured failure information when the build does not complete successfully.

BM-A23 — `build_audit_bundle`

Logical grouping of the proof trail for the build attempt.

BM-A24 — `project_repo_zip`

The final portable exported repository artifact.

---

7. Component contracts

These define the expected behavior of the Build Mode components.

BM-A25 — `build_runner_contract`

Defines orchestration responsibilities and flow control.

BM-A26 — `build_request_handler_contract`

Defines how build requests are accepted and recorded.

BM-A27 — `build_eligibility_checker_contract`

Defines eligibility evaluation behavior and outputs.

BM-A28 — `build_planner_contract`

Defines how the actionable build plan is derived.

BM-A29 — `workspace_manager_contract`

Defines workspace creation and path-handling behavior.

BM-A30 — `repo_generator_contract`

Defines repo materialization behavior and constraints.

BM-A31 — `build_manifest_writer_contract`

Defines record-writing responsibilities.

BM-A32 — `build_verifier_contract`

Defines verification scope execution and pass/fail behavior.

BM-A33 — `export_packager_contract`

Defines packaging/export behavior and rules.

---

8. Minimum viable artifact set

If you want the smallest serious first version, the minimum set is:

Required minimum

- `build_control_plane_contract`

- `build_request`
- `build_state`
- `build_plan`
- `build_manifest`
- `repo_manifest`
- `verification_report`
- `export_record`
- `failure_report`
- `build_runner_contract`
- `build_planner_contract`
- `repo_generator_contract`
- `build_verifier_contract`
- `export_packager_contract`

That is the lean set you already started pointing toward.

---

9. Recommended first build order

The best order to detail these is:

Phase 1 — governance and activation

1. `build_control_plane_contract`

2. `build_request`
3. `build_state`

Phase 2 — execution planning

4. `build_plan`
5. `repo_blueprint`
6. `required_file_inventory`

Phase 3 — output and proof

7. `build_manifest`
8. `repo_manifest`
9. `verification_report`
10. `export_record`
11. `failure_report`

Phase 4 — component contracts

12. `build_runner_contract`
13. `build_planner_contract`
14. `repo_generator_contract`
15. `build_verifier_contract`
16. `export_packager_contract`

---

10. Best next step

The next artifact to detail should be:

BM-A01 — `build_control_plane_contract`

because it governs:

- build authority
- lifecycle control
- no-guess enforcement
- phase progression
- export authority

That should come before the other artifacts.

BM-A01

BM-A01 — `build_control_plane_contract`

1. Purpose

Define the authority, scope, and control rules for the Build Control Plane.

This artifact exists to make Build Mode governed instead of freeform. It defines what the Build Control Plane is allowed to control, what it must enforce, what it consumes, what it produces, and how it interacts with the build agent, the main control plane, and the rest of Build Mode.

---

2. Why this artifact exists

The Agent Kit can tell the build agent what needs to be built, but that alone does not guarantee a low-drift build.

The Build Control Plane exists to provide:

- execution governance
- state control
- phase sequencing
- no-guess enforcement
- build gating
- export authority

Without this artifact, Build Mode can easily turn into:

- loose orchestration
- inconsistent build order
- hidden guessing
- weak operator control

- unclear pass/fail behavior

So this contract is the governing document for how Build Mode is controlled.

---

3. Core definition

`build_control_plane_contract` is the formal contract that defines the Build Control Plane as the build-specific governing layer responsible for controlling Build Mode execution from build request through final pass/fail/export outcome.

It is not the main control plane for all of Axion.
It is the build-scoped control authority that operates under the main control plane.

---

4. Core principle

The Build Control Plane does not define what the product is.
It governs how an approved build is allowed to execute.

In simple terms:

- Agent Kit defines **what must be built**
- Build Control Plane defines **how the build is allowed to proceed**
- Build agent performs **the controlled materialization work**

That is the central principle of this contract.

---

5. Contract objectives

This contract should achieve five things:

5.1 Controlled build activation

Ensure Build Mode only begins on explicit request and valid eligibility.

5.2 Controlled build progression

Ensure the build moves through a fixed and governable sequence.

5.3 Controlled build behavior

Ensure the build agent operates inside locked execution boundaries.

5.4 Controlled completion

Ensure pass/fail/export outcomes are based on rules, not impression.

5.5 Controlled traceability

Ensure build decisions, transitions, and results are recorded in a way the system can inspect later.

---

6. Authority of the Build Control Plane

The Build Control Plane should have authority over the following areas:

6.1 Build lifecycle authority

It controls the official Build Mode state progression.

6.2 Build phase authority

It controls which phases exist, in what order they run, and when transitions are allowed.

6.3 Eligibility authority

It controls whether the build is allowed to begin.

6.4 No-guess authority

It controls what the build agent is not allowed to invent, infer, or silently compensate for.

6.5 Verification authority

It controls what must be checked before the build can be marked passed.

6.6 Export authority

It controls whether packaging/export is allowed and when the state may move to `exported`.

6.7 Failure authority

It controls how failures are classified, recorded, and surfaced.

---

7. What the Build Control Plane does not control

This contract should also explicitly limit its authority.

The Build Control Plane should not control:

- original product definition
- upstream planning logic
- standards creation
- template/spec creation
- the main system-wide run lifecycle outside build scope
- downstream deployment or hosting
- post-export manual development

It governs Build Mode execution only.

---

8. Position in the Axion hierarchy

The Build Control Plane should be positioned like this:

Main Control Plane

Owns:

- overall run governance
- cross-run policy
- stage and system-level control
- broader artifact and audit governance

Build Control Plane

Owns:

- build-specific request handling
- build-specific state and phase control
- build-specific no-guess enforcement
- build-specific verification and export progression

So the Build Control Plane is a subordinate but authoritative control layer within the build branch.

---

9. Inputs consumed by this contract

The Build Control Plane should assume it consumes these input classes:

- build request
- run/build identity context
- approved Agent Kit reference
- locked standards/specs/templates/contracts references
- build profile / repo blueprint references
- policy snapshot
- gate results
- verification outcomes
- export request state

These inputs let it make controlled decisions without inventing new truth.

---

10. Outputs produced by this contract

The Build Control Plane should govern or produce outputs such as:

- build lifecycle state updates
- build transition records
- build gate results
- phase progression decisions
- failure classification decisions
- export eligibility decisions
- closure status decisions

It may not write all physical artifacts directly, but it owns the rules behind them.

---

11. Required responsibilities

At minimum, the `build_control_plane_contract` should define these responsibilities.

11.1 Accept or reject build activation

It must define the rules for moving from no build request to requested/approved build state.

11.2 Enforce valid state transitions

It must define which transitions are legal and illegal.

11.3 Enforce phase order

It must define the official build sequence and block out-of-order execution.

11.4 Enforce no-guess behavior

It must define when the build must stop rather than invent.

11.5 Enforce verification gate

It must define when the build may move from generating to passed or failed.

11.6 Enforce export gate

It must define when the build may move from passed to exported.

11.7 Record transition evidence

It must ensure major lifecycle changes are traceable.

---

12. State authority rules

This contract should define the Build Control Plane as the authority over Build Mode states.

Minimum governed states:

- `not_requested`
- `requested`
- `approved`
- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

The contract should define:

- what each state means
- what can move into it
- what can move out of it
- what evidence is required for the move

---

13. Transition rules

The contract should define allowed and disallowed transitions.

Allowed base path

- `not_requested` → `requested`
- `requested` → `approved`
- `approved` → `building`
- `building` → `verifying`
- `verifying` → `passed`
- `passed` → `exported`

Allowed failure transitions

- `requested` → `failed`
- `approved` → `failed`
- `building` → `failed`
- `verifying` → `failed`
- `passed` → `failed` only if you choose to collapse export failure into overall failure

Disallowed examples

- `not_requested` → `building`
- `requested` → `exported`
- `building` → `exported`

- `failed` → `exported`
- `verifying` → `building` unless you later define explicit retry mechanics

This contract should make those rules explicit.

---

14. Phase authority rules

The Build Control Plane should define the official high-level build phases.

Minimum phase set:

1. request accepted
2. eligibility checked
3. build plan derived
4. workspace prepared
5. repo skeleton/materialization started
6. repo generation completed
7. records/manifests written
8. verification run
9. export eligibility checked
10. packaging/export run
11. final closure recorded

The contract should define:

- phase order
- phase entry conditions

- phase exit conditions
- which failures terminate the phase

---

15. No-guess enforcement rules

This is one of the most important parts of the contract.

The Build Control Plane should enforce the following no-guess rules:

15.1 Missing required input rule

If a required build input is missing, the build must stop.

15.2 Conflict rule

If locked inputs conflict in a way that affects execution, the build must stop unless policy explicitly resolves it.

15.3 Scope expansion rule

The build agent must not add major undeclared features, modules, entities, routes, or screens.

15.4 Dependency rule

The build agent must not introduce undeclared dependencies outside the allowed build profile or dependency rules.

15.5 Naming/identity rule

The build agent must not rename locked identifiers, structures, or contracts without authorization.

15.6 Placeholder rule

Critical unresolved placeholders must block successful completion.

These rules are what make the system feel rigged instead of freeform.

---

16. Gate authority

The Build Control Plane should own the build-specific gates.

Minimum gate set:

16.1 Entry gate

Checks whether the build may begin.

16.2 Verification gate

Checks whether the generated repo may be treated as passed.

16.3 Export gate

Checks whether the build may be packaged/exported.

For each gate, the contract should define:

- required inputs
- pass conditions
- fail conditions
- output record format
- effect on state transitions

---

17. Verification authority

The Build Control Plane should define that no build may move to `passed` unless required verification categories pass.

At minimum, the contract should require verification of:

- required outputs present
- repo structure valid
- critical placeholders absent
- manifests/records complete
- required commands/checks pass

- output consistency sufficient for completion

This keeps completion governed.

---

18. Export authority

The Build Control Plane should define that export is downstream of success, not a shortcut around it.

It should require:

- export explicitly requested
- build already passed
- required output contract satisfied
- required policy approval present
- packaging completed successfully

Only then may the state move to `exported`.

---

19. Failure governance

The Build Control Plane should define failure as a governed terminal or blocking outcome.

At minimum, it should require failure classification for:

- missing Kit input
- invalid/incomplete build inputs
- eligibility failure
- planning failure
- workspace failure

- repo generation failure
- verification failure
- export failure
- manifest/record failure

The contract should also require that failure:

- stops illegal forward progression
- records the failure class
- records the failure point
- preserves relevant evidence

---

20. Operator control relationship

This contract should define the Build Control Plane as honoring operator intent inside governance boundaries.

That means:

- the operator may request build
- the operator may request export
- the operator may inspect status/results

But:

- the operator cannot bypass required gates
- the operator cannot force success
- the operator cannot override no-guess rules mid-build

- the operator cannot export failed builds as if valid

So this contract protects both control and trust.

---

21. Audit relationship

The Build Control Plane should require traceable records for:

- request acceptance
- approval/rejection
- state transitions
- phase transitions
- gate results
- final status
- export status
- failure reasons

This makes Build Mode auditable and governable over time.

---

22. Minimum required fields

At the document level, this artifact should at minimum define:

- contract_id
- contract_name
- version
- purpose

- scope
- governing_principle
- owned_states
- allowed_transitions
- owned_phases
- owned_gates
- no_guess_rules
- entry_conditions
- pass_conditions
- export_conditions
- failure_classes
- operator_control_boundary
- upstream_dependencies
- downstream_effects
- required_audit_events

---

23. Success criteria for this artifact

`build_control_plane_contract` is successful when it clearly defines:

- what the Build Control Plane governs
- what it does not govern
- how Build Mode states and phases are controlled

- what gates exist
- what no-guess rules exist
- how pass/fail/export decisions are made
- how the build stays subordinate to the main control plane while remaining authoritative inside the build branch

---

24. Short version

BM-A01 — `build_control_plane_contract`
Defines the build-scoped governing layer that controls Build Mode execution. It establishes state and phase authority, entry/verification/export gates, no-guess rules, failure governance, and the operator-control boundary so that repo generation follows a locked, traceable, low-drift execution path rather than freeform agent behavior.

BM-A05

BM-A05 — `build_request`

1. Purpose

Define the artifact that records the operator's decision to enter Build Mode.

This artifact exists so Build Mode begins from an explicit, traceable request rather than from assumption. It is the formal trigger record that says the operator chose to continue past the Agent Kit into repo generation, and optionally into export packaging.

---

2. Why this artifact exists

Build Mode should not start automatically just because:

- an Agent Kit exists
- the run is technically buildable
- the system is capable of generating a repo

There needs to be a governed record of intent.

`build_request` exists to:

- capture operator intent
- declare the requested output mode
- bind the request to the originating run
- create the official entry point for the Build Mode lifecycle

Without this artifact, Build Mode activation becomes ambiguous.

---

3. Core definition

`build_request` is the formal runtime artifact that records an operator-issued request to start Build Mode for a specific run, including the requested build outcome and the identity/context needed to govern that request.

---

4. Core principle

A build is not active because it is possible.
A build is active because it was requested and recorded.

That is the main principle behind this artifact.

---

5. What this artifact represents

`build_request` represents:

- the operator's decision to invoke Build Mode
- the requested output level
- the initial governed activation event for the build branch
- the bridge from "Kit complete" to "build lifecycle begins"

It is not the build plan, not the build state, and not proof that the build is allowed.
It is the request artifact.

---

6. What this artifact is responsible for capturing

At minimum, `build_request` should capture:

- who requested the build
- what run the request belongs to
- what outcome was requested
- when the request was made

- what build basis it points to
- whether export is part of the requested outcome

That is the minimum identity + intent set.

---

7. Required responsibilities

The `build_request` artifact should support these responsibilities:

7.1 Activation record

It must act as the official record that Build Mode was requested.

7.2 Run linkage

It must tie the request to the correct originating run.

7.3 Output intent declaration

It must declare whether the request is:

- build repo
- build repo and export zip

7.4 Operator traceability

It must preserve who initiated the request.

7.5 Eligibility handoff

It must provide the input record used by the eligibility checker and Build Control Plane.

---

8. What this artifact does not do

`build_request` should not:

- decide whether the build is eligible

- define the build plan
- store the full build state lifecycle
- declare pass/fail results
- replace the build manifest
- replace export records

It is a trigger/request artifact only.

---

9. Relationship to other artifacts

This artifact sits near the beginning of the Build Mode chain.

Upstream relationship

It depends on:

- an existing originating run
- an available approved Agent Kit or build basis candidate
- a post-Kit operator decision point

Downstream relationship

It feeds into:

- `build_state`
- `build_control_plane_contract`
- eligibility checking
- build session creation
- later manifests and audit records

So `build_request` is the first official runtime artifact in the build branch.

---

10. Request timing

This artifact should be created after:

- the Agent Kit is complete enough for a build decision
- the operator is presented with the outcome options

It should exist before:

- eligibility is finalized
- the build is approved
- the repo generation starts

That timing matters because `build_request` supports the transition from:
`not_requested` → `requested`

---

11. Supported request modes

At minimum, `build_request` should support two build-producing modes:

Mode A — `build_repo`

The operator wants the full project repository generated.

Mode B — `build_repo_and_export`

The operator wants the full project repository generated and packaged as a zip if allowed.

You may optionally model “Kit only” outside Build Mode, since that choice results in no `build_request` being created.

That keeps the artifact clean.

---

12. Request validity rules

A valid `build_request` should satisfy these minimum rules:

12.1 Run must be identified

The originating run must be known.

12.2 Request mode must be explicit

The requested outcome must be declared, not implied.

12.3 Request source must be known

The operator or initiating authority should be identified.

12.4 Request timestamp must exist

The request must be time-stamped.

12.5 Build basis reference should be resolvable

The request should point to the intended build basis, usually the approved Agent Kit or its active reference.

If these are missing, the request should be considered incomplete or invalid.

---

13. Request status vs build status

`build_request` should not be confused with `build_state`.

A request answers:

- was Build Mode asked for?
- what outcome was requested?

A state answers:

- where is the build in its lifecycle now?

So:

- `build_request` is about intent
- `build_state` is about progression

That distinction should stay clear.

---

14. Minimum required fields

At minimum, `build_request` should define fields like:

- `request_id`
- `run_id`
- `build_basis_ref`
- `requested_by`
- `requested_at`
- `request_mode`
- `export_requested`
- `request_source`
- `status` *(optional lightweight request-record status such as active/superseded/cancelled if you want one)*
- `notes` *(optional)*

---

15. Recommended field meanings

Here is the practical meaning of the core fields:

`request_id`

Unique identifier for the build request record.

`run_id`

The originating run this request belongs to.

`build_basis_ref`

Reference to the approved Agent Kit or other active build basis.

`requested_by`

Identity of the operator or initiating authority.

`requested_at`

Timestamp of request creation.

`request_mode`

The requested build-producing outcome.

Suggested enum:

- `build_repo`
- `build_repo_and_export`

`export_requested`

Boolean convenience field for downstream logic.

`request_source`

Where the request came from.

Suggested examples:

- `operator_ui`
- `cli`
- `api`

`status`

Optional record-local status, not the full build lifecycle.

Suggested examples:

- `active`
- `superseded`
- `cancelled`

---

16. Suggested schema shape

A simple logical shape could be:


```
{
 "request_id": "BRQ-0001",
 "run_id": "RUN-0001",
 "build_basis_ref": "KIT-0001",
 "requested_by": "operator_id",
 "requested_at": "2026-03-06T12:00:00Z",
 "request_mode": "build_repo_and_export",
 "export_requested": true,
 "request_source": "operator_ui",
 "status": "active",
 "notes": null
}
```


That is enough for the first version.

---

17. Validation rules

At minimum, validation should require:

- `request_id` present
- `run_id` present

- `requested_by` present
- `requested_at` present
- `request_mode` present and valid enum
- `export_requested` consistent with `request_mode`
- `build_basis_ref` present or resolvable by policy

Suggested consistency rule:

- if `request_mode = build_repo`, then `export_requested = false`
- if `request_mode = build_repo_and_export`, then `export_requested = true`

---

18. Lifecycle behavior of the artifact

The `build_request` artifact should generally be:

- created once when the operator triggers Build Mode
- treated as the input to early Build Control Plane decisions
- preserved for audit even if the build later fails

It may later become:

- superseded by a newer request
- cancelled before approval
- linked to a specific `build_session`

But the original request record should remain traceable.

---

19. Audit significance

This artifact is important for audit because it proves:

- the build was explicitly requested
- by whom
- for which run
- at what time
- with what requested outcome

That makes it the first formal proof event in the Build Mode lifecycle.

---

20. Success criteria for this artifact

`build_request` is successful when it gives the system a clear, valid, traceable activation record for Build Mode that identifies the originating run, the approved build basis reference, the requesting operator, and the requested build outcome without trying to also behave like the build plan or lifecycle state.

---

21. Short version

BM-A05 — `build_request`
Records the operator's explicit request to enter Build Mode for a specific run. It captures the originating run, the build basis reference, the requesting operator, the request time, and the requested outcome (`build_repo` or `build_repo_and_export`). It is the formal activation record for Build Mode and the first runtime artifact in the build lifecycle.

BM-A06

BM-A06 — `build_state`

1. Purpose

Define the artifact that records the current lifecycle status of a Build Mode run.

This artifact exists so the system always knows where a build is in its governed path. It is the official status record for Build Mode progression, from initial request through approval, active generation, verification, pass/fail, and export.

---

2. Why this artifact exists

Once Build Mode starts, the system needs more than a request record.

It needs a governed way to answer:

- has the build only been requested
- has it been approved to start
- is it actively building
- is it being verified
- did it fail
- did it pass
- was it exported

`build_state` exists to hold that answer.

Without it, Build Mode becomes operationally vague and much harder to govern, audit, resume, or display to the operator.

---

3. Core definition

`build_state` is the formal runtime artifact that records the current lifecycle state of a specific Build Mode attempt, along with the minimum metadata needed to govern transitions, status visibility, and final outcome tracking.

---

4. Core principle

A build should never be “sort of running” or “basically done.”

It should always be in a defined state with a clear meaning.

That is the main principle behind this artifact.

---

5. What this artifact represents

`build_state` represents:

- the current position of the build in its lifecycle
- the latest governed status for the build attempt
- the state that the Build Control Plane uses for transition logic
- the operator-visible status of the build path

It does not represent:

- the operator’s original request alone
- the full build plan
- the entire audit trail
- the verification report
- the export record

It is the lifecycle state artifact.

---

6. What this artifact is responsible for capturing

At minimum, `build_state` should capture:

- which build session or run it belongs to
- the current lifecycle state
- when that state was entered
- whether the state is terminal or non-terminal
- the latest status reason or summary
- references to the request and build basis

That is the minimum governance set.

---

7. Required responsibilities

The `build_state` artifact should support these responsibilities.

7.1 Lifecycle visibility

It must tell the system and operator the current official build state.

7.2 Transition control

It must provide the current state needed for allowed/disallowed transition rules.

7.3 Terminal outcome visibility

It must clearly distinguish successful, failed, and exported outcomes.

7.4 Timing traceability

It must preserve when the current state was entered.

7.5 Failure/pass surface

It must give a clean place to attach high-level outcome status and summaries.

---

8. What this artifact does not do

`build_state` should not:

- store the whole build plan
- replace transition history
- replace build phase records
- replace the verification report
- replace the failure report
- replace the export record
- independently decide its own next state

The Build Control Plane governs transitions.
`build_state` records the current governed result.

---

9. Relationship to other artifacts

This artifact sits at the center of runtime Build Mode governance.

Upstream relationship

It depends on:

- `build_request`
- Build Control Plane rules
- run/build context

Downstream relationship

It influences or feeds:

- operator status visibility

- transition records
- phase progression logic
- failure handling
- final closure logic
- export decision flow

So `build_state` is one of the main coordination artifacts in the build branch.

---

10. State model

The minimum state set should be:

- `not_requested`
- `requested`
- `approved`
- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

These are the same governed states already defined for Build Mode.

---

11. State meanings

At minimum, the artifact should support these meanings.

`not_requested`

No build request has been activated for this run.

`requested`

A build request exists, but approval/eligibility progression is not yet complete.

`approved`

The request passed its build-entry checks and may begin execution.

`building`

Repo materialization is actively in progress.

`verifying`

Required build-phase checks are being evaluated.

`failed`

The build path ended unsuccessfully.

`passed`

The build path completed successfully as a repo-generation event.

`exported`

The build passed and the requested export artifact was successfully created.

---

12. Terminal vs non-terminal states

This artifact should distinguish terminal from non-terminal states.

Non-terminal

- `not_requested`
- `requested`

- `approved`
- `building`
- `verifying`

Terminal

- `failed`
- `passed`
- `exported`

This matters for retry, resume, and UI logic later.

---

13. Minimum required fields

At minimum, `build_state` should define fields like:

- `state_id`
- `run_id`
- `build_request_id`
- `build_session_id`
- `current_state`
- `entered_at`
- `is_terminal`
- `status_summary`

- `status_reason_code`
- `build_basis_ref`
- `updated_at`

Optional but useful:

- `previous_state`
- `failure_class`
- `export_requested`

---

14. Recommended field meanings

`state_id`

Unique identifier for the state record.

`run_id`

Originating run ID.

`build_request_id`

Reference to the request that activated the build.

`build_session_id`

Reference to the concrete build attempt/session.

`current_state`

The governed current lifecycle state.

`entered_at`

Timestamp when the current state became active.

`is_terminal`

Boolean indicating whether the current state is terminal.

`status_summary`

Short human-readable summary of the current state.

`status_reason_code`

Machine-usable reason code for the current state or the latest transition.

`build_basis_ref`

Reference to the active Agent Kit or build basis.

`updated_at`

Timestamp of latest state record update.

`previous_state`

Optional prior state reference/value for convenience.

`failure_class`

Optional structured failure category when `current_state = failed`.

`export_requested`

Optional convenience flag for downstream use.

---

15. Suggested enums

`current_state`

- `not_requested`
- `requested`
- `approved`

- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

`status_reason_code`

Examples:

- `request_created`
- `eligibility_pending`
- `eligibility_passed`
- `build_started`
- `verification_started`
- `verification_passed`
- `verification_failed`
- `export_completed`
- `export_failed`

`failure_class`

Examples:

- `missing_kit_input`

- `invalid_build_inputs`
- `eligibility_failure`
- `planning_failure`
- `workspace_failure`
- `repo_generation_failure`
- `verification_failure`
- `export_failure`
- `record_failure`

---

16. Suggested schema shape

A simple logical shape could be:


```
{
 "state_id": "BST-0001",
 "run_id": "RUN-0001",
 "build_request_id": "BRQ-0001",
 "build_session_id": "BSES-0001",
 "current_state": "building",
 "entered_at": "2026-03-06T12:05:00Z",
 "is_terminal": false,
 "status_summary": "Repository generation in progress",
 "status_reason_code": "build_started",
 "build_basis_ref": "KIT-0001",
 "updated_at": "2026-03-06T12:05:00Z",
 "previous_state": "approved",
 "failure_class": null,
 "export_requested": true
}
```


That is enough for a first version.

---

17. Validation rules

At minimum, validation should require:

- `state_id` present
- `run_id` present
- `current_state` present and valid enum
- `entered_at` present
- `is_terminal` consistent with `current_state`
- `updated_at` present
- `build_request_id` present once Build Mode is active
- `failure_class` required if `current_state = failed`

Suggested consistency rules:

- if `current_state = failed`, then `is_terminal = true`
- if `current_state = passed`, then `is_terminal = true`
- if `current_state = exported`, then `is_terminal = true`
- if `current_state` is one of `requested|approved|building|verifying`, then `is_terminal = false`

---

18. Transition behavior

`build_state` should not freeform mutate.

Changes to it should happen only through governed transitions such as:

- request accepted
- eligibility passed/failed
- build started
- verification started
- verification passed/failed
- export completed/failed

That means the state artifact should be treated as control-plane-owned, not casually edited by arbitrary components.

---

19. Relationship to transition records

`build_state` stores the current lifecycle truth.

`build_transition_record` should store the history of how it changed.

That separation is important:

- `build_state` = current snapshot
- `build_transition_record` = historical movement

This keeps both artifacts clean.

---

20. Audit significance

This artifact is important for audit because it shows:

- what the build's current or final lifecycle state was
- when that state became active
- whether the build ended successfully

- what high-level reason explains the state

It is the simplest official answer to “where is this build right now?”

---

21. Success criteria for this artifact

`build_state` is successful when it provides a clear, governed, machine-usable, and operator-visible record of the current Build Mode lifecycle state for a specific run/build attempt, without trying to replace transition history, verification evidence, or failure detail artifacts.

---

22. Short version

BM-A06 — `build_state`
Records the current governed lifecycle status of a Build Mode run or build session. It identifies the originating run and request, stores the active state (`requested`, `approved`, `building`, `verifying`, `failed`, `passed`, or `exported`), records when that state was entered, and provides the official status snapshot used for control-plane decisions, operator visibility, and final outcome tracking.

BM-A09

BM-A09 — `build_plan`

1. Purpose

Define the artifact that turns the approved build basis into a concrete, actionable repo-generation path.

This artifact exists because the Agent Kit and related locked inputs describe what should be built, but Build Mode still needs an execution-ready plan that tells the system how to materialize the repository in a controlled order.

---

2. Why this artifact exists

The Agent Kit is the build-intelligence package, but it is not the same thing as an execution plan.

A build still needs a structured answer to questions like:

- what repo shape are we building
- what phases or slices will be executed
- what files or modules are required
- what order should generation follow
- what checks will matter before completion
- what should block the build if missing

`build_plan` exists to translate upstream approved truth into a governed build path.

Without this artifact, the build agent is forced to infer too much live during execution, which increases drift.

---

3. Core definition

`build_plan` is the formal execution-planning artifact that defines the concrete repo-generation path for a Build Mode run, derived from the approved Agent Kit, locked build inputs, and build profile constraints.

---

4. Core principle

The build should not invent its path while it is already running.

The path should be derived first, then executed.

That is the main principle behind this artifact.

---

5. What this artifact represents

`build_plan` represents:

- the actionable build path for one build attempt
- the translation of approved build intelligence into execution structure
- the controlled order in which repo materialization should happen
- the list of expected build targets and checks needed for completion

It is not:

- the original Agent Kit
- the current lifecycle state
- the verification report
- the export record

It is the execution plan.

---

6. What this artifact is responsible for capturing

At minimum, `build_plan` should capture:

- which build/run it belongs to
- what build basis it was derived from
- what build profile applies
- what repo shape is being targeted
- what phases/slices will run
- what required outputs/files/modules are expected
- what build rules and checks matter for completion

That is the minimum execution model.

---

7. Required responsibilities

The `build_plan` artifact should support these responsibilities.

7.1 Execution path definition

It must define the build path in a concrete enough way that generation can follow it.

7.2 Phase ordering

It must define the controlled order of major build slices.

7.3 Target declaration

It must identify the repo targets the build is expected to materialize.

7.4 Constraint carry-through

It must carry forward the locked profile, repo, and dependency constraints that affect execution.

7.5 Verification handoff

It must make clear what outputs and checks later verification expects.

---

8. What this artifact does not do

`build_plan` should not:

- redefine product scope
- override the approved Agent Kit
- freely invent new repo targets outside locked inputs
- replace low-level file contents
- replace the current build state
- decide final pass/fail on its own

It tells the system what path to execute, not whether the run succeeded in the end.

---

9. Relationship to other artifacts

This artifact sits in the middle of the build branch.

Upstream relationship

It is derived from:

- approved Agent Kit
- locked standards/specs/templates/contracts
- build profile
- repo blueprint if available
- required file inventory if available
- policy and no-guess constraints

Downstream relationship

It feeds:

- workspace setup
- repo generation
- phase records
- verification expectations
- manifests and audit records

So `build_plan` is the operational bridge between approved build basis and actual repo generation.

---

10. Timing in the lifecycle

This artifact should be created after:

- `build_request` exists
- eligibility passes
- the build is approved to proceed

It should exist before:

- workspace preparation
- repo generation
- verification execution

That timing matches the transition from:
`approved` → actionable build execution.

---

11. Planning scope

At minimum, the plan should define the following planning dimensions:

11.1 Build identity scope

What build/run/session this plan belongs to.

11.2 Output scope

What repo output is expected.

11.3 Structure scope

What repo shape, modules, or domains are expected.

11.4 Execution scope

What phases/slices will run and in what order.

11.5 Constraint scope

What no-guess, dependency, and profile rules apply.

11.6 Verification scope handoff

What completion checks should later be satisfied.

---

12. Minimum plan sections

A serious first-version `build_plan` should include at least these sections:

1. plan identity
2. source basis references
3. build profile summary
4. repo target summary
5. phase/slice plan
6. required outputs summary
7. constraints summary
8. verification expectation summary

That gives you a clean and usable planning artifact.

---

13. Minimum required fields

At minimum, `build_plan` should define fields like:

- `plan_id`
- `run_id`
- `build_request_id`
- `build_session_id`
- `build_basis_ref`
- `derived_from_refs`
- `build_profile_ref`
- `repo_blueprint_ref`
- `required_file_inventory_ref`
- `plan_created_at`
- `target_repo_type`
- `target_repo_summary`
- `phases`
- `required_outputs`
- `constraints`
- `verification_expectations`

- `status`

Optional but useful:

- `notes`
- `planner_version`
- `policy_snapshot_ref`

---

14. Recommended field meanings

`plan_id`

Unique identifier for the build plan.

`run_id`

Originating run ID.

`build_request_id`

Reference to the build request that triggered this plan.

`build_session_id`

Reference to the build attempt/session this plan belongs to.

`build_basis_ref`

Primary reference to the approved Agent Kit.

`derived_from_refs`

List of supporting locked artifacts used to derive the plan.

Examples:

- standards snapshot

- template selections
- route specs
- entity specs
- schema specs
- flow specs

`build_profile_ref`

Reference to the locked stack/profile definition.

`repo_blueprint_ref`

Reference to the expected repo structure artifact if one exists.

`required_file_inventory_ref`

Reference to required file inventory if one exists.

`plan_created_at`

Timestamp of plan creation.

`target_repo_type`

High-level repo type or stack form.

Examples:

- `web_app`
- `mobile_app`
- `api_service`
- `fullstack_monorepo`

`target_repo_summary`

Short human-readable summary of what repo is being built.

`phases`

Ordered list of build phases/slices and their scope.

`required_outputs`

Summary of what the build must materialize.

`constraints`

Build-time constraints that generation must obey.

`verification_expectations`

Summary of checks or conditions the generated repo must later satisfy.

`status`

Optional plan-local status.

Examples:

- `draft`
- `active`
- `superseded`

---

15. Phase model inside the plan

The `phases` field should be one of the most important parts.

A minimum first-version phase sequence could be:

1. `prepare_workspace`
2. `create_repo_skeleton`
3. `materialize_core_config`

4. `materialize_shared_contracts`
5. `materialize_backend`
6. `materialize_frontend`
7. `materialize_tests_and_supporting_files`
8. `write_manifests`
9. `handoff_to_verification`

You may later vary this by build profile, but the first version should still keep an ordered phase list.

Each phase entry should ideally identify:

- phase ID
- purpose
- target areas
- entry dependencies
- expected outputs

---

16. Required outputs section

The `required_outputs` section should summarize the materialization targets for the build.

At minimum, it should be able to say things like:

- repo root must exist
- required module areas must exist
- required config/manifests must exist

- required shared/backend/frontend/test/docs targets must exist as applicable
- required build artifacts for completion must exist

This section does not need to store every single line of content, but it should declare what the build is responsible for producing.

---

17. Constraints section

The `constraints` section should carry forward the rules that keep the plan low-drift.

At minimum, it should support constraints like:

- no undeclared scope expansion
- no undeclared dependency introduction
- no renaming of locked identifiers
- stop on missing required inputs
- stop on conflicting locked inputs
- block completion on critical unresolved placeholders

This helps the Repo Generator know the rules of execution before generation starts.

---

18. Verification expectations section

The `verification_expectations` section should tell downstream verification what classes of conditions the plan expects the final repo to satisfy.

At minimum, this should include:

- required output presence
- repo structure validity

- placeholder resolution expectations
- manifest completeness expectations
- command/check expectations
- export readiness expectations if export was requested

This makes the plan connect directly to later verification.

---

19. Suggested schema shape

A simplified logical shape could be:


```
{
 "plan_id": "BPL-0001",
 "run_id": "RUN-0001",
 "build_request_id": "BRQ-0001",
 "build_session_id": "BSES-0001",
 "build_basis_ref": "KIT-0001",
 "derived_from_refs": [
   "STD-0001",
   "ROUTE-SPEC-0001",
   "ENTITY-SPEC-0001"
 ],
 "build_profile_ref": "PROFILE-0001",
 "repo_blueprint_ref": "RBP-0001",
 "required_file_inventory_ref": "RFI-0001",
 "plan_created_at": "2026-03-06T12:03:00Z",
 "target_repo_type": "fullstack_monorepo",
 "target_repo_summary": "Generate a fullstack monorepo with frontend, backend, shared
contracts, tests, and supporting config files.",
 "phases": [
   { "phase_id": "prepare_workspace", "order": 1 },
   { "phase_id": "create_repo_skeleton", "order": 2 },
   { "phase_id": "materialize_core_config", "order": 3 },
   { "phase_id": "materialize_shared_contracts", "order": 4 },
   { "phase_id": "materialize_backend", "order": 5 },
   { "phase_id": "materialize_frontend", "order": 6 },
   { "phase_id": "materialize_tests_and_supporting_files", "order": 7 },
   { "phase_id": "write_manifests", "order": 8 },
   { "phase_id": "handoff_to_verification", "order": 9 }
```

```
  ],
  "required_outputs": {
    "repo_root_required": true,
    "manifests_required": true,
    "verification_required": true
  },
  "constraints": {
    "allow_scope_expansion": false,
    "allow_undeclared_dependencies": false,
    "block_on_missing_required_inputs": true,
    "block_on_critical_placeholders": true
  },
  "verification_expectations": {
    "require_structure_check": true,
    "require_placeholder_scan": true,
    "require_manifest_check": true
  },
  "status": "active"
}
```

That is enough for a first serious version.

---

20. Validation rules

At minimum, validation should require:

- `plan_id` present
- `run_id` present
- `build_basis_ref` present
- `build_profile_ref` present
- `plan_created_at` present
- `phases` present and ordered
- `constraints` present
- `verification_expectations` present

- `required_outputs` present

Suggested consistency rules:

- phase order must be unique and increasing
- required output expectations must align with requested outcome
- export-related expectations should not be required unless export was requested or profile policy makes them necessary
- constraints must not conflict with policy snapshot

---

21. Audit significance

This artifact is important for audit because it proves:

- what execution path the system intended to follow
- what locked sources informed that path
- what the build was expected to produce
- what rules applied before generation started

It is one of the key anti-drift artifacts because it makes the build path inspectable before generation happens.

---

22. Success criteria for this artifact

`build_plan` is successful when it gives Build Mode a concrete, traceable, constraint-aware repo-generation path derived from the approved build basis, with explicit phase order, required outputs, and verification expectations, without redefining product scope or leaving the generation path to live improvisation.

---

23. Short version

BM-A09 — `build_plan`

Defines the actionable repo-generation path for a Build Mode run. It is derived from the approved Agent Kit and other locked build inputs, and it specifies the target repo type, ordered build phases, required outputs, execution constraints, and verification expectations. It is the bridge between approved build intelligence and controlled repo materialization.

BM-A16

BM-A16 — `build_manifest`

1. Purpose

Define the artifact that records the build event as a governed output of Build Mode.

This artifact exists so every build attempt leaves a formal record of what was built, from what approved basis, under what build profile, with what outcome, and with what produced artifacts. It is the main summary record for the build as an execution event.

---

2. Why this artifact exists

A generated repo alone is not enough.

Build Mode needs a central artifact that answers:

- what build happened
- what run it belonged to
- what build basis it used
- what profile and plan were applied
- what state/outcome the build reached
- what outputs were produced
- when the build started and finished

`build_manifest` exists to provide that summary.

Without it, the system has code outputs and scattered records, but no single authoritative build event document.

---

3. Core definition

`build_manifest` is the formal output artifact that summarizes one Build Mode execution event, including its identity, lineage, build basis, execution references, outcome, and produced artifacts.

---

4. Core principle

Every build should leave behind one canonical summary artifact that explains the build as a governed event.

That is the main principle behind this artifact.

---

5. What this artifact represents

`build_manifest` represents:

- the official summary of a build attempt
- the build's identity and lineage
- the build's relationship to the originating run and build basis
- the build's final recorded outcome
- the produced output set at a summary level

It is not:

- the current lifecycle state snapshot
- the full transition history
- the repo manifest
- the verification report
- the export record
- the failure report

It is the build event summary artifact.

---

6. What this artifact is responsible for capturing

At minimum, `build_manifest` should capture:

- build identity
- run/build request/session linkage
- build basis reference
- build plan/profile references
- lifecycle outcome
- produced output references
- timing summary
- export result summary if applicable

That is the minimum serious summary set.

---

7. Required responsibilities

The `build_manifest` artifact should support these responsibilities.

7.1 Build event summary

It must summarize the build as one governed event.

7.2 Lineage recording

It must tie the build to its originating run, request, session, and build basis.

7.3 Output summary

It must identify the major outputs the build produced.

7.4 Outcome summary

It must record whether the build failed, passed, or exported.

7.5 Audit anchor

It must give the system and operator a canonical summary document for the build.

---

8. What this artifact does not do

`build_manifest` should not:

- replace the live `build_state`
- replace the detailed `verification_report`
- replace the detailed `failure_report`
- define the repo structure in depth
- replace the export artifact record
- replace low-level transition logs

It is a summary record, not every record.

---

9. Relationship to other artifacts

This artifact sits in the output/proof layer.

Upstream relationship

It is populated from:

- `build_request`
- `build_state`
- `build_plan`

- `build_session`
- `verification_report`
- `export_record`
- `failure_report` if applicable

Downstream relationship

It supports:

- audit review
- operator summary views
- retention and artifact indexing
- handoff traceability
- maintenance/lifecycle management later

So `build_manifest` is a key anchor artifact after generation and verification.

---

10. Timing in the lifecycle

This artifact should generally be:

- initialized during build execution once the build is active
- updated as the build progresses through major milestones
- finalized at completion or failure

It should exist before final closure is considered complete.

That means it becomes one of the required output artifacts for a successful build, and still exists in some form for a failed one.

---

11. Manifest scope

At minimum, the manifest should summarize these areas:

11.1 Identity

What build this is.

11.2 Lineage

What run and upstream build basis it came from.

11.3 Execution basis

What plan/profile/governance basis shaped the build.

11.4 Outcome

What final result the build reached.

11.5 Output set

What major outputs were produced.

11.6 Timing

When the build started and ended.

That gives you a clean and comprehensive summary without turning the artifact into a dump of everything.

---

12. Minimum required fields

At minimum, `build_manifest` should define fields like:

- `build_manifest_id`
- `run_id`
- `build_request_id`
- `build_session_id`

- `build_basis_ref`
- `build_plan_ref`
- `build_profile_ref`
- `policy_snapshot_ref`
- `started_at`
- `completed_at`
- `final_state`
- `result`
- `export_requested`
- `export_completed`
- `repo_ref`
- `repo_manifest_ref`
- `verification_report_ref`
- `export_record_ref`
- `failure_report_ref`
- `status_summary`
- `created_at`
- `updated_at`

Optional but useful:

- `build_control_plane_contract_ref`
- `phase_summary`
- `notes`

---

13. Recommended field meanings

`build_manifest_id`

Unique identifier for the build manifest.

`run_id`

Originating run ID.

`build_request_id`

Reference to the request that activated Build Mode.

`build_session_id`

Reference to the concrete build attempt.

`build_basis_ref`

Reference to the approved Agent Kit.

`build_plan_ref`

Reference to the concrete build plan used.

`build_profile_ref`

Reference to the locked build profile.

`policy_snapshot_ref`

Reference to the effective build/export policy snapshot used for this build.

`started_at`

Timestamp when active build execution began.

`completed_at`

Timestamp when the build reached final closure.

`final_state`

Final lifecycle state.

Suggested enum:

- `failed`
- `passed`
- `exported`

`result`

High-level outcome summary.

Suggested enum:

- `failure`
- `success`
- `success_with_export`

`export_requested`

Whether the requested outcome included export.

`export_completed`

Whether export actually completed.

`repo_ref`

Reference/location for the generated repository.

`repo_manifest_ref`

Reference to `repo_manifest`.

`verification_report_ref`

Reference to `verification_report`.

`export_record_ref`

Reference to `export_record` if present.

`failure_report_ref`

Reference to `failure_report` if present.

`status_summary`

Short human-readable summary of the build outcome.

`created_at`

Timestamp manifest record was initialized.

`updated_at`

Timestamp manifest record was last updated.

`build_control_plane_contract_ref`

Optional reference to the governing contract version.

`phase_summary`

Optional compact summary of major phase results.

---

14. Suggested enums

`final_state`

- `failed`

- `passed`
- `exported`

`result`

- `failure`
- `success`
- `success_with_export`

---

15. Suggested schema shape

A simplified logical shape could be:


```
{
 "build_manifest_id": "BMAN-0001",
 "run_id": "RUN-0001",
 "build_request_id": "BRQ-0001",
 "build_session_id": "BSES-0001",
 "build_basis_ref": "KIT-0001",
 "build_plan_ref": "BPL-0001",
 "build_profile_ref": "PROFILE-0001",
 "policy_snapshot_ref": "BPS-0001",
 "started_at": "2026-03-06T12:05:00Z",
 "completed_at": "2026-03-06T12:19:00Z",
 "final_state": "exported",
 "result": "success_with_export",
 "export_requested": true,
 "export_completed": true,
 "repo_ref": "runs/RUN-0001/build/repo",
 "repo_manifest_ref": "RMAN-0001",
 "verification_report_ref": "VREP-0001",
 "export_record_ref": "EXPR-0001",
 "failure_report_ref": null,
 "status_summary": "Build completed successfully and export zip was created.",
 "created_at": "2026-03-06T12:05:00Z",
 "updated_at": "2026-03-06T12:19:00Z",
```

```
 "build_control_plane_contract_ref": "BM-A01@1.0.0",
 "phase_summary": {
   "prepare_workspace": "passed",
   "create_repo_skeleton": "passed",
   "materialize_backend": "passed",
   "materialize_frontend": "passed",
   "verification": "passed",
   "export": "passed"
 },
 "notes": null
}
```


That is enough for a strong first version.

---

16. Validation rules

At minimum, validation should require:

- `build_manifest_id` present
- `run_id` present
- `build_request_id` present
- `build_basis_ref` present
- `build_plan_ref` present
- `build_profile_ref` present
- `final_state` present once finalized
- `result` present once finalized
- `created_at` present
- `updated_at` present

Suggested consistency rules:

- if `final_state = exported`, then `export_requested = true` and `export_completed = true`
- if `result = failure`, then `failure_report_ref` should exist
- if `result = success`, then `final_state = passed`
- if `result = success_with_export`, then `final_state = exported`
- if build completed successfully, `verification_report_ref` should exist
- if repo was generated, `repo_ref` should exist

---

17. Relationship to successful and failed builds

`build_manifest` should exist for both successful and failed runs.

For successful builds

It should summarize:

- success result
- repo output references
- verification success
- export result if applicable

For failed builds

It should summarize:

- failure result
- failure-linked references
- partial output refs if relevant

- final failed state

That way every build attempt has one canonical summary artifact.

---

18. Relationship to audit

This artifact is one of the main audit anchors because it gives a single summary answer to:

- what build happened
- what it used
- what it produced
- how it ended

Other artifacts provide detail, but `build_manifest` provides the build-level summary view.

---

19. Relationship to output contract

This artifact should be part of the required Build Outputs Contract for successful builds.

That means a build should not count as fully complete if the repo exists but `build_manifest` does not.

That keeps the build result governable instead of purely file-based.

---

20. Success criteria for this artifact

`build_manifest` is successful when it provides a canonical, traceable, and machine-usable summary of the build event, linking the originating run, request, build basis, plan, outcome, and produced outputs into one governed build record without replacing more detailed artifacts like verification or failure reports.

---

21. Short version

BM-A16 — `build_manifest`

Summarizes one Build Mode execution event as a governed output artifact. It records the originating run, build request, build basis, plan/profile references, timing, final outcome, and the major produced artifacts such as the generated repo, repo manifest, verification report, export record, and failure report where applicable. It is the canonical build-level summary record for audit, status review, and output traceability.

BM-A17

BM-A17 — `repo_manifest`

1. Purpose

Define the artifact that describes the generated repository as an output object.

This artifact exists so Build Mode does not only say “a repo was created,” but can also describe what that repo contains at a structural level. It is the primary summary artifact for the repository itself, separate from the build event summary.

---

2. Why this artifact exists

`build_manifest` summarizes the build event.

But Build Mode also needs a focused artifact that answers repo-specific questions like:

- what kind of repo was generated
- what major areas or modules it contains
- what structure it follows
- what required parts are present
- what commands/checks are associated with it
- what output shape the operator is receiving

`repo_manifest` exists to provide that repository-centered view.

Without it, the system knows a build happened, but has a weaker description of the repo that came out of it.

---

3. Core definition

`repo_manifest` is the formal output artifact that describes the generated repository’s structure, composition, major contents, and expected operational metadata as a Build Mode output object.

---

4. Core principle

The generated repository should be describable as a governed artifact, not just as a folder path.

That is the main principle behind this artifact.

---

5. What this artifact represents

`repo_manifest` represents:

- the generated repository as an output object
- the repo's structure and composition
- the major modules/areas present in the repo
- the repo-level metadata needed for inspection and handoff

It is not:

- the build event summary
- the current lifecycle state
- the full file index
- the detailed verification report
- the export record

It is the repository summary artifact.

---

6. What this artifact is responsible for capturing

At minimum, `repo_manifest` should capture:

- which build/run the repo belongs to
- what type of repo was generated
- where the repo lives
- what major areas/modules it contains
- what required structural elements are present
- what commands/checks are associated with it
- what summary of completeness applies

That is the minimum serious repo summary set.

---

7. Required responsibilities

The `repo_manifest` artifact should support these responsibilities.

7.1 Repo identity

It must identify the generated repo as a distinct output artifact.

7.2 Repo structure summary

It must summarize the structural form of the repo.

7.3 Module/composition summary

It must show the major areas that were materialized.

7.4 Operational handoff metadata

It must carry the basic repo-level metadata the operator or later system components need for inspection.

7.5 Verification support

It must provide a stable structural summary that verification and audit can compare against.

---

8. What this artifact does not do

`repo_manifest` should not:

- replace the actual repo
- replace a detailed file index
- replace the build plan
- replace verification results
- replace low-level dependency manifests like package files
- replace the export zip

It is a repo summary, not the repo itself.

---

9. Relationship to other artifacts

This artifact sits beside the build manifest in the output/proof layer.

Upstream relationship

It is populated from:

- `build_plan`
- `repo_blueprint`
- generated repo outputs
- required file inventory
- workspace/output paths
- verification results as needed for summary status

Downstream relationship

It supports:

- operator inspection
- audit and traceability
- verification interpretation
- export packaging decisions
- maintenance/lifecycle processes later

So `repo_manifest` is one of the main descriptive artifacts of the generated output.

---

10. Timing in the lifecycle

This artifact should generally be:

- initialized once repo shape becomes real
- updated as major repo areas are materialized
- finalized before or during verification completion

It should exist as part of the required successful output set.

That means the repo should not count as cleanly complete if the repo exists but `repo_manifest` does not.

---

11. Manifest scope

At minimum, the manifest should summarize these areas:

11.1 Repo identity

What repo output this is.

11.2 Repo type and structure

What general structural form it follows.

11.3 Major components/modules

What major areas exist in the repo.

11.4 Required structure presence

Whether the required structural pieces are present.

11.5 Repo-level operational metadata

What commands, root files, or usage metadata are associated with it.

This keeps the artifact useful without turning it into a file-by-file dump.

---

12. Minimum required fields

At minimum, `repo_manifest` should define fields like:

- `repo_manifest_id`
- `run_id`
- `build_manifest_id`
- `build_session_id`
- `repo_ref`
- `repo_type`
- `repo_shape`
- `repo_root_path`
- `module_areas`
- `required_structure_summary`
- `root_artifacts`
- `command_summary`

- `build_profile_ref`
- `repo_blueprint_ref`
- `required_file_inventory_ref`
- `completeness_status`
- `created_at`
- `updated_at`

Optional but useful:

- `workspace_manifest_ref`
- `file_index_ref`
- `notes`

---

13. Recommended field meanings

`repo_manifest_id`

Unique identifier for the repo manifest.

`run_id`

Originating run ID.

`build_manifest_id`

Reference to the build manifest that produced this repo.

`build_session_id`

Reference to the concrete build attempt/session.

`repo_ref`

Reference/location for the generated repo artifact.

`repo_type`

High-level repo type.

Examples:

- `web_app`
- `mobile_app`
- `api_service`
- `fullstack_monorepo`

`repo_shape`

Human/machine-usable summary of the repo structure style.

Examples:

- `single_app`
- `client_server_split`
- `monorepo_with_shared`
- `mobile_plus_backend`

`repo_root_path`

The root location/path of the generated repo.

`module_areas`

List of major module or structural areas present in the repo.

Examples:

- `frontend`
- `backend`
- `shared`
- `tests`
- `docs`
- `config`

`required_structure_summary`

Summary of whether required structural areas are present.

`root_artifacts`

Summary list of important top-level repo artifacts.

Examples:

- `package.json`
- `tsconfig.json`
- `.env.example`
- `README.md`

`command_summary`

Summary of repo-level commands/checks associated with the output.

Examples:

- install command
- dev/start command

- typecheck
- test
- build

`build_profile_ref`

Reference to the locked build profile.

`repo_blueprint_ref`

Reference to the expected repo blueprint if one exists.

`required_file_inventory_ref`

Reference to required file inventory if one exists.

`completeness_status`

Repo-summary-level completeness classification.

Suggested enum:

- `partial`
- `structurally_complete`
- `verified_complete`

`created_at`

Timestamp repo manifest was initialized.

`updated_at`

Timestamp repo manifest was last updated.

`workspace_manifest_ref`

Optional reference to workspace/staging layout artifact.

`file_index_ref`

Optional reference to a detailed file index artifact.

---

14. Suggested enums

`repo_type`

Examples:

- `web_app`
- `mobile_app`
- `api_service`
- `fullstack_monorepo`

`repo_shape`

Examples:

- `single_app`
- `client_server_split`
- `monorepo_with_shared`
- `mobile_plus_backend`

`completeness_status`

- `partial`
- `structurally_complete`
- `verified_complete`

---

15. Suggested schema shape

A simplified logical shape could be:


```
{
 "repo_manifest_id": "RMAN-0001",
 "run_id": "RUN-0001",
 "build_manifest_id": "BMAN-0001",
 "build_session_id": "BSES-0001",
 "repo_ref": "runs/RUN-0001/build/repo",
 "repo_type": "fullstack_monorepo",
 "repo_shape": "monorepo_with_shared",
 "repo_root_path": "runs/RUN-0001/build/repo",
 "module_areas": [
   "frontend",
   "backend",
   "shared",
   "tests",
   "docs",
   "config"
 ],
 "required_structure_summary": {
   "frontend_present": true,
   "backend_present": true,
   "shared_present": true,
   "tests_present": true,
   "docs_present": true,
   "config_present": true
 },
 "root_artifacts": [
   "package.json",
   "tsconfig.json",
   ".env.example",
   "README.md"
 ],
 "command_summary": {
   "install": "npm install",
   "dev": "npm run dev",
   "typecheck": "npm run typecheck",
   "test": "npm test",
   "build": "npm run build"
 },
 "build_profile_ref": "PROFILE-0001",
 "repo_blueprint_ref": "RBP-0001",
```

"required_file_inventory_ref": "RFI-0001",
"completeness_status": "verified_complete",
"created_at": "2026-03-06T12:10:00Z",
"updated_at": "2026-03-06T12:19:00Z",
"workspace_manifest_ref": "WMAN-0001",
"file_index_ref": "FINDEX-0001",
"notes": null
}

That is enough for a strong first version.

---

16. Validation rules

At minimum, validation should require:

- `repo_manifest_id` present
- `run_id` present
- `repo_ref` present
- `repo_type` present
- `repo_shape` present
- `module_areas` present
- `required_structure_summary` present
- `build_profile_ref` present
- `created_at` present
- `updated_at` present

Suggested consistency rules:

- `completeness_status = verified_complete` should only be allowed once verification has passed

- required structural areas should align with the build profile and build plan
- module areas should not claim major sections that do not actually exist
- root artifacts should align with repo type/profile expectations

---

17. Relationship to verification

`repo_manifest` is descriptive, but it also supports verification.

Why?

Because it gives the system a declared summary of what the repo is supposed to look like once generated. Verification can compare the actual repo against:

- required structure summary
- module areas
- root artifacts
- command summary expectations

So it is a useful bridge between generation and validation.

---

18. Relationship to output contract

This artifact should be part of the required Build Outputs Contract for successful builds.

That means successful completion should require more than:

- repo folder exists

It should also require:

- repo summary exists
- repo summary is linked to the run/build

- repo summary reflects the generated output coherently

That helps make the repo output inspectable.

---

19. Audit significance

This artifact is important for audit because it answers:

- what repo did this build produce
- what major structure did it contain
- what kind of output did the operator receive

It complements `build_manifest` by focusing on the repo rather than the whole build event.

---

20. Success criteria for this artifact

`repo_manifest` is successful when it provides a clear, traceable, and structurally useful summary of the generated repository as an output object, including repo type, structure shape, major module areas, required structure presence, and key operational metadata, without trying to replace the repo itself or the more detailed file index and verification artifacts.

---

21. Short version

BM-A17 — `repo_manifest`
Describes the generated repository as a governed output artifact. It records the repo's identity, type, structure shape, major module areas, required structural presence, key root artifacts, and command summary, and links that description back to the originating run and build. It is the primary summary artifact for the repo itself, separate from the overall build event summary.

BM-A19

BM-A19 — `verification_report`

1. Purpose

Define the artifact that records whether the generated repository passed the required Build Mode checks.

This artifact exists so Build Mode has a formal proof record for validation. It is the document that says what was checked, what passed, what failed, and whether the build is allowed to count as complete and, if requested, to move on to export.

---

2. Why this artifact exists

A generated repo should not be treated as valid just because files exist.

Build Mode needs a structured artifact that answers:

- what verification was run
- what categories were checked
- what the results were
- what blocked completion if anything
- whether the repo passed
- whether export is now allowed

`verification_report` exists to provide that answer.

Without it, completion becomes subjective and export becomes easier to trust than it should be.

---

3. Core definition

`verification_report` is the formal output artifact that records the build-phase verification results for a generated repository, including the checks performed, their outcomes, blocking findings, and the final pass/fail decision for Build Mode completion.

---

4. Core principle

Build completion must be proven, not assumed.

That is the main principle behind this artifact.

---

5. What this artifact represents

`verification_report` represents:

- the formal result of Build Mode verification
- the proof that required checks were evaluated
- the pass/fail decision for build validity
- the basis for allowing or blocking export

It is not:

- the build plan
- the repo manifest
- the current live build state
- the export record
- the failure report as a whole

It is the verification truth artifact.

---

6. What this artifact is responsible for capturing

At minimum, `verification_report` should capture:

- what build/run it belongs to
- what repo/build outputs were verified
- what categories/checks were run
- pass/fail result per category or check
- blocking findings
- final verification result
- export readiness result

That is the minimum serious verification record.

---

7. Required responsibilities

The `verification_report` artifact should support these responsibilities.

7.1 Verification evidence

It must prove that Build Mode verification actually occurred.

7.2 Result recording

It must record the pass/fail outcome of required checks.

7.3 Blocking finding visibility

It must identify what blocked completion if verification failed.

7.4 Completion decision support

It must support the transition from `verifying` to `passed` or `failed`.

7.5 Export readiness support

It must indicate whether export may proceed.

---

8. What this artifact does not do

`verification_report` should not:

- replace the detailed command output log if you store one separately
- replace the current build state
- replace the full failure report
- redefine the verification scope on its own
- act as the repo manifest
- act as the export artifact record

It records verification results, not every surrounding concern.

---

9. Relationship to other artifacts

This artifact sits in the output/proof layer and is one of the main completion gates.

Upstream relationship

It is derived from:

- `build_plan`
- generated repo
- `repo_manifest`

- `verification_command_set`
- structural checks
- placeholder scans
- manifest checks
- command/check results

Downstream relationship

It feeds:

- `build_state`
- `build_manifest`
- export eligibility
- failure handling
- audit review
- operator verification visibility

So `verification_report` is a central handoff artifact between generation and completion.

---

10. Timing in the lifecycle

This artifact should generally be:

- initialized when Build Mode enters `verifying`
- updated as verification categories/checks complete
- finalized before the build may move to `passed` or `failed`

It should exist as part of the required successful output set, and also for failed verification outcomes.

---

11. Verification scope sections

A strong first-version `verification_report` should capture at least these sections:

1. verification identity
2. verified target references
3. verification categories/checks
4. findings
5. final verification decision
6. export readiness decision
7. timing summary

That gives you a complete but controlled artifact.

---

12. Minimum verification categories

At minimum, the report should be able to record results for these categories:

12.1 Required output presence

Did the required outputs exist?

12.2 Repo structure correctness

Did the repo match the expected structure?

12.3 Placeholder / unresolved gap scan

Did critical unresolved placeholders remain?

12.4 Manifest and record completeness

Did required manifests/records exist and validate?

12.5 Required command/check execution

Did required executable/static checks pass?

12.6 Export readiness

Is the build eligible to move on to export if export was requested?

These match the Build Mode verification scope you already defined.

---

13. Minimum required fields

At minimum, `verification_report` should define fields like:

- `verification_report_id`
- `run_id`
- `build_manifest_id`
- `build_session_id`
- `build_plan_ref`
- `repo_ref`

- `repo_manifest_ref`
- `verification_started_at`
- `verification_completed_at`
- `verification_scope_version`
- `categories`
- `findings`
- `final_result`
- `blocking_failure_count`
- `export_ready`
- `command_results_ref`
- `created_at`
- `updated_at`

Optional but useful:

- `verification_command_set_ref`
- `notes`
- `summary`

---

14. Recommended field meanings

`verification_report_id`

Unique identifier for the verification report.

`run_id`

Originating run ID.

`build_manifest_id`

Reference to the related build manifest.

`build_session_id`

Reference to the concrete build attempt.

`build_plan_ref`

Reference to the plan whose expectations were verified.

`repo_ref`

Reference to the generated repo that was checked.

`repo_manifest_ref`

Reference to the repo manifest describing the checked repo.

`verification_started_at`

Timestamp when verification began.

`verification_completed_at`

Timestamp when verification concluded.

`verification_scope_version`

Version/ref for the verification scope rules applied.

`categories`

Ordered list or object describing verification categories and their results.

`findings`

Structured list of findings discovered during verification.

`final_result`

Overall verification outcome.

Suggested enum:

- `passed`
- `failed`

`blocking_failure_count`

Number of blocking failures found.

`export_ready`

Boolean indicating whether export may proceed.

`command_results_ref`

Reference to detailed command/check outputs if stored separately.

`created_at`

Timestamp report record was initialized.

`updated_at`

Timestamp report was last updated.

`verification_command_set_ref`

Optional reference to the command set definition used.

`summary`

Optional short human-readable verification summary.

---

15. Category result model

Each category entry should ideally capture:

- `category_id`
- `category_name`
- `required`
- `status`
- `blocking`
- `checks_run`
- `summary`

Suggested `status` enum:

- `passed`
- `failed`
- `skipped`
- `not_applicable`

Examples of `category_id`:

- `required_output_presence`
- `repo_structure_correctness`
- `placeholder_scan`

- `manifest_completeness`
- `required_command_execution`
- `export_readiness`

---

16. Findings model

The `findings` section should capture the problems or notable results discovered during verification.

Each finding should ideally include:

- `finding_id`
- `severity`
- `category_id`
- `code`
- `message`
- `blocking`
- `target_ref`

Suggested `severity` values:

- `info`
- `warning`
- `error`

Suggested `blocking` behavior:

- blocking findings prevent `final_result = passed`

This helps separate raw category status from detailed issues.

---

17. Suggested schema shape

A simplified logical shape could be:

```
{
  "verification_report_id": "VREP-0001",
  "run_id": "RUN-0001",
  "build_manifest_id": "BMAN-0001",
```

"build_session_id": "BSES-0001",
"build_plan_ref": "BPL-0001",
"repo_ref": "runs/RUN-0001/build/repo",
"repo_manifest_ref": "RMAN-0001",
"verification_started_at": "2026-03-06T12:15:00Z",
"verification_completed_at": "2026-03-06T12:18:00Z",
"verification_scope_version": "BM-13@1.0.0",
"categories": [
  {
    "category_id": "required_output_presence",
    "category_name": "Required Output Presence",
    "required": true,
    "status": "passed",
    "blocking": false,
    "checks_run": 8,
    "summary": "All required output artifacts were present."
  },
  {
    "category_id": "repo_structure_correctness",
    "category_name": "Repo Structure Correctness",
    "required": true,
    "status": "passed",
    "blocking": false,
    "checks_run": 6,
    "summary": "Repo structure matched the expected profile."
  },
  {
    "category_id": "placeholder_scan",
    "category_name": "Placeholder Scan",
    "required": true,
    "status": "passed",
    "blocking": false,
    "checks_run": 3,
    "summary": "No blocking placeholders remained."
  },
  {
    "category_id": "manifest_completeness",
    "category_name": "Manifest Completeness",
    "required": true,
    "status": "passed",
    "blocking": false,
    "checks_run": 4,
    "summary": "Required manifests were complete."
  },

```
    {
      "category_id": "required_command_execution",
      "category_name": "Required Command Execution",
      "required": true,
      "status": "passed",
      "blocking": false,
      "checks_run": 3,
      "summary": "Required commands completed successfully."
    },
    {
      "category_id": "export_readiness",
      "category_name": "Export Readiness",
      "required": true,
      "status": "passed",
      "blocking": false,
      "checks_run": 2,
      "summary": "Build is eligible for export."
    }
  ],
  "findings": [],
  "final_result": "passed",
  "blocking_failure_count": 0,
  "export_ready": true,
  "command_results_ref": "CMDR-0001",
  "created_at": "2026-03-06T12:15:00Z",
  "updated_at": "2026-03-06T12:18:00Z",
  "verification_command_set_ref": "VCS-0001",
  "summary": "All required verification categories passed; export is allowed."
}
```


That is enough for a strong first version.

---

18. Validation rules

At minimum, validation should require:

- `verification_report_id` present
- `run_id` present
- `repo_ref` present
- `verification_started_at` present
- `categories` present

- `final_result` present once finalized
- `blocking_failure_count` present once finalized
- `export_ready` present once finalized
- `created_at` present
- `updated_at` present

Suggested consistency rules:

- if any required blocking category has `status = failed`, then `final_result = failed`
- if `final_result = passed`, then `blocking_failure_count = 0`
- if `export_ready = true`, then `final_result` should normally be `passed`
- `verification_completed_at` should exist once the report is final
- `command_results_ref` should exist if command execution is part of the required scope

---

19. Relationship to failure

A failed verification report should still be complete enough to explain failure.

That means when `final_result = failed`, the artifact should still clearly show:

- which category failed
- what findings were blocking
- whether export is blocked
- whether command or structural checks caused the failure

This keeps verification failure inspectable instead of opaque.

---

20. Relationship to output contract

This artifact should be part of the required Build Outputs Contract for successful builds.

That means a repo is not fully successful unless:

- verification was performed
- a verification report exists
- the report shows the required pass result

That protects Build Mode from equating generation with completion.

---

21. Audit significance

This artifact is one of the main proof artifacts in Build Mode because it answers:

- what was checked
- what passed
- what failed
- whether the repo was actually valid
- whether export was allowed

It is the strongest single artifact for proving that a build earned completion rather than merely reaching it.

---

22. Success criteria for this artifact

`verification_report` is successful when it provides a clear, structured, and traceable record of the verification categories and checks run against the generated repository, including their outcomes, blocking findings, final pass/fail result, and export readiness decision, without trying to replace more detailed command logs or broader failure artifacts.

---

23. Short version

BM-A19 — `verification_report`
Records the formal verification result for a generated Build Mode repository. It identifies the repo and build it applies to, lists the verification categories and checks that were run, records their outcomes and any blocking findings, and declares the final pass/fail decision and export readiness result. It is the main proof artifact that a generated repo has actually satisfied the required Build Mode completion checks.

BM-A21

BM-A21 — `export_record`

1. Purpose

Define the artifact that records the packaging and export outcome for a Build Mode run.

This artifact exists so export is treated as its own governed event rather than being implied by build success. It records whether export was requested, whether export was allowed, what was packaged, what artifact was produced, and whether the export step succeeded or failed.

---

2. Why this artifact exists

A build can pass without being exported.

And a build can also pass but fail during packaging.

So Build Mode needs a dedicated artifact that answers:

- was export requested
- was export allowed
- what was packaged
- where is the exported artifact
- did export succeed
- if it failed, why

`export_record` exists to preserve that distinction.

Without it, export either gets buried inside the build manifest or gets mistaken for something automatic.

---

3. Core definition

`export_record` is the formal output artifact that records the packaging and export event for a Build Mode run, including export intent, export eligibility, package contents summary, produced export artifact reference, and final export result.

---

4. Core principle

Export is a downstream governed event, not an assumed side effect of build success.

That is the main principle behind this artifact.

---

5. What this artifact represents

`export_record` represents:

- the export step as its own governed event
- the packaging decision and result
- the portable handoff artifact produced by the build
- the proof that export succeeded or failed

It is not:

- proof that the repo itself is valid by itself
- the verification report
- the build manifest
- the zip file itself
- the full package contents inventory

It is the export-event record.

---

6. What this artifact is responsible for capturing

At minimum, `export_record` should capture:

- what build/run it belongs to
- whether export was requested
- whether export was allowed
- when export started and ended
- what was packaged at a summary level
- where the exported artifact is
- whether export succeeded or failed
- what blocked export if it failed

That is the minimum serious export record set.

---

7. Required responsibilities

The `export_record` artifact should support these responsibilities.

7.1 Export event recording

It must record export as a distinct lifecycle event.

7.2 Eligibility traceability

It must show whether export was actually allowed before packaging ran.

7.3 Artifact traceability

It must point to the portable export artifact produced.

7.4 Result recording

It must record export success or failure.

7.5 Failure visibility

It must show why export failed when it did not complete.

---

8. What this artifact does not do

`export_record` should not:

- replace the build manifest
- replace the verification report
- replace the zip file
- store the entire repo contents
- redefine whether the build passed
- replace a broader failure report if export failure is part of one

It records export, not the whole build.

---

9. Relationship to other artifacts

This artifact sits late in the output/proof layer.

Upstream relationship

It depends on:

- `build_manifest`
- `verification_report`
- export request intent from `build_request`
- packaging logic/output
- export gate result or export eligibility decision

Downstream relationship

It supports:

- final state transition to `exported`
- operator download visibility
- audit traceability
- artifact indexing and retention
- later maintenance/lifecycle processes

So `export_record` is the main bridge between passed build output and portable handoff.

---

10. Timing in the lifecycle

This artifact should generally be:

- initialized when export begins or is approved to begin
- updated during packaging if needed
- finalized once export succeeds or fails

It should exist whenever export was requested or attempted, not only when export succeeds.

That way failed export still leaves a clear trace.

---

11. Export record scope

At minimum, the artifact should summarize these areas:

11.1 Export identity

What export event this is.

11.2 Export intent

Whether export was requested.

11.3 Export eligibility

Whether export was allowed and on what basis.

11.4 Package summary

What logical contents were included.

11.5 Artifact reference

What portable artifact was produced.

11.6 Result and timing

How export ended and when.

That gives you a clean, governed export record.

---

12. Minimum required fields

At minimum, `export_record` should define fields like:

- `export_record_id`
- `run_id`
- `build_manifest_id`
- `build_session_id`
- `verification_report_ref`
- `export_requested`
- `export_allowed`
- `export_started_at`
- `export_completed_at`
- `package_format`

- `package_contents_summary`
- `export_artifact_ref`
- `export_artifact_name`
- `result`
- `failure_reason_code`
- `status_summary`
- `created_at`
- `updated_at`

Optional but useful:

- `export_gate_result_ref`
- `artifact_size_bytes`
- `artifact_hash`
- `notes`

---

13. Recommended field meanings

`export_record_id`

Unique identifier for the export record.

`run_id`

Originating run ID.

`build_manifest_id`

Reference to the build manifest for the build being exported.

`build_session_id`

Reference to the concrete build attempt.

`verification_report_ref`

Reference to the verification report that supported export eligibility.

`export_requested`

Whether the operator requested export.

`export_allowed`

Whether policy/gating allowed export to proceed.

`export_started_at`

Timestamp when packaging/export began.

`export_completed_at`

Timestamp when export completed or failed terminally.

`package_format`

Packaging format used.

Suggested enum:

- `zip`

`package_contents_summary`

Summary of what the export package includes.

Examples:

- generated repo only
- generated repo + manifests

- generated repo + manifests + verification report

`export_artifact_ref`

Reference/path to the produced export artifact.

`export_artifact_name`

Portable artifact filename.

Example:

- `project_repo.zip`

`result`

Final export outcome.

Suggested enum:

- `succeeded`
- `failed`
- `blocked`

`failure_reason_code`

Machine-usable reason code when export did not succeed.

Examples:

- `export_not_requested`
- `export_not_allowed`
- `packaging_failure`
- `artifact_write_failure`

- `missing_required_export_inputs`

`status_summary`

Short human-readable summary of export outcome.

`created_at`

Timestamp export record was initialized.

`updated_at`

Timestamp export record was last updated.

`export_gate_result_ref`

Optional reference to the export gate decision artifact.

`artifact_size_bytes`

Optional export artifact size.

`artifact_hash`

Optional integrity hash for the exported artifact.

---

14. Suggested enums

`package_format`

- `zip`

`result`

- `succeeded`
- `failed`
- `blocked`

---

15. Suggested schema shape

A simplified logical shape could be:


```
{
 "export_record_id": "EXPR-0001",
 "run_id": "RUN-0001",
 "build_manifest_id": "BMAN-0001",
 "build_session_id": "BSES-0001",
 "verification_report_ref": "VREP-0001",
 "export_requested": true,
 "export_allowed": true,
 "export_started_at": "2026-03-06T12:18:30Z",
 "export_completed_at": "2026-03-06T12:19:00Z",
 "package_format": "zip",
 "package_contents_summary": {
   "include_repo": true,
   "include_build_manifest": true,
   "include_repo_manifest": true,
   "include_verification_report": true
 },
 "export_artifact_ref": "runs/RUN-0001/build/exports/project_repo.zip",
 "export_artifact_name": "project_repo.zip",
 "result": "succeeded",
 "failure_reason_code": null,
 "status_summary": "Export zip created successfully.",
 "created_at": "2026-03-06T12:18:30Z",
 "updated_at": "2026-03-06T12:19:00Z",
 "export_gate_result_ref": "BGR-0003",
 "artifact_size_bytes": 842113,
 "artifact_hash": "sha256:abc123...",
 "notes": null
}
```


That is enough for a strong first version.

---

16. Validation rules

At minimum, validation should require:

- `export_record_id` present
- `run_id` present
- `build_manifest_id` present
- `export_requested` present
- `export_allowed` present
- `result` present once finalized
- `created_at` present
- `updated_at` present

Suggested consistency rules:

- if `result = succeeded`, then `export_requested = true`, `export_allowed = true`, and `export_artifact_ref` must exist
- if `result = blocked`, then `export_allowed = false` or required preconditions were not met
- if `result = failed`, then `failure_reason_code` should exist
- if export was not requested, the system should normally not create a successful export record
- `export_completed_at` should exist once the export result is final

---

17. Relationship to build state

`export_record` supports the transition from:

- `passed` → `exported`

A clean interpretation is:

- build passes first
- export begins
- `export_record` captures the packaging/export event
- success allows final `exported` state
- failure blocks that final state

So this artifact is part of the proof that `exported` was earned.

---

18. Relationship to failure

A failed export should still leave a complete enough `export_record` to explain what happened.

That means when export does not succeed, the artifact should still show:

- whether export was requested
- whether it was allowed
- where failure happened at a summary level
- why no valid export artifact was produced

This is important because export failure is not the same as repo-generation failure.

---

19. Relationship to output contract

This artifact should be required when the requested outcome includes export.

That means:

- if the requested outcome was build-only, `export_record` may be absent or optional

- if the requested outcome was build-and-export, `export_record` should be part of the required successful output set

That keeps the export path measurable.

---

20. Audit significance

This artifact is important for audit because it answers:

- was export requested
- was export allowed
- what package was created
- where is it
- did export succeed or fail

It protects the distinction between:

- build success
  and
- portable handoff success

---

21. Success criteria for this artifact

`export_record` is successful when it provides a clear, traceable, and machine-usable record of the Build Mode export event, including export intent, export eligibility, package summary, portable artifact reference, final export result, and failure reason when applicable, without trying to replace the build manifest, verification report, or zip artifact itself.

---

22. Short version

BM-A21 — `export_record`

Records the packaging and export event for a Build Mode run. It captures whether export was requested and allowed, when export ran, what the package contained at a summary level, where the portable artifact was written, and whether the export succeeded, failed, or was blocked. It is the main proof artifact for the portable handoff step and the transition from `passed` to `exported`.

BM-A22

BM-A22 — `failure_report`

1. Purpose

Define the artifact that records structured failure information when a Build Mode run does not complete successfully.

This artifact exists so failure is not reduced to a vague status label. It gives Build Mode a dedicated record for what failed, where it failed, why it failed, what it blocked, and what partial outputs or evidence remain after the failure.

---

2. Why this artifact exists

A `build_state` can say the build is `failed`, but that is not enough.

Build Mode needs a structured artifact that answers:

- what failure class occurred
- at what state or phase it occurred
- what specifically caused the failure
- whether the failure blocked build completion, export, or both
- what artifacts or partial outputs still exist
- what evidence is available for review or retry planning

`failure_report` exists to provide that answer.

Without it, failure becomes harder to inspect, harder to audit, and harder to recover from cleanly.

---

3. Core definition

`failure_report` is the formal output artifact that records the structured details of a Build Mode failure, including failure class, failure point, blocking reason, affected lifecycle stage, related evidence, and partial output references when applicable.

---

4. Core principle

Failure must be explainable, not just detectable.

That is the main principle behind this artifact.

---

5. What this artifact represents

`failure_report` represents:

- the failure event as a governed outcome
- the classified reason the build did not complete
- the point in the lifecycle where the build was blocked or failed
- the evidence needed to inspect the failure and determine next steps

It is not:

- the current state snapshot
- the build manifest
- the verification report
- the export record
- a full transition ledger

It is the failure-detail artifact.

---

6. What this artifact is responsible for capturing

At minimum, `failure_report` should capture:

- what build/run/session it belongs to
- what failure class occurred
- where the failure occurred
- what the blocking reason was
- what major evidence supports the failure
- what outputs, if any, were partially produced
- whether export was blocked or failed as a consequence

That is the minimum serious failure record set.

---

7. Required responsibilities

The `failure_report` artifact should support these responsibilities.

7.1 Failure classification

It must identify the failure class in a structured way.

7.2 Failure-point recording

It must show where in the lifecycle or build phases the failure occurred.

7.3 Blocking-reason recording

It must explain what stopped successful progression.

7.4 Evidence linkage

It must point to the relevant records, checks, or artifacts that demonstrate the failure.

7.5 Partial-output visibility

It must show what usable outputs, if any, still exist after failure.

---

8. What this artifact does not do

`failure_report` should not:

- replace the current `build_state`
- replace the high-level `build_manifest`
- replace the verification report when the failure is verification-related
- replace export record details when the failure is export-related
- decide recovery policy by itself

It explains failure. It does not govern the full response alone.

---

9. Relationship to other artifacts

This artifact sits in the output/proof layer and is primarily used when the build does not end successfully.

Upstream relationship

It is populated from:

- `build_state`
- `build_transition_record`
- `build_plan`

- build phase records
- `verification_report` if verification failed
- `export_record` if export failed
- gate results
- command/check results
- planner/generator/workspace errors

Downstream relationship

It supports:

- operator failure visibility
- audit review
- retry/resume analysis later
- maintenance/system trend analysis later
- final build summary completion in `build_manifest`

So `failure_report` is the detailed explanatory artifact for failed builds.

---

10. Timing in the lifecycle

This artifact should generally be:

- initialized when a terminal failure is recognized
- populated with the relevant failure details and evidence
- finalized before the build is closed as failed

It should exist for any failed build attempt serious enough to prevent successful completion of the requested outcome.

That means it should exist for:

- entry/eligibility failures
- planning failures
- generation failures
- verification failures
- export failures, if the requested outcome depended on export

---

11. Failure classes

At minimum, the report should support these failure classes:

- `missing_kit_input`
- `invalid_build_inputs`
- `eligibility_failure`
- `planning_failure`
- `workspace_failure`
- `repo_generation_failure`
- `verification_failure`
- `export_failure`
- `record_failure`

These align with the Build Mode failure model already defined.

---

12. Failure scope sections

A strong first-version `failure_report` should capture at least these sections:

1. failure identity
2. failure classification
3. failure point
4. blocking reason
5. evidence and related references
6. partial outputs and consequences
7. timing summary

That gives you a serious but controlled artifact.

---

13. Minimum required fields

At minimum, `failure_report` should define fields like:

- `failure_report_id`
- `run_id`
- `build_manifest_id`
- `build_session_id`
- `build_request_id`
- `failure_class`
- `failure_stage`
- `failure_phase_id`

- `failure_state`
- `reason_code`
- `reason_message`
- `blocking`
- `detected_at`
- `related_artifact_refs`
- `partial_output_refs`
- `export_impacted`
- `final_outcome`
- `created_at`
- `updated_at`

Optional but useful:

- `verification_report_ref`
- `export_record_ref`
- `command_results_ref`
- `retry_hint`
- `notes`

---

14. Recommended field meanings

`failure_report_id`

Unique identifier for the failure report.

`run_id`

Originating run ID.

`build_manifest_id`

Reference to the related build manifest.

`build_session_id`

Reference to the concrete build attempt.

`build_request_id`

Reference to the request that activated the failed build.

`failure_class`

Structured classification of failure type.

`failure_stage`

High-level lifecycle stage where failure occurred.

Examples:

- `entry`
- `planning`
- `workspace`
- `generation`
- `verification`
- `export`
- `closure`

`failure_phase_id`

Specific phase identifier if the failure happened inside a known build phase.

Examples:

- `prepare_workspace`
- `materialize_backend`
- `write_manifests`
- `handoff_to_verification`

`failure_state`

The Build Mode lifecycle state active when failure was recognized.

Examples:

- `requested`
- `approved`
- `building`
- `verifying`
- `passed` *(if export failed after pass)*

`reason_code`

Machine-usable code for the direct blocking reason.

Examples:

- `missing_required_spec`
- `conflicting_locked_inputs`
- `workspace_init_failed`
- `required_files_not_generated`
- `placeholder_scan_failed`
- `artifact_write_failed`

`reason_message`

Human-readable explanation of the failure.

`blocking`

Boolean indicating whether the failure blocked successful completion.

`detected_at`

Timestamp when the failure was detected/finalized.

`related_artifact_refs`

List of references to artifacts/evidence related to the failure.

Examples:

- verification report
- command results
- gate result
- planner output
- export record

`partial_output_refs`

List of refs to outputs that were produced before failure.

Examples:

- repo path
- partial manifests
- generated module directories

`export_impacted`

Boolean indicating whether export was blocked or failed because of this failure.

`final_outcome`

High-level failure outcome summary.

Suggested enum:

- `build_failed`
- `export_failed`
- `build_blocked`

`created_at`

Timestamp failure report was initialized.

`updated_at`

Timestamp report was last updated.

`verification_report_ref`

Optional direct reference if the failure came from verification.

`export_record_ref`

Optional direct reference if the failure came from export.

`command_results_ref`

Optional direct reference if command/check output is relevant.

`retry_hint`

Optional high-level hint for later recovery analysis.

Examples:

- `fix_inputs_and_retry`
- `resolve_verification_failures`
- `retry_export_only`

---

15. Suggested enums

`failure_class`

- `missing_kit_input`
- `invalid_build_inputs`
- `eligibility_failure`
- `planning_failure`
- `workspace_failure`
- `repo_generation_failure`
- `verification_failure`
- `export_failure`
- `record_failure`

`failure_stage`

- `entry`
- `planning`
- `workspace`
- `generation`
- `verification`
- `export`
- `closure`

`final_outcome`

- `build_failed`
- `export_failed`
- `build_blocked`

---

16. Suggested schema shape

A simplified logical shape could be:


```
{
  "failure_report_id": "FREP-0001",
  "run_id": "RUN-0001",
  "build_manifest_id": "BMAN-0001",
  "build_session_id": "BSES-0001",
  "build_request_id": "BRQ-0001",
  "failure_class": "verification_failure",
  "failure_stage": "verification",
```

```
  "failure_phase_id": "handoff_to_verification",
  "failure_state": "verifying",
  "reason_code": "placeholder_scan_failed",
  "reason_message": "Critical unresolved placeholders remained in generated backend files.",
  "blocking": true,
  "detected_at": "2026-03-06T12:18:00Z",
  "related_artifact_refs": [
    "VREP-0001",
    "CMDR-0001",
    "BGR-0002"
  ],
  "partial_output_refs": [
    "runs/RUN-0001/build/repo",
    "RMAN-0001"
  ],
  "export_impacted": true,
  "final_outcome": "build_failed",
  "created_at": "2026-03-06T12:18:00Z",
  "updated_at": "2026-03-06T12:18:00Z",
  "verification_report_ref": "VREP-0001",
  "export_record_ref": null,
  "command_results_ref": "CMDR-0001",
  "retry_hint": "resolve_verification_failures",
  "notes": null
}
```


That is enough for a strong first version.

---

17. Validation rules

At minimum, validation should require:

- `failure_report_id` present
- `run_id` present
- `failure_class` present
- `failure_stage` present
- `failure_state` present
- `reason_code` present
- `reason_message` present
- `blocking` present

- `detected_at` present
- `final_outcome` present
- `created_at` present
- `updated_at` present

Suggested consistency rules:

- if `failure_class = verification_failure`, then `verification_report_ref` should usually exist
- if `failure_class = export_failure`, then `export_record_ref` should usually exist
- if `final_outcome = export_failed`, then the build may already have passed but export did not complete
- if `blocking = true`, successful completion must not be recorded
- `partial_output_refs` may be empty, but should be present when partial outputs exist

---

18. Relationship to verification and export

This artifact should work alongside, not instead of, the verification and export artifacts.

If verification failed

`verification_report` shows what checks failed.
`failure_report` explains that this verification failure became the build-blocking outcome.

If export failed

`export_record` shows the export event failed.
`failure_report` explains that this prevented the requested build-and-export outcome from completing.

That separation keeps each artifact clean.

---

19. Relationship to build manifest

For failed builds, `build_manifest` should summarize the failure at a high level.

`failure_report` should hold the deeper failure details.

So:

- `build_manifest` = summary of failed build outcome
- `failure_report` = structured explanation of failure

That is the right division.

---

20. Relationship to output contract

`failure_report` is not part of the successful output contract, but it is part of the failed-build proof contract.

A failed build should still be considered well-recorded only if there is a failure artifact that explains:

- what failed
- where it failed
- why it failed
- what evidence supports that failure

That keeps failures governable, not just visible.

---

21. Audit significance

This artifact is important for audit because it answers:

- why the build did not complete
- what failure class applies
- what stage/state the failure happened in
- what evidence supports the failure
- what outputs, if any, still exist

It is the main explanatory artifact for failed runs.

---

22. Success criteria for this artifact

`failure_report` is successful when it provides a clear, structured, and traceable explanation of a Build Mode failure, including failure class, failure point, blocking reason, related evidence,

and partial output references when applicable, without trying to replace the build summary, verification result, or export event artifacts.

---

23. Short version

BM-A22 — `failure_report`
Records the structured explanation of a Build Mode failure. It identifies the failed run/build/session, classifies the failure, records the lifecycle stage and phase where it occurred, captures the blocking reason and related evidence, and notes any partial outputs or export impact. It is the main detail artifact for failed builds and the core proof record for why a requested build outcome did not complete.

BM-A25

BM-A25 — `build_runner_contract`

1. Purpose

Define the contract for the component that orchestrates the full Build Mode flow from request-approved execution through final pass/fail/export closure.

This artifact exists because Build Mode has multiple governed components, but it still needs one coordinating authority at the execution layer that runs them in the correct order, enforces the high-level sequence, and ensures the build does not skip required steps.

---

2. Why this artifact exists

Even with:

- a build request
- a build state model
- a build plan
- manifests
- verification
- export logic

the system still needs one component responsible for saying:

- what runs first
- what runs next
- what happens on failure
- when the build may progress
- when the build must stop
- when final closure happens

`build_runner_contract` exists to define that orchestration role.

Without it, Build Mode risks turning into a collection of components with unclear sequencing and inconsistent lifecycle handling.

---

3. Core definition

`build_runner_contract` is the formal component contract that defines the responsibilities, inputs, outputs, sequencing rules, and lifecycle-governance obligations of the Build Runner, the component responsible for orchestrating one Build Mode execution path end to end.

---

4. Core principle

The Build Runner does not invent the build.
It coordinates the governed execution of the build.

That is the main principle behind this contract.

---

5. What this contract represents

`build_runner_contract` represents:

- the orchestration authority for Build Mode execution
- the official sequencing logic across Build Mode components
- the component boundary for end-to-end build coordination
- the high-level runtime conductor for one build attempt

It is not:

- the Build Control Plane itself
- the Build Planner
- the Repo Generator
- the Verifier
- the Export Packager

It is the orchestrator contract.

---

6. What this component is responsible for

At minimum, the Build Runner should be responsible for:

- accepting a build session that is ready to run
- invoking the right components in the right order
- enforcing the major Build Mode flow
- updating or triggering governed state transitions
- stopping on blocking failures
- ensuring final output and closure logic happen correctly

- ensuring export only happens after pass conditions are satisfied

That is the minimum orchestration responsibility set.

---

7. Required responsibilities

The `build_runner_contract` should support these responsibilities.

7.1 Flow orchestration

It must coordinate the end-to-end Build Mode sequence.

7.2 Phase sequencing

It must execute build phases in the governed order.

7.3 State progression coordination

It must trigger or request the correct state transitions at the correct times.

7.4 Failure stop behavior

It must stop forward progression when blocking failures occur.

7.5 Completion coordination

It must ensure the build only reaches closure after required records, verification, and export logic are handled properly.

---

8. What this component does not do

The Build Runner should not:

- decide build eligibility policy on its own
- define the build plan on its own
- perform the repo generation logic itself
- redefine verification rules on its own
- bypass control-plane rules
- silently fix missing required inputs
- behave like an all-in-one god object

It coordinates specialized components.
It should not absorb all of them.

---

9. Relationship to other artifacts and components

This contract sits in the component-contract layer and coordinates multiple Build Mode artifacts/components.

It consumes or depends on

- `build_request`
- `build_state`
- `build_plan`
- Build Control Plane rules
- workspace manager
- repo generator
- manifest writer
- verifier
- export packager

It contributes to

- state progression
- build manifests
- verification flow
- export flow
- failure handling
- final closure

So this contract defines the orchestration center of the active build run.

---

10. Position in the Build Mode flow

The Build Runner should own orchestration starting after the build is approved for execution.

A clean flow position is:

1. build request exists
2. eligibility passes
3. build plan exists
4. **Build Runner begins governed execution**
5. workspace prepared
6. repo generated
7. records written

8. verification run
9. export run if requested/allowed
10. final closure recorded

This means the Build Runner begins after planning readiness and owns the active execution path.

---

11. Main flow obligations

The `build_runner_contract` should define that the Build Runner must coordinate, at minimum, this sequence:

1. confirm execution prerequisites are present
2. start active build execution
3. invoke workspace preparation
4. invoke repo generation according to plan
5. invoke record/manifest writing
6. invoke verification
7. evaluate whether export should run
8. invoke export if requested and allowed
9. finalize success/failure closure
10. ensure final artifacts are linked to the run

This is the minimum runner flow.

---

12. Inputs to the Build Runner

At minimum, the contract should define these input classes:

- `run_id`
- `build_request_id`
- `build_session_id`
- `build_plan_ref`
- `build_basis_ref`
- current `build_state`
- policy/control-plane context
- component contract references/configuration
- workspace/output location context

Optional but useful:

- `repo_blueprint_ref`
- `required_file_inventory_ref`
- `verification_command_set_ref`

These inputs should be enough to run the governed flow without live improvisation.

---

13. Outputs from the Build Runner

At minimum, the contract should define that the Build Runner coordinates production of:

- state transition triggers/results
- build phase completion outcomes
- `build_manifest`
- `repo_manifest`
- `verification_report`
- `export_record` if applicable
- `failure_report` if applicable
- final closure outcome

It may not author all these directly, but it is responsible for ensuring the sequence that produces them.

---

14. Phase sequencing rules

The Build Runner should enforce the official high-level build order.

Minimum phase sequence:

1. `prepare_workspace`
2. `create_repo_skeleton`
3. `materialize_repo_contents`
4. `write_build_records`
5. `run_verification`
6. `run_export_if_requested`
7. `finalize_closure`

You may internally break `materialize_repo_contents` into more slices, but the runner should still enforce the outer order.

Rules:

- no phase may run before its dependencies are satisfied
- no export phase before verification passes
- no final success closure before required output artifacts exist
- no illegal skipping of required phases

---

15. State coordination rules

The contract should require the Build Runner to coordinate state progression like this:

- move or request move to `building` when active generation starts
- move or request move to `verifying` when generation is complete enough for checks
- move or request move to `passed` only when verification succeeds
- move or request move to `exported` only when export succeeds
- move or request move to `failed` when blocking failure occurs

The runner should not directly mutate state arbitrarily outside governed transition rules.

---

16. Failure-handling rules

The `build_runner_contract` should define these failure behaviors:

16.1 Stop on blocking failure

If a blocking failure occurs in any required phase, the runner must stop illegal forward progression.

16.2 Record failure context

The runner must ensure failure details are captured in the proper artifacts.

16.3 Preserve useful partial outputs

Where appropriate, the runner should preserve partial artifacts rather than destroying evidence.

16.4 Prevent false success

The runner must not allow failed or incomplete builds to move into successful completion states.

These are core trust-preserving rules.

---

17. Export-handling rules

The Build Runner should only invoke export when:

- export was requested
- verification passed
- export is allowed by policy
- required artifacts for packaging exist

If these are not true, the runner must:

- skip export if not requested
- block export if not allowed
- fail the requested outcome if export was required but could not be completed

This protects the difference between:

- passed build
- exported build

---

18. Coordination with specialized components

The contract should define the Build Runner as a coordinator of specialized component boundaries.

At minimum it should coordinate:

- request/build session context initialization
- workspace manager
- repo generator
- manifest writer
- verifier
- export packager

It should not absorb their internal domain logic.
It should invoke them and enforce the flow around them.

---

19. Audit obligations

The Build Runner should ensure that major execution events are traceable.

At minimum, the contract should require orchestration-level traceability for:

- build execution start
- phase entry/exit
- verification start/completion
- export start/completion
- final closure
- failure termination

This does not mean the runner owns the entire audit ledger, but it must ensure these events happen in a governable way.

---

20. Minimum required fields

At the document level, this contract should at minimum define:

- `contract_id`
- `contract_name`
- `version`
- `purpose`
- `scope`
- `governing_principle`
- `orchestration_scope`
- `required_inputs`
- `coordinated_components`
- `phase_sequence`
- `state_coordination_rules`
- `failure_handling_rules`
- `export_handling_rules`
- `required_outputs`
- `audit_obligations`
- `non_responsibilities`

---

21. Suggested contract shape

A compact logical shape could be:

```
{
  "contract_id": "BM-A25",
```

```
  "contract_name": "build_runner_contract",
  "version": "1.0.0",
  "purpose": "Coordinate the governed Build Mode execution flow from active build start through
final closure.",
  "scope": "One Build Mode execution path for one build session.",
  "governing_principle": "Coordinate the build; do not invent it.",
  "orchestration_scope": [
    "workspace_preparation",
    "repo_generation",
    "record_writing",
    "verification",
    "export_if_requested",
    "final_closure"
  ],
  "required_inputs": [
    "build_request",
    "build_state",
    "build_plan",
    "build_basis_ref",
    "policy_context"
  ],
  "coordinated_components": [
    "workspace_manager",
    "repo_generator",
    "build_manifest_writer",
    "build_verifier",
    "export_packager"
  ],
  "phase_sequence": [
    "prepare_workspace",
    "create_repo_skeleton",
    "materialize_repo_contents",
    "write_build_records",
    "run_verification",
    "run_export_if_requested",
    "finalize_closure"
  ],
  "state_coordination_rules": {
    "start_building_on_execution_start": true,
    "enter_verifying_after_generation": true,
    "enter_passed_only_after_verification": true,
    "enter_exported_only_after_successful_export": true,
    "enter_failed_on_blocking_failure": true
  },
```

```
  "failure_handling_rules": {
    "stop_on_blocking_failure": true,
    "preserve_partial_outputs": true,
    "require_failure_report": true
  },
  "export_handling_rules": {
    "export_requires_request": true,
    "export_requires_passed_verification": true,
    "export_requires_allowed_policy": true
  },
  "required_outputs": [
    "build_manifest",
    "repo_manifest",
    "verification_report"
  ],
  "audit_obligations": [
    "record_execution_start",
    "record_phase_transitions",
    "record_verification_start_and_end",
    "record_export_start_and_end",
    "record_final_closure"
  ],
  "non_responsibilities": [
    "product_definition",
    "eligibility_policy_definition",
    "repo_generation_logic",
    "verification_scope_definition"
  ]
}
```

That is enough for a first version.

---

22. Validation rules

At minimum, validation should require:

- contract identity fields present
- phase sequence present and ordered
- coordinated components present
- state coordination rules present
- failure handling rules present
- export handling rules present

- non-responsibilities present

Suggested consistency rules:

- phase sequence must reflect Build Mode high-level flow
- export handling rules must not allow export before pass
- non-responsibilities must not contradict orchestration scope
- runner required outputs should align with Build Outputs Contract

---

23. Success criteria for this artifact

`build_runner_contract` is successful when it clearly defines the Build Runner as the orchestration component for Build Mode execution, including what it coordinates, what order it enforces, how it handles state progression and blocking failure, how it invokes export correctly, and what it explicitly does not own.

---

24. Short version

BM-A25 — `build_runner_contract`
Defines the orchestration contract for the Build Runner component. It specifies how the active Build Mode flow is coordinated across phases and specialized components, how state progression and failure stops are handled, how export is invoked only after valid completion, and how final closure is reached without letting the runner absorb planning, generation, verification, or policy logic that belongs elsewhere.

BM-A28

BM-A28 — `build_planner_contract`

1. Purpose

Define the contract for the component that derives the actionable build plan from the approved build basis.

This artifact exists because Build Mode should not go straight from Agent Kit to repo generation without first translating the approved build basis into a concrete, execution-ready plan. The Build Planner is the component that performs that translation in a governed way.

---

2. Why this artifact exists

The Agent Kit contains build intelligence, but it is not automatically the same as an execution plan.

The system still needs a component responsible for answering:

- what repo shape should be targeted
- what build phases should exist
- what outputs are required
- what constraints must be carried into execution
- what verification expectations apply
- whether the inputs are concrete enough to yield a stable build path

`build_planner_contract` exists to define that responsibility clearly.

Without it, planning logic either gets scattered across the system or gets improvised by the Repo Generator, which increases drift.

---

3. Core definition

`build_planner_contract` is the formal component contract that defines the responsibilities, inputs, outputs, derivation rules, and no-guess constraints of the Build Planner, the component responsible for producing the governed `build_plan` artifact from approved upstream build inputs.

---

4. Core principle

The build path should be derived before generation begins, and that derivation should be explicit, constrained, and traceable.

That is the main principle behind this contract.

---

5. What this contract represents

`build_planner_contract` represents:

- the planning logic boundary inside Build Mode
- the component that converts approved build basis into execution structure
- the rules for deriving `build_plan`
- the anti-drift planning layer that sits between upstream truth and active generation

It is not:

- the Agent Kit itself
- the Build Runner
- the Repo Generator
- the Build Control Plane as a whole

It is the planner component contract.

---

6. What this component is responsible for

At minimum, the Build Planner should be responsible for:

- reading the approved build basis
- identifying the target repo type/shape
- deriving the controlled phase sequence
- determining required output targets
- carrying forward execution constraints
- defining verification expectations for the run
- producing a valid `build_plan` or failing when a stable plan cannot be derived

That is the minimum serious planning responsibility set.

---

7. Required responsibilities

The `build_planner_contract` should support these responsibilities.

7.1 Build-plan derivation

It must produce a concrete `build_plan` artifact.

7.2 Repo-target derivation

It must determine the repo type and structural target from the approved inputs.

7.3 Phase-sequence derivation

It must define the execution order needed for controlled materialization.

7.4 Constraint carry-forward

It must carry build profile, no-guess, and dependency constraints into the plan.

7.5 Verification handoff definition

It must define the verification expectations the plan requires downstream.

7.6 Planning failure enforcement

It must fail rather than invent a plan when approved inputs are too weak or contradictory.

---

8. What this component does not do

The Build Planner should not:

- define the product from scratch
- invent missing major requirements
- expand scope beyond approved inputs
- generate repo files directly
- mutate lifecycle state on its own
- redefine policy rules
- silently resolve conflicting locked inputs by preference

It derives the plan.
It does not execute the build or repair upstream truth by improvisation.

---

9. Relationship to other artifacts and components

This contract sits between the build basis and active execution.

It consumes or depends on

- approved Agent Kit
- locked standards/specs/templates/contracts
- build profile
- repo blueprint if available
- required file inventory if available
- policy/no-guess constraints
- build request/build session context

It produces or feeds

- `build_plan`
- repo target summary
- phase sequence
- required outputs summary
- execution constraints
- verification expectations
- planning-failure outcome if derivation is not possible

So the Build Planner is the component that turns build truth into build structure.

---

10. Position in the Build Mode flow

The Build Planner should run after:

- build request exists
- eligibility passes
- build is approved

It should run before:

- workspace preparation
- repo generation
- verification

So its flow position is:

1. request
2. eligibility

3. approval
4. **planning**
5. execution

That makes it the first major execution-preparation component.

---

11. Inputs to the Build Planner

At minimum, the contract should define these input classes:

- `run_id`
- `build_request_id`
- `build_session_id`
- `build_basis_ref`
- approved Agent Kit
- locked standards snapshot refs
- locked template/spec/contract refs
- `build_profile_ref`
- `repo_blueprint_ref` if available
- `required_file_inventory_ref` if available
- policy/no-guess context

Optional but useful:

- operator-selected export intent
- domain/module map
- command-set definitions

These inputs should be enough to derive the plan without guessing beyond allowed bounds.

---

12. Outputs from the Build Planner

At minimum, the contract should define that the Build Planner produces:

- `build_plan`
- derived phase sequence
- target repo summary
- required outputs summary
- execution constraints summary
- verification expectations summary

If planning fails, it should produce or trigger:

- structured planning failure result
- planning-related failure reason/evidence

---

13. Planning derivation rules

The contract should define that planning must derive, at minimum:

13.1 Repo target

What repo type and structure the build is aiming to produce.

13.2 Build phases

What phases/slices are required and in what order.

13.3 Required outputs

What the build must materialize to count as complete.

13.4 Constraints

What rules the generator must obey.

13.5 Verification expectations

What conditions must later be checked before pass/export.

The planner should not leave these undefined if the build is to proceed.

---

14. No-guess planning rules

This is one of the most important parts of the contract.

The Build Planner should enforce these minimum no-guess rules:

14.1 No silent scope expansion

It must not add undeclared major features, domains, routes, screens, entities, or modules.

14.2 No conflict-by-preference resolution

If locked sources conflict materially, it must fail or defer to explicit policy—not choose the version it “likes.”

14.3 No missing-input substitution

If a required build-defining input is absent, it must not invent a replacement.

14.4 No unsupported dependency assumptions

It must not design a plan that requires undeclared dependencies unless the build profile explicitly allows them.

14.5 No ambiguous phase invention without basis

If the build basis is too vague to derive a stable phase sequence, it must fail planning rather than create an arbitrary path.

These rules make planning deterministic enough to reduce downstream drift.

---

15. Planning failure rules

The `build_planner_contract` should require the planner to fail planning when:

- required upstream inputs are missing
- the repo target cannot be derived safely
- execution phases cannot be determined coherently
- build-defining sources conflict materially
- required outputs cannot be identified for the selected profile
- verification expectations cannot be derived meaningfully

This is where the system chooses explicit failure over hidden improvisation.

---

16. Planning success rules

The planner should only succeed when it can produce a `build_plan` that is:

- traceable to approved inputs
- structurally coherent
- sufficiently explicit for execution
- aligned with the locked build profile
- aligned with no-guess constraints
- sufficient to hand off to the Build Runner and Repo Generator

If the plan is too vague for controlled execution, that should count as planning failure.

---

17. Coordination with the Build Runner

The planner should hand the Build Runner a plan that is ready for execution, not a loose draft.

That means the contract should require the planner to output:

- an ordered phase list
- repo target summary
- required outputs summary
- constraints
- verification expectations

So the Build Runner can coordinate execution without inventing the plan itself.

---

18. Audit obligations

The Build Planner should ensure planning is traceable enough to answer:

- what sources were used to derive the plan
- what repo target was selected
- what phases were defined
- what constraints were carried forward
- why planning failed if it failed

This does not mean the planner owns the full audit ledger, but it must produce enough structured planning evidence to support later inspection.

---

19. Minimum required fields

At the document level, this contract should at minimum define:

- `contract_id`
- `contract_name`
- `version`
- `purpose`
- `scope`
- `governing_principle`

- `required_inputs`
- `required_derivations`
- `planning_success_conditions`
- `planning_failure_conditions`
- `no_guess_rules`
- `produced_artifacts`
- `handoff_requirements`
- `audit_obligations`
- `non_responsibilities`

---

20. Suggested contract shape

A compact logical shape could be:


```
{
  "contract_id": "BM-A28",
  "contract_name": "build_planner_contract",
  "version": "1.0.0",
  "purpose": "Derive the governed build_plan from the approved build basis before active repo generation begins.",
  "scope": "One Build Mode planning event for one build session.",
  "governing_principle": "Derive the build path explicitly before execution; do not invent it during generation.",
  "required_inputs": [
    "approved_agent_kit",
    "locked_standards_refs",
    "locked_spec_refs",
    "build_profile_ref",
    "repo_blueprint_ref_optional",
    "required_file_inventory_ref_optional",
    "policy_context"
  ],
  "required_derivations": [
    "target_repo_summary",
    "phase_sequence",
    "required_outputs_summary",
    "execution_constraints",
    "verification_expectations"
  ],
  "planning_success_conditions": [
    "repo_target_is_derivable",
```

```
    "phase_sequence_is_coherent",
    "required_outputs_are_identifiable",
    "constraints_are_consistent",
    "verification_expectations_are_defined"
  ],
  "planning_failure_conditions": [
    "missing_required_inputs",
    "material_conflicting_locked_inputs",
    "repo_target_not_derivable",
    "phase_sequence_not_derivable",
    "required_outputs_not_identifiable"
  ],
  "no_guess_rules": [
    "no_silent_scope_expansion",
    "no_conflict_resolution_by_preference",
    "no_missing_input_substitution",
    "no_undeclared_dependency_assumptions"
  ],
  "produced_artifacts": [
    "build_plan"
  ],
  "handoff_requirements": [
    "ordered_phase_list",
    "constraints_summary",
    "verification_expectations_summary"
  ],
  "audit_obligations": [
    "record_plan_basis_refs",
    "record_phase_derivation",
    "record_planning_failure_reason_if_any"
  ],
  "non_responsibilities": [
    "repo_generation",
    "verification_execution",
    "policy_definition",
    "scope_creation_from_scratch"
  ]
}
```


That is enough for a first version.

---

21. Validation rules

At minimum, validation should require:

- contract identity fields present
- required inputs present
- required derivations present
- planning success conditions present
- planning failure conditions present
- no-guess rules present
- handoff requirements present
- non-responsibilities present

Suggested consistency rules:

- success conditions must imply the planner can hand off to execution without major ambiguity
- failure conditions must align with Build Mode failure classes
- no-guess rules must not conflict with the approved build profile or policy context
- produced artifacts must include `build_plan`

---

22. Success criteria for this artifact

`build_planner_contract` is successful when it clearly defines the Build Planner as the component that derives the `build_plan` from approved build inputs, including what it must read, what it must derive, when it must fail instead of guessing, what it must hand off to execution, and what it explicitly does not own.

---

23. Short version

BM-A28 — `build_planner_contract`
Defines the component contract for the Build Planner. It specifies how the approved Agent Kit and other locked build inputs are translated into a concrete `build_plan`, including repo target, phase sequence, required outputs, execution constraints, and verification expectations. It also defines the no-guess rules and planning-failure conditions that prevent Build Mode from entering execution with an unstable or improvised plan.

BM-A30

BM-A30 — `repo_generator_contract`

1. Purpose

Define the contract for the component that materializes the actual project repository from the approved build plan.

This artifact exists because Build Mode ultimately has to create code, folders, config, manifests, and supporting files. The Repo Generator is the component that performs that materialization, but it must do so inside locked constraints rather than behaving like a freeform coding agent.

---

2. Why this artifact exists

Once the build is approved and the plan is derived, the system still needs one component responsible for the actual repo creation work.

That component needs a clear contract for:

- what it is allowed to generate
- what it must generate
- what order it should follow
- what it must not invent
- when it must stop instead of guessing
- what outputs it must leave behind for verification and packaging

`repo_generator_contract` exists to define that boundary.

Without it, repo generation becomes the highest-risk drift point in the whole Build Mode system.

---

3. Core definition

`repo_generator_contract` is the formal component contract that defines the responsibilities, inputs, outputs, generation rules, no-guess constraints, and stop conditions of

the Repo Generator, the component responsible for materializing the repository defined by the approved `build_plan`.

---

4. Core principle

The Repo Generator should materialize the approved plan, not reinterpret it into a different app.

That is the main principle behind this contract.

---

5. What this contract represents

`repo_generator_contract` represents:

- the controlled code/materialization boundary in Build Mode
- the component that turns the governed plan into a repo
- the main execution contract for file/folder generation
- the most important anti-drift component boundary in the build path

It is not:

- the Build Planner
- the Build Runner
- the Build Control Plane itself
- the Verifier
- the Export Packager

It is the repo-materialization contract.

---

6. What this component is responsible for

At minimum, the Repo Generator should be responsible for:

- creating the repo skeleton
- creating required folders and files
- materializing code/config/docs/supporting artifacts according to plan
- following the ordered build phases and target structure
- honoring dependency and scope constraints
- leaving the repo in a structurally coherent state for verification
- stopping when required inputs are missing or blocked

That is the minimum serious generation responsibility set.

---

7. Required responsibilities

The `repo_generator_contract` should support these responsibilities.

7.1 Repo skeleton creation

It must create the required structural base of the repo.

7.2 Planned target materialization

It must generate the files, modules, and areas declared by the build plan.

7.3 Constraint-respecting generation

It must honor build profile, dependency, naming, and no-guess constraints.

7.4 Ordered execution

It must generate according to the phase and target order defined by the plan and runner.

7.5 Output handoff readiness

It must produce outputs in a form that can be verified and later packaged.

---

8. What this component does not do

The Repo Generator should not:

- redefine scope
- invent major features or modules
- choose its own architecture when the plan already defines one
- add undeclared dependencies outside policy/profile
- decide final pass/fail
- perform export
- override control-plane stops

It generates the repo.
It does not govern the whole build.

---

9. Relationship to other artifacts and components

This contract sits at the core of active build execution.

It consumes or depends on

- `build_plan`
- `build_profile_ref`
- `repo_blueprint_ref` if available
- `required_file_inventory_ref` if available
- no-guess rules and dependency rules
- workspace/output paths
- phase sequence from the Build Runner

It produces or feeds

- generated repo structure
- generated files/modules/config/docs
- partial output refs during execution
- repo-ready data for manifests
- repo outputs for verification

So the Repo Generator is the component that turns the abstract plan into concrete repo output.

---

10. Position in the Build Mode flow

The Repo Generator should run after:

- the build is approved
- the `build_plan` exists
- the workspace is prepared

It should run before:

- verification
- export
- final closure

So its flow position is:

1. request
2. approval
3. planning

4. workspace prep
5. **repo generation**
6. verification
7. export
8. closure

This makes it the main execution component in the middle of the lifecycle.

---

11. Inputs to the Repo Generator

At minimum, the contract should define these input classes:

- `run_id`
- `build_session_id`
- `build_plan_ref`
- `build_profile_ref`
- `repo_blueprint_ref` if available
- `required_file_inventory_ref` if available
- `dependency_rule_set_ref` if available
- workspace/output path context
- active phase/target scope
- no-guess rules

Optional but useful:

- domain/module map
- target command expectations
- naming convention pack

These inputs should be enough for controlled generation without freeform planning during execution.

---

12. Outputs from the Repo Generator

At minimum, the contract should define that the Repo Generator produces:

- repo root structure
- generated folder hierarchy
- generated files and contents
- materialized module areas
- generated config/manifests/docs/supporting files as required by plan
- generation status per phase or target area
- partial output references if generation stops early

It should not claim to produce verification truth or export truth.

---

13. Generation scope rules

The contract should define that the Repo Generator may generate only what is inside the approved generation scope.

At minimum, this includes:

- repo areas declared by the `build_plan`

- files required by the repo blueprint or required file inventory
- code/config/docs/supporting outputs required for the selected build profile
- necessary local implementation details inside already-approved targets

It should not generate:

- undeclared major modules
- undeclared routes/entities/screens/features
- optional extras outside allowed scope unless the plan explicitly includes them

---

14. Ordered generation rules

The Repo Generator should follow the plan's ordered materialization path.

Minimum outer order:

1. repo skeleton
2. core config/manifests
3. shared contracts/types
4. backend targets
5. frontend targets
6. tests/supporting files
7. docs/env/example files
8. manifest-ready output completion

You may vary this by profile, but the contract should still require:

- no out-of-order materialization that breaks dependency assumptions

- no verification before generation completes enough to be checked
- no export-related output generation as a substitute for real build completion

---

15. No-guess generation rules

This is the most important part of the contract.

The Repo Generator should enforce these minimum no-guess rules:

15.1 No undeclared feature invention

It must not add major features, domains, modules, routes, screens, or entities outside declared scope.

15.2 No missing-input substitution

If a required target cannot be generated because a required input is missing, it must stop rather than inventing a substitute.

15.3 No conflict smoothing by intuition

If locked inputs conflict materially at generation time, it must stop or surface the conflict rather than picking a path arbitrarily.

15.4 No undeclared dependency introduction

It must not introduce libraries, frameworks, or tooling outside what the build profile and dependency rules allow.

15.5 No locked-identifier drift

It must not rename locked structures, interfaces, modules, or identities without explicit basis.

15.6 No hidden placeholder acceptance

It must not treat critical unresolved placeholders as valid completion.

These are the rules that make generation feel rigged instead of open-ended.

---

16. Stop conditions

The contract should require the Repo Generator to stop when:

- required build targets cannot be identified
- required inputs for a target are missing
- a material conflict prevents coherent generation
- the plan demands a target that is impossible under the active profile
- no-guess rules are violated
- output paths cannot be written safely

This is critical, because the generator is the place most likely to “just keep going” unless explicitly constrained.

---

17. Partial-output behavior

If generation stops after some outputs already exist, the Repo Generator should:

- preserve partial outputs when safe
- record what phases or targets were completed
- surface partial output refs for failure reporting
- avoid pretending partial generation equals build success

This helps later with audit and possible retry analysis.

---

18. Quality / coherence obligations

The Repo Generator should leave the repo in a state that is coherent enough to be verified.

That means it should aim for:

- structurally valid folder layout

- files placed in expected locations
- generated code aligned with declared targets
- root artifacts present where required
- no obvious plan-breaking omissions in completed phases

It does not have to declare the build successful.
But it must hand off something that can be checked.

---

19. Coordination with the Build Runner

The generator should not decide the overall lifecycle flow.
It should run inside the sequence coordinated by the Build Runner.

That means the contract should require the generator to accept:

- phase scope from the runner
- stop/fail signals from governed logic
- path/context from workspace management
- handoff requirements for manifest writing and verification

This keeps the generator powerful but contained.

---

20. Audit obligations

The Repo Generator should ensure generation is traceable enough to answer:

- what was materialized
- what phase/target was being generated
- where generation stopped if it failed

- what partial outputs exist
- what rules blocked further generation if any

This does not mean the generator writes the entire audit system, but it must expose enough structured output for auditability.

---

21. Minimum required fields

At the document level, this contract should at minimum define:

- `contract_id`
- `contract_name`
- `version`
- `purpose`
- `scope`
- `governing_principle`
- `required_inputs`
- `generation_scope_rules`
- `ordered_generation_rules`
- `no_guess_rules`
- `stop_conditions`
- `produced_outputs`
- `partial_output_behavior`

- `handoff_requirements`
- `audit_obligations`
- `non_responsibilities`

---

22. Suggested contract shape

A compact logical shape could be:


```
{
 "contract_id": "BM-A30",
 "contract_name": "repo_generator_contract",
 "version": "1.0.0",
 "purpose": "Materialize the repository defined by the approved build_plan inside controlled generation boundaries.",
 "scope": "One Build Mode repo-generation execution for one build session.",
 "governing_principle": "Materialize the approved plan; do not reinterpret it into a different app.",
 "required_inputs": [
   "build_plan",
   "build_profile_ref",
   "repo_blueprint_ref_optional",
   "required_file_inventory_ref_optional",
   "workspace_context",
   "no_guess_rules"
 ],
 "generation_scope_rules": [
   "generate_only_declared_targets",
   "generate_only_profile_allowed_outputs",
   "do_not_add_undeclared_major_modules"
 ],
 "ordered_generation_rules": [
   "repo_skeleton_first",
   "core_config_before_feature_layers",
   "shared_contracts_before_dependent_layers",
   "tests_and_supporting_files_after_core_materialization"
 ],
 "no_guess_rules": [
   "no_undeclared_feature_invention",
   "no_missing_input_substitution",
   "no_conflict_smoothing_by_intuition",
```

```
  "no_undeclared_dependency_introduction",
  "no_locked_identifier_drift",
  "no_hidden_placeholder_acceptance"
],
"stop_conditions": [
  "missing_required_target_input",
  "material_conflicting_locked_inputs",
  "target_impossible_under_profile",
  "unsafe_output_path_failure"
],
"produced_outputs": [
  "repo_root_structure",
  "generated_files",
  "module_areas",
  "partial_output_refs_optional"
],
"partial_output_behavior": {
  "preserve_partial_outputs_when_safe": true,
  "record_completed_targets": true,
  "surface_partial_output_refs": true
},
"handoff_requirements": [
  "repo_ready_for_manifesting",
  "repo_ready_for_verification"
],
"audit_obligations": [
  "record_generation_phase_context",
  "record_generation_stop_point_if_any",
  "record_partial_outputs_if_any"
],
"non_responsibilities": [
  "product_definition",
  "verification_decision",
  "export_packaging",
  "policy_definition"
]
}
```

That is enough for a first version.

---

23. Validation rules

At minimum, validation should require:

- contract identity fields present
- required inputs present
- generation scope rules present
- ordered generation rules present
- no-guess rules present
- stop conditions present
- produced outputs present
- non-responsibilities present

Suggested consistency rules:

- generation scope rules must align with the build plan concept
- no-guess rules must align with Build Control Plane rules
- stop conditions must align with Build Mode failure classes
- handoff requirements must align with manifest and verification expectations

---

24. Success criteria for this artifact

`repo_generator_contract` is successful when it clearly defines the Repo Generator as the component that materializes the approved build plan into a repository, including what it may generate, what it must not invent, what order it should follow, when it must stop, how it should preserve partial outputs, and how it must hand off a coherent repo to the next stages of Build Mode.

---

25. Short version

BM-A30 — `repo_generator_contract`
Defines the component contract for the Repo Generator. It specifies how the approved `build_plan` is materialized into the actual repository, including scope boundaries, ordered

generation rules, no-guess constraints, stop conditions, partial-output behavior, and handoff requirements for manifesting and verification. It is the main anti-drift contract for the code/materialization step of Build Mode.

BM-A32

BM-A32 — `build_verifier_contract`

1. Purpose

Define the contract for the component that evaluates whether the generated repository satisfies the required Build Mode completion checks.

This artifact exists because repo generation alone is not enough. The Build Verifier is the component that decides whether the generated output is valid enough to count as a successful build result and, if requested, whether the repo is eligible to move into export.

---

2. Why this artifact exists

After generation, the system still needs one component responsible for answering:

- are the required outputs present
- is the repo structurally correct
- do critical placeholders remain
- are the required manifests complete
- did required commands/checks pass
- is export allowed now

`build_verifier_contract` exists to define that responsibility clearly.

Without it, verification logic gets scattered, weakened, or treated like an optional courtesy instead of a real completion gate.

---

3. Core definition

`build_verifier_contract` is the formal component contract that defines the responsibilities, inputs, outputs, verification rules, pass/fail behavior, and export-readiness obligations of the Build Verifier, the component responsible for evaluating the generated repo against the required Build Mode verification scope.

---

4. Core principle

A generated repo must earn completion through explicit checks.

That is the main principle behind this contract.

---

5. What this contract represents

`build_verifier_contract` represents:

- the verification logic boundary inside Build Mode
- the component that turns verification scope into actual evaluation
- the pass/fail decision boundary for build completion
- the gatekeeper between generation and successful closure

It is not:

- the Build Runner
- the Repo Generator
- the Export Packager
- the Build Control Plane as a whole

It is the verifier component contract.

---

6. What this component is responsible for

At minimum, the Build Verifier should be responsible for:

- evaluating the generated repo against required verification categories
- checking structural completeness and required output presence

- checking for critical unresolved placeholders or unfinished scaffold markers
- checking required manifests/records
- running or evaluating required commands/checks
- producing a `verification_report`
- declaring whether the build passed verification
- declaring whether export is allowed

That is the minimum serious verification responsibility set.

---

7. Required responsibilities

The `build_verifier_contract` should support these responsibilities.

7.1 Verification execution

It must perform the required Build Mode checks.

7.2 Verification evidence production

It must produce a structured `verification_report`.

7.3 Blocking failure enforcement

It must fail the build when required blocking checks do not pass.

7.4 Export-readiness evaluation

It must determine whether export may proceed after verification.

7.5 Verification traceability

It must make the verification outcome inspectable and auditable.

---

8. What this component does not do

The Build Verifier should not:

- define product scope
- generate repo files
- repair missing required build truth by inventing outputs
- package exports
- redefine policy or verification scope on its own
- silently downgrade required failures into success

It verifies the build result.
It does not create the build result or relax the rules around it.

---

9. Relationship to other artifacts and components

This contract sits after repo generation and before export.

It consumes or depends on

- generated repo outputs
- `build_plan`
- `repo_manifest`
- verification scope rules
- `verification_command_set`
- required file/structure expectations
- command/check results where applicable

It produces or feeds

- `verification_report`

- pass/fail outcome for Build Mode
- export readiness signal
- failure evidence when verification fails
- summary inputs for `build_manifest`

So the Build Verifier is the formal gatekeeper between repo generation and build completion.

---

10. Position in the Build Mode flow

The Build Verifier should run after:

- repo generation is complete enough to inspect
- required manifests/records are available enough to check

It should run before:

- final success closure
- export packaging

So its flow position is:

1. request
2. approval
3. planning
4. workspace prep
5. repo generation
6. record writing
7. **verification**

8. export if allowed
9. closure

This makes it the final validation step before handoff.

---

11. Inputs to the Build Verifier

At minimum, the contract should define these input classes:

- `run_id`
- `build_session_id`
- `build_plan_ref`
- `repo_ref`
- `repo_manifest_ref`
- `verification_command_set_ref`
- verification scope rules
- required output expectations
- required structure expectations
- manifest/record expectations

Optional but useful:

- `required_file_inventory_ref`
- `repo_blueprint_ref`
- command execution environment context

- placeholder scan rule pack

These inputs should be enough to run verification without inventing new completion standards midstream.

---

12. Outputs from the Build Verifier

At minimum, the contract should define that the Build Verifier produces:

- `verification_report`
- category-level verification results
- final pass/fail verification result
- export-readiness result
- blocking findings
- detailed command/check references if applicable

It should not claim to produce the export itself or final lifecycle closure by itself.

---

13. Required verification categories

The contract should require the verifier to cover, at minimum, these verification categories:

13.1 Required output presence

Confirm required outputs exist.

13.2 Repo structure correctness

Confirm repo structure matches the expected shape/profile.

13.3 Placeholder / unresolved gap scan

Confirm no critical unresolved placeholders remain.

13.4 Manifest and record completeness

Confirm required manifests/records exist and are sufficiently complete.

13.5 Required command/check execution

Confirm required executable/static checks passed.

13.6 Export readiness

Confirm export can proceed if it was requested.

These are the minimum Build Mode verification categories.

---

14. Verification pass/fail rules

The `build_verifier_contract` should define that verification passes only when:

- all required blocking categories pass
- no critical unresolved findings remain
- required records are complete enough
- required commands/checks pass where required
- output consistency is sufficient for completion

Verification fails when:

- any required blocking category fails
- a critical unresolved placeholder remains
- manifest completeness is insufficient
- required command/check execution fails
- output inconsistency blocks trusted completion

This keeps pass/fail real.

---

15. No-guess verification rules

The Build Verifier should enforce these minimum no-guess rules:

15.1 No courtesy pass rule

It must not pass a repo just because it is “close enough.”

15.2 No hidden placeholder tolerance

It must not ignore critical unresolved placeholder failures unless policy explicitly allows them.

15.3 No inferred completeness rule

It must not assume missing manifests, missing files, or missing outputs are “probably fine.”

15.4 No export-by-assumption rule

It must not mark export as ready unless the required conditions truly passed.

These rules keep verification from quietly weakening the whole system.

---

16. Failure behavior rules

The contract should require the verifier to:

- record failed categories
- record blocking findings
- produce a failed `verification_report`
- surface enough detail for `failure_report` generation if needed
- prevent the build from moving to `passed`
- prevent export when required checks fail

This is the heart of verification as a gate rather than a formality.

---

17. Export-readiness rules

The Build Verifier should only mark export ready when:

- export was requested or export evaluation is relevant
- required verification categories passed
- no blocking findings remain
- required output artifacts exist
- required manifest/record checks passed

If those conditions are not true, it must set export readiness to false.

That protects the transition from:
`passed` → `exported`

---

18. Coordination with the Build Runner

The verifier should run inside the sequence coordinated by the Build Runner.

That means the contract should require the verifier to:

- accept the repo/check targets handed off by the runner
- return a structured verification result to the runner
- support state progression to `passed` or `failed`
- avoid making unrelated lifecycle decisions outside verification scope

This keeps verification authoritative but bounded.

---

19. Audit obligations

The Build Verifier should ensure verification is traceable enough to answer:

- what was checked

- what categories passed
- what categories failed
- what findings blocked completion
- whether export was allowed
- when verification started and ended

This does not mean the verifier owns all audit infrastructure, but it must produce enough structured evidence for auditability.

---

20. Minimum required fields

At the document level, this contract should at minimum define:

- `contract_id`
- `contract_name`
- `version`
- `purpose`
- `scope`
- `governing_principle`
- `required_inputs`
- `required_verification_categories`
- `pass_conditions`
- `fail_conditions`
- `no_guess_rules`

- `produced_artifacts`
- `export_readiness_rules`
- `audit_obligations`
- `non_responsibilities`

---

21. Suggested contract shape

A compact logical shape could be:


```
{
 "contract_id": "BM-A32",
 "contract_name": "build_verifier_contract",
 "version": "1.0.0",
 "purpose": "Evaluate whether the generated repository satisfies the required Build Mode completion checks.",
 "scope": "One Build Mode verification event for one build session.",
 "governing_principle": "A generated repo must earn completion through explicit checks.",
 "required_inputs": [
   "build_plan",
   "repo_ref",
   "repo_manifest_ref",
   "verification_scope_rules",
   "verification_command_set_ref"
 ],
 "required_verification_categories": [
   "required_output_presence",
   "repo_structure_correctness",
   "placeholder_scan",
   "manifest_completeness",
   "required_command_execution",
   "export_readiness"
 ],
 "pass_conditions": [
   "all_required_blocking_categories_pass",
   "no_critical_placeholders_remain",
   "required_records_are_complete",
   "required_checks_pass"
```

```
],
"fail_conditions": [
  "required_blocking_category_failed",
  "critical_placeholder_found",
  "required_manifest_incomplete",
  "required_command_failed"
],
"no_guess_rules": [
  "no_courtesy_pass",
  "no_hidden_placeholder_tolerance",
  "no_inferred_completeness",
  "no_export_by_assumption"
],
"produced_artifacts": [
  "verification_report"
],
"export_readiness_rules": [
  "export_requires_required_checks_passed",
  "export_requires_no_blocking_findings",
  "export_requires_required_output_artifacts"
],
"audit_obligations": [
  "record_categories_checked",
  "record_blocking_findings",
  "record_final_verification_result",
  "record_export_readiness_result"
],
"non_responsibilities": [
  "repo_generation",
  "export_packaging",
  "product_scope_definition",
  "policy_definition"
]
}
```


That is enough for a first version.

---

22. Validation rules

At minimum, validation should require:

- contract identity fields present

- required inputs present
- required verification categories present
- pass conditions present
- fail conditions present
- no-guess rules present
- export readiness rules present
- produced artifacts present
- non-responsibilities present

Suggested consistency rules:

- verification categories must align with Build Mode verification scope
- pass/fail conditions must align with `verification_report`
- export readiness rules must not allow export before required pass conditions
- non-responsibilities must not contradict the verification scope

---

23. Success criteria for this artifact

`build_verifier_contract` is successful when it clearly defines the Build Verifier as the component that evaluates the generated repository against the required Build Mode checks, including what it must inspect, what counts as pass or fail, what no-guess rules it must enforce, when export may be allowed, and what verification evidence it must produce.

---

24. Short version

BM-A32 — `build_verifier_contract`
Defines the component contract for the Build Verifier. It specifies how the generated repository is evaluated against the required Build Mode verification categories, what counts as pass or fail,

what no-guess verification rules apply, how export readiness is determined, and how the `verification_report` is produced as the formal proof that the build either earned completion or failed it.

BM-A33

BM-A33 — `export_packager_contract`

1. Purpose

Define the contract for the component that packages a successful Build Mode output into its portable export artifact.

This artifact exists because export is not just “zip the folder.” It is a governed handoff step that happens only after the build has passed the required checks. The Export Packager is the component that performs that packaging, but only inside the export rules already defined by Build Mode.

---

2. Why this artifact exists

A build can be valid without being exported, and export can fail even after the build passes.

So the system needs one component responsible for:

- checking export-packaging prerequisites at the execution level
- collecting the allowed package contents
- creating the portable artifact
- recording export-package output details
- stopping if packaging cannot complete correctly

`export_packager_contract` exists to define that role cleanly.

Without it, export behavior gets treated like an afterthought or gets mixed into the Build Runner or verifier in a way that muddies responsibility.

---

3. Core definition

`export_packager_contract` is the formal component contract that defines the responsibilities, inputs, outputs, packaging rules, export-stop conditions, and handoff obligations of the Export Packager, the component responsible for creating the portable export artifact for a successful Build Mode run.

---

4. Core principle

Only a passed build may be packaged, and packaging must reflect the governed output set rather than invent a new one.

That is the main principle behind this contract.

---

5. What this contract represents

`export_packager_contract` represents:

- the packaging logic boundary inside Build Mode
- the component that turns a passed repo output into a portable artifact
- the final handoff-packaging contract in the build branch
- the controlled bridge from `passed` to `exported`

It is not:

- the Build Runner
- the Build Verifier
- the build manifest
- the export record itself
- the zip artifact itself

It is the export-packaging component contract.

---

6. What this component is responsible for

At minimum, the Export Packager should be responsible for:

- checking that export was requested and is allowed
- collecting the permitted export contents
- assembling the package according to the active package rules
- writing the portable export artifact
- producing the export-event details needed for `export_record`
- failing clearly when packaging cannot complete

That is the minimum serious packaging responsibility set.

---

7. Required responsibilities

The `export_packager_contract` should support these responsibilities.

7.1 Export-package creation

It must create the portable export artifact when conditions allow.

7.2 Package-content enforcement

It must include only the governed export contents.

7.3 Export-artifact writing

It must write the package to the correct export location.

7.4 Export-result handoff

It must provide the data needed to finalize `export_record` and closure.

7.5 Packaging-failure enforcement

It must fail explicitly when export cannot be completed correctly.

---

8. What this component does not do

The Export Packager should not:

- decide whether the build passed verification
- redefine export policy
- invent missing export prerequisites
- repair failed builds by packaging them anyway
- redefine the repo contents
- replace the export record or build manifest

It packages the eligible output.
It does not decide whether ineligible output should become eligible.

---

9. Relationship to other artifacts and components

This contract sits at the very end of active Build Mode execution.

It consumes or depends on

- `build_manifest`

- `verification_report`
- export request intent
- export eligibility decision/gate result
- generated repo output
- package-content rules
- export destination/workspace context

It produces or feeds

- export artifact
- export artifact metadata
- package-content summary
- export completion/failure details
- data used to finalize `export_record`
- final transition support to `exported`

So the Export Packager is the component that makes the portable handoff real.

---

10. Position in the Build Mode flow

The Export Packager should run after:

- verification has passed
- export is requested
- export is allowed
- required output artifacts exist

It should run before:

- final `exported` closure

So its flow position is:

1. request
2. approval
3. planning
4. generation
5. verification
6. **export packaging**
7. final exported closure

That makes it the last active component in the happy path.

---

11. Inputs to the Export Packager

At minimum, the contract should define these input classes:

- `run_id`
- `build_session_id`
- `build_manifest_ref`
- `verification_report_ref`
- export-request state
- export-eligibility state or gate result
- repo/output paths
- package-content rules
- export destination path/context

Optional but useful:

- integrity-hash settings
- artifact-naming convention rules
- inclusion/exclusion filters
- file-size or artifact policy constraints

These inputs should be enough to package the output without deciding new policy during packaging.

---

12. Outputs from the Export Packager

At minimum, the contract should define that the Export Packager produces:

- portable export artifact
- export artifact path/reference
- package format
- package-content summary
- packaging success/failure outcome
- artifact metadata such as size/hash if enabled

It should not claim to produce verification truth or build truth.

---

13. Package-content rules

The contract should define what content the Export Packager may include.

At minimum, it should always include:

- generated repo

Depending on policy/profile, it may also include:

- `build_manifest`
- `repo_manifest`
- `verification_report`

The contract should explicitly forbid packaging content that is:

- outside the governed export set
- unrelated to the run
- incomplete or invalid in a way that violates export rules

So package composition must be governed, not casual.

---

14. Packaging preconditions

The Export Packager should only run when all of these are true:

- export was requested
- verification passed
- export is allowed
- required output artifacts exist
- export destination is writable/valid
- package contents can be resolved coherently

If any of those fail, the component must not fabricate a successful export.

---

15. No-guess packaging rules

The Export Packager should enforce these minimum no-guess rules:

15.1 No export without request

It must not create a portable artifact when export was not requested.

15.2 No export without pass

It must not package outputs that did not pass required verification.

15.3 No missing-artifact substitution

If a required export content item is missing, it must fail or block rather than silently omitting it when that omission would violate policy.

15.4 No package-scope expansion

It must not add unrelated artifacts or extra outputs outside the governed export set.

15.5 No false success artifact

It must not create a package and imply valid export when packaging actually failed or was incomplete.

These rules protect the trustworthiness of the zip artifact.

---

16. Packaging stop conditions

The contract should require the Export Packager to stop when:

- export was not requested
- export is not allowed
- required verified output artifacts are missing
- package destination cannot be written
- package contents cannot be assembled coherently
- artifact creation fails
- required export metadata cannot be produced when policy requires it

This keeps packaging honest.

---

17. Packaging failure behavior

If packaging fails, the Export Packager should:

- surface explicit export failure outcome
- preserve enough detail for `export_record`
- avoid marking export as successful
- leave the already-passed build result intact when appropriate
- support failure explanation without pretending the portable handoff exists

This is important because:

- build success
  and
- export success

are related but not identical.

---

18. Handoff obligations

The Export Packager should hand off enough data to finalize export-related records.

At minimum, it should hand off:

- export artifact reference/path
- package format
- package-content summary
- packaging result
- artifact metadata if tracked
- failure reason if packaging failed

This supports `export_record` and final lifecycle closure.

---

19. Coordination with the Build Runner

The Export Packager should operate inside the sequence coordinated by the Build Runner.

That means the contract should require the packager to:

- run only when invoked in an eligible export context
- return a structured export result to the runner
- support the final transition to `exported` only on success
- avoid making unrelated lifecycle or verification decisions

This keeps packaging specialized and bounded.

---

20. Audit obligations

The Export Packager should ensure export packaging is traceable enough to answer:

- what was packaged
- when it was packaged

- what artifact was produced
- whether packaging succeeded or failed
- why it failed if it failed

This does not mean the packager owns the full audit system, but it must provide enough export evidence for auditability.

---

21. Minimum required fields

At the document level, this contract should at minimum define:

- `contract_id`
- `contract_name`
- `version`
- `purpose`
- `scope`
- `governing_principle`
- `required_inputs`
- `packaging_preconditions`
- `package_content_rules`
- `no_guess_rules`
- `stop_conditions`
- `produced_outputs`
- `handoff_requirements`
- `audit_obligations`
- `non_responsibilities`

---

22. Suggested contract shape

A compact logical shape could be:

```
{
  "contract_id": "BM-A33",
  "contract_name": "export_packager_contract",
  "version": "1.0.0",
  "purpose": "Create the portable export artifact for a successful Build Mode run.",
  "scope": "One Build Mode export-packaging event for one build session.",
  "governing_principle": "Only a passed build may be packaged, and packaging must reflect the governed output set.",
```

"required_inputs": [
  "build_manifest_ref",
  "verification_report_ref",
  "export_request_state",
  "export_eligibility_state",
  "repo_output_path",
  "package_content_rules",
  "export_destination_context"
],
"packaging_preconditions": [
  "export_requested",
  "verification_passed",
  "export_allowed",
  "required_output_artifacts_exist",
  "destination_is_writable"
],
"package_content_rules": [
  "include_generated_repo",
  "include_only_governed_supporting_artifacts",
  "exclude_unrelated_run_artifacts"
],
"no_guess_rules": [
  "no_export_without_request",
  "no_export_without_pass",
  "no_missing_artifact_substitution",
  "no_package_scope_expansion",
  "no_false_success_artifact"
],
"stop_conditions": [
  "export_not_requested",
  "export_not_allowed",
  "required_export_content_missing",
  "destination_write_failure",
  "artifact_creation_failure"
],
"produced_outputs": [
  "portable_export_artifact",
  "artifact_reference",
  "package_content_summary",
  "packaging_result"
],
"handoff_requirements": [
  "artifact_ref",
  "package_format",

```
    "result",
    "failure_reason_if_any"
  ],
  "audit_obligations": [
    "record_packaged_content_summary",
    "record_artifact_reference",
    "record_packaging_result",
    "record_packaging_failure_reason_if_any"
  ],
  "non_responsibilities": [
    "verification_decision",
    "policy_definition",
    "repo_generation",
    "build_state_definition"
  ]
}
```

That is enough for a first version.

---

23. Validation rules

At minimum, validation should require:

- contract identity fields present
- required inputs present
- packaging preconditions present
- package-content rules present
- no-guess rules present
- stop conditions present
- produced outputs present
- non-responsibilities present

Suggested consistency rules:

- packaging preconditions must align with Build Mode packaging rules
- no-guess rules must prevent export-before-pass behavior
- produced outputs must align with `export_record`
- non-responsibilities must not contradict packaging scope

---

24. Success criteria for this artifact

`export_packager_contract` is successful when it clearly defines the Export Packager as the component that creates the portable export artifact only after a valid passed build, including what it must read, what it may package, what it must never assume, when it must stop, and what export-result data it must hand off for final closure and audit.

---

25. Short version

BM-A33 — `export_packager_contract`
Defines the component contract for the Export Packager. It specifies how a passed Build Mode output is packaged into its portable export artifact, including preconditions for packaging, allowed package contents, no-guess packaging rules, stop conditions, produced outputs, and the export-result data required to finalize `export_record` and the final `exported` state.

Repo mapping

Build Mode Repo Mapping

1. Main rule

Build Mode should live **inside the Axion engine**, not in the root server or App layer.

So the center of gravity is:

`AXION-TEST-SUITE-V2/Axion/src/core/`

The repo already supports this split because you have:

- `core/controlPlane/`
- `core/kit/`
- `core/verification/`
- `core/agents/`
- `core/schemas/`
- `cli/commands/`
- `.axion/runs/`

That means Build Mode can be added as a clean first-class subsystem.

---

2. New primary folder

Create a new core branch here:

`Axion/src/core/buildMode/`

This should be the main Build Mode implementation folder.

Put these implementation files there

Axion/src/core/buildMode/
index.ts
model.ts
runner.ts
planner.ts
generator.ts
verifier.ts
exporter.ts
workspace.ts
manifest.ts
failures.ts
policies.ts
phases.ts

What each file maps to

`model.ts`

Owns runtime types/models for:

- `build_request`
- `build_state`
- `build_plan`
- `build_manifest`
- `repo_manifest`
- `verification_report`
- `export_record`
- `failure_report`

`runner.ts`

Maps to:

- `build_runner_contract`

`planner.ts`

Maps to:

- `build_planner_contract`

`generator.ts`

Maps to:

- `repo_generator_contract`

`verifier.ts`

Maps to:

- `build_verifier_contract`

`exporter.ts`

Maps to:

- `export_packager_contract`

`workspace.ts`

Owns:

- build workspace creation
- repo output folders
- export folders

- manifest locations

`manifest.ts`

Owns writing:

- `build_manifest`
- `repo_manifest`

`failures.ts`

Owns:

- failure classification
- failure helpers
- `failure_report` writing support

`policies.ts`

Owns build-mode-specific policy helpers:

- entry checks
- export checks
- no-guess enforcement helpers

`phases.ts`

Owns:

- build phase IDs
- phase order
- phase transition helpers

`index.ts`

Exports the Build Mode module surface.

---

3. Control plane extension points

Do **not** put all Build Mode control logic into `buildMode/` only.
The main control plane should still know that builds exist.

Extend:

```
Axion/src/core/controlPlane/
 api.ts
 model.ts
 store.ts
 policies.ts
 releases.ts
 audit.ts
```

What to add there

`controlPlane/model.ts`

Add build-related control-plane models for:

- build request refs
- build session refs
- build state refs
- export refs

`controlPlane/store.ts`

Store and retrieve:

- current build state

- active build session
- build output refs

`controlPlane/api.ts`

Expose control-plane actions like:

- request build
- approve build
- start build execution
- finalize build
- register export

`controlPlane/policies.ts`

Add hooks for:

- build allowed?
- export allowed?
- unresolved blockers tolerated?

`controlPlane/releases.ts`

Only for the relationship to release/export packaging, not actual repo generation.

`controlPlane/audit.ts`

Register:

- build request event
- build start

- verification result
- export result
- failure result

---

4. Agent layer mapping

You already have:

`Axion/src/core/agents/build.ts`

Keep that file as the **build agent definition layer**, not the whole build engine.

Keep in `core/agents/build.ts`

- build-agent identity
- permissions
- allowed capabilities
- prompt/behavior envelope
- constraints for build agent execution

Do not put here

- runner flow
- repo generation pipeline
- manifest writing
- export logic

So:

- `core/agents/build.ts` = build agent role
- `core/buildMode/*` = actual Build Mode system

---

5. Schema location

You already have:

`Axion/src/core/schemas/`

That is where the Build Mode schemas should go.

Create:

```
Axion/src/core/schemas/buildMode/
 build_request.schema.json
 build_state.schema.json
 build_plan.schema.json
 build_manifest.schema.json
 repo_manifest.schema.json
 verification_report.schema.json
 export_record.schema.json
 failure_report.schema.json
```

And update:

`Axion/src/core/schemas/index.ts`

to export/register them.

This keeps schema truth centralized instead of burying JSON schemas inside runtime files.

---

6. Kit integration points

Build Mode should consume Agent Kits through the existing kit system, not bypass it.

Relevant existing folder:

`Axion/src/core/kit/`

Build Mode should integrate with

- `kit/loader.ts`
- `kit/manifest.ts`
- `kit/validate.ts`
- `kit/build.ts`

What Build Mode needs from kit layer

- approved kit load
- kit manifest lookup
- build-basis ref resolution
- kit validation before planning

Do **not** duplicate kit loading inside Build Mode if the kit layer already owns it.

---

7. Verification integration points

You already have:

`Axion/src/core/verification/`

Build Mode verifier should use or extend this, not replace it completely.

Recommended split

- `core/verification/` = shared verification infrastructure

- `core/buildMode/verifier.ts` = Build-Mode-specific verification orchestration

So Build Mode verifier can call into:

- `verification/runner.ts`
- `verification/completion.ts`
- `verification/policy.ts`

while still owning Build-Mode-specific report writing.

---

8. CLI mapping

You already have command infrastructure at:

`Axion/src/cli/commands/`

Add Build Mode commands there.

Create:

Axion/src/cli/commands/requestBuild.ts
Axion/src/cli/commands/runBuildMode.ts
Axion/src/cli/commands/exportProjectRepo.ts

Purpose

`requestBuild.ts`

Creates or registers:

- `build_request`

`runBuildMode.ts`

Main end-to-end Build Mode trigger:

- load build request
- derive plan
- run build
- verify
- export if requested

`exportProjectRepo.ts`

Optional explicit export command if you want export callable separately after a passed build.

Also update

`Axion/src/cli/axion.ts`

to register the commands.

---

9. Run-folder output mapping

This is very important.

Build Mode runtime artifacts should live under the existing run storage:

`Axion/.axion/runs/<RUN_ID>/`

Add this branch:

```
Axion/.axion/runs/<RUN_ID>/build/
 build_request.json
 build_state.json
 build_plan.json
 failure_report.json

 repo/
```

```
  ...generated project repo...

manifests/
  build_manifest.json
  repo_manifest.json

verification/
  verification_report.json
  command_results.json

exports/
  project_repo.zip
  export_record.json

workspace/
  ...temp/staging/intermediate files...
```

This is the cleanest runtime contract for your current repo.

Mapping by artifact

Root build folder

- `build_request.json`
- `build_state.json`
- `build_plan.json`
- `failure_report.json`

`manifests/`

- `build_manifest.json`
- `repo_manifest.json`

`verification/`

- `verification_report.json`
- `command_results.json`

`exports/`

- `export_record.json`
- `project_repo.zip`

`repo/`

- actual generated codebase

`workspace/`

- temp/staging/in-progress materialization

---

10. Docs/contracts placement

You have a docs system in:

`Axion/docs_system/`

So the Build Mode contract docs should live there, not only in code comments.

Add a new branch:

```
Axion/docs_system/BUILD/
 BM-00.md
 BM-01.md
 ...
 BM-18.md

 artifacts/
```

BM-A01-build_control_plane_contract.md
BM-A05-build_request.md
BM-A06-build_state.md
BM-A09-build_plan.md
BM-A16-build_manifest.md
BM-A17-repo_manifest.md
BM-A19-verification_report.md
BM-A21-export_record.md
BM-A22-failure_report.md
BM-A25-build_runner_contract.md
BM-A28-build_planner_contract.md
BM-A30-repo_generator_contract.md
BM-A32-build_verifier_contract.md
BM-A33-export_packager_contract.md

That keeps your paper design inside the same docs discipline as the rest of Axion.

---

11. Root server mapping

The root app server should only expose/trip Build Mode, not own it.

Relevant existing files:

- `server/routes.ts`
- `server/pipeline-runner.ts`

Later, add API hooks there for:

- request build
- get build status
- get verification result
- download export zip

But do **not** put core repo generation there.

So:

- `server/` = delivery/API layer
- `Axion/src/core/buildMode/` = real logic

---

12. App mapping

The front-end app at:

`App/src/`

should only become the UI surface later.

Likely future UI pieces:

- Build request button
- Build status panel
- Verification result panel
- Download zip button

But none of that should come before the core engine mapping above.

---

13. Exact artifact-to-file map

Core contracts/docs

```
Axion/docs_system/BUILD/artifacts/
 BM-A01-build_control_plane_contract.md
 BM-A25-build_runner_contract.md
 BM-A28-build_planner_contract.md
 BM-A30-repo_generator_contract.md
 BM-A32-build_verifier_contract.md
```

BM-A33-export_packager_contract.md

JSON schemas

Axion/src/core/schemas/buildMode/
- build_request.schema.json
- build_state.schema.json
- build_plan.schema.json
- build_manifest.schema.json
- repo_manifest.schema.json
- verification_report.schema.json
- export_record.schema.json
- failure_report.schema.json

Runtime/core implementation

Axion/src/core/buildMode/
- model.ts
- runner.ts
- planner.ts
- generator.ts
- verifier.ts
- exporter.ts
- workspace.ts
- manifest.ts
- failures.ts
- policies.ts
- phases.ts
- index.ts

CLI

Axion/src/cli/commands/
- requestBuild.ts
- runBuildMode.ts
- exportProjectRepo.ts

Run artifacts

Axion/.axion/runs/<RUN_ID>/build/
- build_request.json
- build_state.json
- build_plan.json
- failure_report.json

manifests/build_manifest.json
manifests/repo_manifest.json
verification/verification_report.json
verification/command_results.json
exports/export_record.json
exports/project_repo.zip
repo/
workspace/

---

14. Best implementation order in this repo

First

- `src/core/schemas/buildMode/`
- `src/core/buildMode/model.ts`
- run-folder contract

Second

- `src/core/buildMode/planner.ts`
- `src/core/buildMode/phases.ts`
- `src/core/buildMode/policies.ts`

Third

- `src/core/buildMode/generator.ts`
- `src/core/buildMode/workspace.ts`
- `src/core/buildMode/manifest.ts`

Fourth

- `src/core/buildMode/verifier.ts`
- `src/core/buildMode/failures.ts`
- `src/core/buildMode/exporter.ts`

Fifth

- `src/core/buildMode/runner.ts`

Sixth

- control-plane extensions
- CLI commands

Last

- server routes
- UI

---

15. Bottom line

The clean mapping is:

- **Build Mode engine** → `Axion/src/core/buildMode/`
- **schema truth** → `Axion/src/core/schemas/buildMode/`

- **main control-plane hooks** → `Axion/src/core/controlPlane/`
- **kit integration** → `Axion/src/core/kit/`
- **verification infrastructure reuse** → `Axion/src/core/verification/`
- **CLI entry** → `Axion/src/cli/commands/`
- **runtime artifacts** → `Axion/.axion/runs/<RUN_ID>/build/`
- **UI/API later** → root `server/` and `App/`

Repo Mapping Physical List

Build Mode — Physical File List

1. Core implementation files

Create these first inside the Axion engine:

```
Axion/src/core/buildMode/
  index.ts
  model.ts
  phases.ts
  policies.ts
  workspace.ts
  manifest.ts
  failures.ts
  planner.ts
  generator.ts
  verifier.ts
  exporter.ts
  runner.ts
```

---

2. Schema files

These are the artifact schemas:

```
Axion/src/core/schemas/buildMode/
  build_request.schema.json
  build_state.schema.json
  build_plan.schema.json
  build_manifest.schema.json
  repo_manifest.schema.json
  verification_report.schema.json
  export_record.schema.json
  failure_report.schema.json
```

Also update:

Axion/src/core/schemas/index.ts

to register/export them.

---

3. Control plane files to extend

Do not create a separate control plane folder for Build Mode first. Extend the existing one:

```
Axion/src/core/controlPlane/
  model.ts
  store.ts
  api.ts
  policies.ts
  audit.ts
  releases.ts
```

---

4. CLI files

These are the first CLI entry points:

```
Axion/src/cli/commands/
  requestBuild.ts
  runBuildMode.ts
  exportProjectRepo.ts
```

Also update:

Axion/src/cli/axion.ts

---

5. Docs files

These are the Build Mode docs/contract files:

```
Axion/docs_system/BUILD/
  BM-00.md
  BM-01.md
  BM-02.md
  BM-03.md
  BM-04.md
  BM-05.md
  BM-06.md
  BM-07.md
```

```
  BM-08.md
  BM-09.md
  BM-10.md
  BM-11.md
  BM-12.md
  BM-13.md
  BM-14.md
  BM-15.md
  BM-16.md
  BM-17.md
  BM-18.md
```

Artifact docs:

```
Axion/docs_system/BUILD/artifacts/
  BM-A01-build_control_plane_contract.md
  BM-A05-build_request.md
  BM-A06-build_state.md
  BM-A09-build_plan.md
  BM-A16-build_manifest.md
  BM-A17-repo_manifest.md
  BM-A19-verification_report.md
  BM-A21-export_record.md
  BM-A22-failure_report.md
  BM-A25-build_runner_contract.md
  BM-A28-build_planner_contract.md
  BM-A30-repo_generator_contract.md
  BM-A32-build_verifier_contract.md
  BM-A33-export_packager_contract.md
```

---

6. Runtime artifact file contract

Per run, Build Mode should write to:

```
Axion/.axion/runs/<RUN_ID>/build/
  build_request.json
  build_state.json
  build_plan.json
  failure_report.json

  repo/
```

```
manifests/
  build_manifest.json
  repo_manifest.json

verification/
  verification_report.json
  command_results.json

exports/
  export_record.json
  project_repo.zip

workspace/
```

---

7. What each implementation file owns

`model.ts`

Owns TS types/interfaces for:

- build_request
- build_state
- build_plan
- build_manifest
- repo_manifest
- verification_report
- export_record
- failure_report

`phases.ts`

Owns:

- phase IDs
- phase order
- phase constants

`policies.ts`

Owns:

- build allowed checks
- export allowed checks
- no-guess helpers

workspace.ts

Owns:

- build folder creation
- repo path resolution
- manifest/export/verification path helpers

manifest.ts

Owns:

- build_manifest writing
- repo_manifest writing

failures.ts

Owns:

- failure classes
- failure reason helpers
- failure_report writing helpers

planner.ts

Owns:

- build_plan derivation

generator.ts

Owns:

- repo materialization

verifier.ts

Owns:

- verification execution

- verification_report production

`exporter.ts`

Owns:

- zip packaging
- export output metadata

`runner.ts`

Owns:

- orchestration of full Build Mode flow

`index.ts`

Owns:

- module exports

---

8. Exact first-build order

Wave 1

Axion/src/core/schemas/buildMode/
Axion/src/core/buildMode/model.ts
Axion/src/core/buildMode/phases.ts

Wave 2

Axion/src/core/buildMode/policies.ts
Axion/src/core/buildMode/workspace.ts
Axion/src/core/buildMode/planner.ts

Wave 3

Axion/src/core/buildMode/manifest.ts
Axion/src/core/buildMode/failures.ts
Axion/src/core/buildMode/generator.ts

Wave 4

Axion/src/core/buildMode/verifier.ts
Axion/src/core/buildMode/exporter.ts
Axion/src/core/buildMode/runner.ts

Wave 5

Axion/src/core/controlPlane/*
Axion/src/cli/commands/*
Axion/src/cli/axion.ts

---

9. Minimum first creation set

If you want the smallest real starting set, create these first:

Axion/src/core/buildMode/model.ts
Axion/src/core/buildMode/phases.ts
Axion/src/core/buildMode/planner.ts
Axion/src/core/buildMode/generator.ts
Axion/src/core/buildMode/verifier.ts
Axion/src/core/buildMode/exporter.ts
Axion/src/core/buildMode/runner.ts

Axion/src/core/schemas/buildMode/build_request.schema.json
Axion/src/core/schemas/buildMode/build_state.schema.json
Axion/src/core/schemas/buildMode/build_plan.schema.json
Axion/src/core/schemas/buildMode/build_manifest.schema.json
Axion/src/core/schemas/buildMode/repo_manifest.schema.json
Axion/src/core/schemas/buildMode/verification_report.schema.json
Axion/src/core/schemas/buildMode/export_record.schema.json
Axion/src/core/schemas/buildMode/failure_report.schema.json

model.ts

`model.ts` — Contents Outline

Purpose

This file should be the single TypeScript model surface for Build Mode runtime objects.

It should define the TS types and enums that the rest of Build Mode uses so:

- planner
- generator
- verifier
- exporter
- runner
- control-plane hooks

are all speaking the same object language.

---

1. File path

Axion/src/core/buildMode/model.ts

---

2. What `model.ts` should contain

A. ID/type aliases

Start with shared identity aliases for Build Mode objects.

Examples:

- `BuildRequestId`
- `BuildSessionId`
- `BuildPlanId`
- `BuildManifestId`
- `RepoManifestId`
- `VerificationReportId`
- `ExportRecordId`
- `FailureReportId`

Also shared refs like:

- `RunId`
- `ArtifactRef`
- `TimestampIso`

B. Enums / literal unions

This file should define the main Build Mode enums.

`BuildRequestMode`

"build_repo" | "build_repo_and_export"

`BuildLifecycleState`

"not_requested" | "requested" | "approved" | "building" | "verifying" | "failed" | "passed" | "exported"

BuildFailureClass

"missing_kit_input"
| "invalid_build_inputs"
| "eligibility_failure"
| "planning_failure"
| "workspace_failure"
| "repo_generation_failure"
| "verification_failure"
| "export_failure"
| "record_failure"

BuildFailureStage

"entry" | "planning" | "workspace" | "generation" | "verification" | "export" | "closure"

BuildFinalResult

"failure" | "success" | "success_with_export"

ExportResult

"succeeded" | "failed" | "blocked"

RepoType

Examples:

"web_app" | "mobile_app" | "api_service" | "fullstack_monorepo"

RepoShape

Examples:

"single_app" | "client_server_split" | "monorepo_with_shared" | "mobile_plus_backend"

RepoCompletenessStatus

"partial" | "structurally_complete" | "verified_complete"

VerificationCategoryStatus

"passed" | "failed" | "skipped" | "not_applicable"

FindingSeverity

"info" | "warning" | "error"

`PackageFormat`

For now:

"zip"

C. Core helper interfaces

Define some reusable small interfaces.

Examples:

`ArtifactPointer`

```
{
 ref: string;
 path?: string;
}
```

`PhaseStatus`

```
{
 phaseId: string;
 order: number;
 status: "pending" | "running" | "passed" | "failed" | "skipped";
 summary?: string;
}
```

`Finding`

```
{
 findingId: string;
 severity: FindingSeverity;
 categoryId: string;
 code: string;
 message: string;
 blocking: boolean;
 targetRef?: string;
}
```

`CommandSummary`

```
{
 install?: string;
```

```
 dev?: string;
 typecheck?: string;
 test?: string;
 build?: string;
}
```

---

3. Main interfaces in `model.ts`

A. `BuildRequest`

Should match your artifact design.

Core fields:

- `requestId`
- `runId`
- `buildBasisRef`
- `requestedBy`
- `requestedAt`
- `requestMode`
- `exportRequested`
- `requestSource`
- `status`
- `notes`

B. `BuildState`

Core fields:

- `stateId`
- `runId`
- `buildRequestId`
- `buildSessionId`
- `currentState`
- `enteredAt`
- `isTerminal`
- `statusSummary`
- `statusReasonCode`
- `buildBasisRef`
- `updatedAt`
- `previousState?`
- `failureClass?`
- `exportRequested?`

C. `BuildPlan`

Core fields:

- `planId`
- `runId`
- `buildRequestId`
- `buildSessionId`
- `buildBasisRef`
- `derivedFromRefs`
- `buildProfileRef`
- `repoBlueprintRef?`
- `requiredFileInventoryRef?`
- `planCreatedAt`
- `targetRepoType`
- `targetRepoSummary`
- `phases`
- `requiredOutputs`
- `constraints`
- `verificationExpectations`
- `status`

D. `BuildManifest`

Core fields:

- `buildManifestId`
- `runId`
- `buildRequestId`
- `buildSessionId`
- `buildBasisRef`
- `buildPlanRef`
- `buildProfileRef`
- `policySnapshotRef?`
- `startedAt`
- `completedAt?`
- `finalState`
- `result`
- `exportRequested`
- `exportCompleted`
- `repoRef?`
- `repoManifestRef?`
- `verificationReportRef?`
- `exportRecordRef?`

- `failureReportRef?`
- `statusSummary`
- `createdAt`
- `updatedAt`
- `phaseSummary?`
- `notes?`

E. `RepoManifest`

Core fields:

- `repoManifestId`
- `runId`
- `buildManifestId`
- `buildSessionId`
- `repoRef`
- `repoType`
- `repoShape`
- `repoRootPath`
- `moduleAreas`
- `requiredStructureSummary`

- `rootArtifacts`
- `commandSummary`
- `buildProfileRef`
- `repoBlueprintRef?`
- `requiredFileInventoryRef?`
- `completenessStatus`
- `createdAt`
- `updatedAt`
- `workspaceManifestRef?`
- `fileIndexRef?`
- `notes?`

F. `VerificationReport`

Core fields:

- `verificationReportId`
- `runId`
- `buildManifestId`
- `buildSessionId`
- `buildPlanRef`

- `repoRef`
- `repoManifestRef`
- `verificationStartedAt`
- `verificationCompletedAt?`
- `verificationScopeVersion`
- `categories`
- `findings`
- `finalResult`
- `blockingFailureCount`
- `exportReady`
- `commandResultsRef?`
- `createdAt`
- `updatedAt`
- `verificationCommandSetRef?`
- `summary?`

G. `ExportRecord`

Core fields:

- `exportRecordId`

- `runId`
- `buildManifestId`
- `buildSessionId`
- `verificationReportRef`
- `exportRequested`
- `exportAllowed`
- `exportStartedAt?`
- `exportCompletedAt?`
- `packageFormat`
- `packageContentsSummary`
- `exportArtifactRef?`
- `exportArtifactName?`
- `result`
- `failureReasonCode?`
- `statusSummary`
- `createdAt`
- `updatedAt`
- `exportGateResultRef?`
- `artifactSizeBytes?`

- `artifactHash?`
- `notes?`

H. `FailureReport`

Core fields:

- `failureReportId`
- `runId`
- `buildManifestId?`
- `buildSessionId`
- `buildRequestId`
- `failureClass`
- `failureStage`
- `failurePhaseId?`
- `failureState`
- `reasonCode`
- `reasonMessage`
- `blocking`
- `detectedAt`
- `relatedArtifactRefs`

- `partialOutputRefs`
- `exportImpacted`
- `finalOutcome`
- `createdAt`
- `updatedAt`
- `verificationReportRef?`
- `exportRecordRef?`
- `commandResultsRef?`
- `retryHint?`
- `notes?`

---

4. Support types that should live in `model.ts`

`BuildPlanPhase`

Fields:

- `phaseId`
- `order`
- `purpose?`

- `targetAreas?`
- `entryDependencies?`
- `expectedOutputs?`

`VerificationCategoryResult`

Fields:

- `categoryId`
- `categoryName`
- `required`
- `status`
- `blocking`
- `checksRun`
- `summary`

`BuildConstraints`

Fields like:

- `allowScopeExpansion`
- `allowUndeclaredDependencies`
- `blockOnMissingRequiredInputs`
- `blockOnCriticalPlaceholders`

`BuildVerificationExpectations`

Fields like:

- `requireStructureCheck`
- `requirePlaceholderScan`
- `requireManifestCheck`
- `requireCommandChecks`
- `requireExportReadinessWhenRequested`

---

5. Utility functions `model.ts` can contain

Keep this light.

Useful helpers:

- `isTerminalBuildState(state)`
- `isSuccessfulBuildState(state)`
- `buildStateAllowsExport(state)`
- `isExportRequested(mode)`

Do not turn `model.ts` into business logic heavy code.

---

6. What `model.ts` should not contain

Do not put:

- planning logic
- generation logic
- verification execution
- export packaging logic
- filesystem code
- control-plane persistence code

This file should stay a **pure model/types layer**.

---

7. Best internal order for `model.ts`

Write it in this order:

1. primitive aliases
2. enums/literal unions
3. small reusable interfaces
4. main artifact interfaces
5. helper functions
6. exports

That will keep it readable.

---

8. Next schema files

After `model.ts`, define the schema contents in this order:

1. `build_request.schema.json`
2. `build_state.schema.json`
3. `build_plan.schema.json`
4. `build_manifest.schema.json`
5. `repo_manifest.schema.json`
6. `verification_report.schema.json`
7. `export_record.schema.json`
8. `failure_report.schema.json`

We should do them one by one so they stay clean.

Schema

build_request.schema.json

build_state.schema.json

build_plan.schema.json

build_manifest.schema.json

repo_manifest.schema.json

verification_report.schema.json

export_record.schema.json

failure_report.schema.json

build_request.schema.json

`build_request.schema.json` — Contents Outline

1. File path

Axion/src/core/schemas/buildMode/build_request.schema.json

---

2. Purpose

This schema validates the runtime artifact that records the operator's explicit request to enter Build Mode.

It should guarantee that Build Mode only starts from a properly formed request object with:

- a known run
- a known requester
- an explicit request mode
- a valid build-basis reference
- a consistent export flag

---

3. Schema role

This schema is the validation surface for:

- CLI build requests
- API/UI build requests
- persisted `build_request.json` files in run folders
- control-plane ingestion of build activation records

So it should be strict and simple.

---

4. Top-level schema metadata

The schema should include standard metadata like:

- `$schema`
- `$id`
- `title`
- `description`
- `type`

Recommended shape:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "axion/schemas/buildMode/build_request.schema.json",
 "title": "BuildRequest",
 "description": "Records an explicit operator request to enter Build Mode for a specific run.",
 "type": "object"
}
```

---

5. Required properties

The required core properties should be:

- `request_id`
- `run_id`
- `build_basis_ref`
- `requested_by`
- `requested_at`
- `request_mode`

- `export_requested`
- `request_source`

Those are the minimum fields needed for a valid build request.

---

6. Allowed optional properties

Optional fields can be:

- `status`
- `notes`

You can keep this artifact lean by not adding more until needed.

---

7. Property definitions

`request_id`

Type:

- `string`

Purpose:

- unique identifier for the build request record

Recommended constraint:

- non-empty string

Example:

- `BRQ-0001`

`run_id`

Type:

- `string`

Purpose:

- identifies the originating run

Recommended constraint:

- non-empty string

Example:

- `RUN-0001`

`build_basis_ref`

Type:

- `string`

Purpose:

- reference to the approved Agent Kit or active build basis

Recommended constraint:

- non-empty string

Example:

- `KIT-0001`

`requested_by`

Type:

- `string`

Purpose:

- identifies the operator or initiating authority

Recommended constraint:

- non-empty string

Example:

- `operator_id`

`requested_at`

Type:

- `string`
- `format: "date-time"`

Purpose:

- ISO timestamp for when the request was created

Example:

- `2026-03-06T12:00:00Z`

`request_mode`

Type:

- `string`

Enum:

- `build_repo`
- `build_repo_and_export`

Purpose:

- declares the requested output mode

`export_requested`

Type:

- `boolean`

Purpose:

- convenience flag for downstream logic

Consistency:

- must match `request_mode`

`request_source`

Type:

- `string`

Enum:

- `operator_ui`
- `cli`
- `api`

Purpose:

- indicates where the request came from

`status`

Type:

- `string`

Enum:

- `active`
- `superseded`
- `cancelled`

Purpose:

- optional request-record status

Default suggestion:

- `active`

`notes`

Type:

- `string` or `null`

Purpose:

- optional freeform note field

---

8. `additionalProperties`

Set:

"additionalProperties": false

Reason:
Build requests should be strict.
You do not want random fields sneaking into the runtime activation surface.

---

9. Validation rules beyond basic types

The schema should enforce these consistency rules:

Rule 1

If `request_mode = build_repo`
then `export_requested` must be `false`

Rule 2

If `request_mode = build_repo_and_export`
then `export_requested` must be `true`

This should be handled with `allOf` + `if/then`.

---

10. Recommended schema structure

The schema should have these sections:

1. metadata
2. `type: object`
3. `required`
4. `properties`
5. `allOf` conditional consistency rules
6. `additionalProperties: false`

---

11. Example property block

A clean property section would look like:

{

```
  "properties": {
    "request_id": {
      "type": "string",
      "minLength": 1
    },
    "run_id": {
      "type": "string",
      "minLength": 1
    },
    "build_basis_ref": {
      "type": "string",
      "minLength": 1
    },
    "requested_by": {
      "type": "string",
      "minLength": 1
    },
    "requested_at": {
      "type": "string",
      "format": "date-time"
    },
    "request_mode": {
      "type": "string",
      "enum": ["build_repo", "build_repo_and_export"]
    },
    "export_requested": {
      "type": "boolean"
    },
    "request_source": {
      "type": "string",
      "enum": ["operator_ui", "cli", "api"]
    },
    "status": {
      "type": "string",
      "enum": ["active", "superseded", "cancelled"]
    },
    "notes": {
      "type": ["string", "null"]
    }
  }
}
```

---

12. Example conditional rules

Use:

```
{
 "allOf": [
   {
     "if": {
       "properties": { "request_mode": { "const": "build_repo" } },
       "required": ["request_mode"]
     },
     "then": {
       "properties": { "export_requested": { "const": false } }
     }
   },
   {
     "if": {
       "properties": { "request_mode": { "const": "build_repo_and_export" } },
       "required": ["request_mode"]
     },
     "then": {
       "properties": { "export_requested": { "const": true } }
     }
   }
 ]
}
```

---

13. Example valid JSON

```
{
 "request_id": "BRQ-0001",
 "run_id": "RUN-0001",
 "build_basis_ref": "KIT-0001",
 "requested_by": "operator_id",
 "requested_at": "2026-03-06T12:00:00Z",
 "request_mode": "build_repo_and_export",
 "export_requested": true,
 "request_source": "operator_ui",
 "status": "active",
 "notes": null
}
```

---

14. What this schema should reject

It should reject:

- missing `run_id`
- missing `request_mode`
- invalid `request_mode`
- invalid `request_source`
- invalid datetime in `requested_at`
- `build_repo` with `export_requested: true`
- `build_repo_and_export` with `export_requested: false`
- unknown extra fields

---

15. Success criteria

`build_request.schema.json` is successful when it enforces a strict, minimal, and consistent request object that makes Build Mode activation explicit, traceable, and machine-valid.

build_state.schema.json

`build_state.schema.json` — Contents Outline

1. File path

Axion/src/core/schemas/buildMode/build_state.schema.json

---

2. Purpose

This schema validates the runtime artifact that records the current governed lifecycle state of a Build Mode run.

It should guarantee that:

- every build has a clear current state
- terminal vs non-terminal state is consistent
- failed states carry failure classification
- the state object is strict enough for control-plane use

---

3. Schema role

This schema validates:

- persisted `build_state.json` files in run folders
- control-plane state snapshots
- runner/verifier/exporter state writes
- UI/API build-status reads

So it should be strict and lifecycle-aware.

---

4. Top-level schema metadata

Recommended shape:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "axion/schemas/buildMode/build_state.schema.json",
 "title": "BuildState",
 "description": "Records the current governed lifecycle state of a Build Mode run or build session.",
 "type": "object"
}
```

---

5. Required properties

The required core properties should be:

- `state_id`
- `run_id`
- `build_request_id`
- `build_session_id`
- `current_state`
- `entered_at`
- `is_terminal`
- `status_summary`
- `status_reason_code`
- `build_basis_ref`
- `updated_at`

These are the minimum fields needed for a usable state artifact.

---

6. Allowed optional properties

Optional fields can be:

- `previous_state`
- `failure_class`
- `export_requested`

Keep it lean.

---

7. Property definitions

`state_id`

Type:

- `string`

Purpose:

- unique identifier for the state record

Constraint:

- non-empty string

Example:

- `BST-0001`

`run_id`

Type:

- `string`

Purpose:

- originating run ID

`build_request_id`

Type:

- `string`

Purpose:

- reference to the request that activated Build Mode

`build_session_id`

Type:

- `string`

Purpose:

- reference to the concrete build attempt/session

`current_state`

Type:

- `string`

Enum:

- `not_requested`
- `requested`
- `approved`
- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

Purpose:

- official current lifecycle state

`entered_at`

Type:

- `string`
- `format: "date-time"`

Purpose:

- when the current state became active

`is_terminal`

Type:

- `boolean`

Purpose:

- whether the current state is terminal

`status_summary`

Type:

- `string`

Purpose:

- human-readable summary of the state

Constraint:

- non-empty string

`status_reason_code`

Type:

- `string`

Purpose:

- machine-usable summary reason

Suggested examples:

- `request_created`
- `eligibility_passed`
- `build_started`
- `verification_started`
- `verification_passed`
- `verification_failed`
- `export_completed`
- `export_failed`

Constraint:

- non-empty string

`build_basis_ref`

Type:

- `string`

Purpose:

- reference to the approved Agent Kit or active build basis

---

updated_at

Type:

- `string`
- `format: "date-time"`

Purpose:

- last update timestamp for the state record

---

previous_state

Type:

- `string`

Enum:

- `not_requested`
- `requested`
- `approved`
- `building`
- `verifying`
- `failed`
- `passed`
- `exported`

Purpose:

- optional prior state reference

`failure_class`

Type:

- `string`

Enum:

- `missing_kit_input`
- `invalid_build_inputs`
- `eligibility_failure`
- `planning_failure`
- `workspace_failure`
- `repo_generation_failure`
- `verification_failure`
- `export_failure`
- `record_failure`

Purpose:

- required when the build is failed

`export_requested`

Type:

- `boolean`

Purpose:

- convenience flag for downstream/UI logic

---

8. `additionalProperties`

Set:

"additionalProperties": false

Reason:
State artifacts should be strict because control logic depends on them.

---

9. Validation rules beyond basic types

The schema should enforce these consistency rules:

Rule 1

If `current_state = failed`
then:

- `is_terminal = true`
- `failure_class` must be present

Rule 2

If `current_state = passed`
then:

- `is_terminal = true`

Rule 3

If `current_state = exported`
then:

- `is_terminal = true`

Rule 4

If `current_state` is one of:

- `requested`
- `approved`
- `building`
- `verifying`

then:

- `is_terminal = false`

Rule 5

If `current_state = not_requested`
then:

- `is_terminal = false`

You can implement these with `allOf` + `if/then`.

---

10. Recommended schema structure

The schema should have these sections:

1. metadata
2. `type: object`
3. `required`
4. `properties`
5. `allOf` lifecycle consistency rules
6. `additionalProperties: false`

---

11. Example property block

```
{
 "properties": {
   "state_id": {
     "type": "string",
     "minLength": 1
   },
   "run_id": {
     "type": "string",
     "minLength": 1
   },
   "build_request_id": {
     "type": "string",
     "minLength": 1
   },
   "build_session_id": {
     "type": "string",
     "minLength": 1
   },
   "current_state": {
     "type": "string",
     "enum": [
       "not_requested",
       "requested",
```

```
    "approved",
    "building",
    "verifying",
    "failed",
    "passed",
    "exported"
  ]
},
"entered_at": {
  "type": "string",
  "format": "date-time"
},
"is_terminal": {
  "type": "boolean"
},
"status_summary": {
  "type": "string",
  "minLength": 1
},
"status_reason_code": {
  "type": "string",
  "minLength": 1
},
"build_basis_ref": {
  "type": "string",
  "minLength": 1
},
"updated_at": {
  "type": "string",
  "format": "date-time"
},
"previous_state": {
  "type": "string",
  "enum": [
    "not_requested",
    "requested",
    "approved",
    "building",
    "verifying",
    "failed",
    "passed",
    "exported"
  ]
},
```

```
  "failure_class": {
    "type": "string",
    "enum": [
      "missing_kit_input",
      "invalid_build_inputs",
      "eligibility_failure",
      "planning_failure",
      "workspace_failure",
      "repo_generation_failure",
      "verification_failure",
      "export_failure",
      "record_failure"
    ]
  },
  "export_requested": {
    "type": "boolean"
  }
 }
}
```

---

12. Example conditional rules

```
{
 "allOf": [
  {
    "if": {
      "properties": { "current_state": { "const": "failed" } },
      "required": ["current_state"]
    },
    "then": {
      "properties": {
        "is_terminal": { "const": true }
      },
      "required": ["failure_class"]
    }
  },
  {
    "if": {
      "properties": { "current_state": { "const": "passed" } },
      "required": ["current_state"]
    },
    "then": {
      "properties": {
        "is_terminal": { "const": true }
```

```
      }
    }
  },
  {
    "if": {
      "properties": { "current_state": { "const": "exported" } },
      "required": ["current_state"]
    },
    "then": {
      "properties": {
        "is_terminal": { "const": true }
      }
    }
  },
  {
    "if": {
      "properties": {
        "current_state": {
          "enum": ["not_requested", "requested", "approved", "building", "verifying"]
        }
      },
      "required": ["current_state"]
    },
    "then": {
      "properties": {
        "is_terminal": { "const": false }
      }
    }
  }
]
}
```

---

13. Example valid JSON

```
{
 "state_id": "BST-0001",
 "run_id": "RUN-0001",
 "build_request_id": "BRQ-0001",
 "build_session_id": "BSES-0001",
 "current_state": "building",
 "entered_at": "2026-03-06T12:05:00Z",
 "is_terminal": false,
 "status_summary": "Repository generation in progress",
 "status_reason_code": "build_started",
```

"build_basis_ref": "KIT-0001",
"updated_at": "2026-03-06T12:05:00Z",
"previous_state": "approved",
"failure_class": null,
"export_requested": true
}

Note: if you keep `failure_class` optional, omit it when not failed.
Do not allow `null` unless you explicitly choose to.

Cleaner version:

{
"state_id": "BST-0001",
"run_id": "RUN-0001",
"build_request_id": "BRQ-0001",
"build_session_id": "BSES-0001",
"current_state": "building",
"entered_at": "2026-03-06T12:05:00Z",
"is_terminal": false,
"status_summary": "Repository generation in progress",
"status_reason_code": "build_started",
"build_basis_ref": "KIT-0001",
"updated_at": "2026-03-06T12:05:00Z",
"previous_state": "approved",
"export_requested": true
}

---

14. What this schema should reject

It should reject:

- unknown state values
- `failed` with no `failure_class`
- `failed` with `is_terminal: false`
- `passed` with `is_terminal: false`
- `exported` with `is_terminal: false`

- `building` with `is_terminal: true`
- missing `state_id`
- missing `entered_at`
- invalid datetime fields
- unknown extra fields

---

15. Success criteria

`build_state.schema.json` is successful when it enforces a strict, lifecycle-consistent state object that the control plane, runner, and UI can trust as the official current status of a Build Mode run.

build_plan.schema.json

`build_plan.schema.json` — Contents Outline

1. File path

Axion/src/core/schemas/buildMode/build_plan.schema.json

---

2. Purpose

This schema validates the artifact that defines the actionable build path for Build Mode.

It should guarantee that a build plan is not vague. A valid plan must clearly state:

- what run/build it belongs to
- what build basis it came from
- what repo type is being targeted
- what phases will run
- what outputs are required
- what constraints apply
- what verification expectations must later be satisfied

---

3. Schema role

This schema validates:

- persisted `build_plan.json` files in run folders
- planner output before runner execution
- handoff from planning to generation
- any replay/resume logic that depends on a stable plan

So it should be one of the stricter Build Mode schemas.

---

4. Top-level schema metadata

Recommended shape:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "axion/schemas/buildMode/build_plan.schema.json",
 "title": "BuildPlan",
 "description": "Defines the actionable repo-generation path for a Build Mode run.",
 "type": "object"
}
```

---

5. Required properties

The required core properties should be:

- `plan_id`
- `run_id`
- `build_request_id`
- `build_session_id`
- `build_basis_ref`
- `derived_from_refs`
- `build_profile_ref`
- `plan_created_at`
- `target_repo_type`
- `target_repo_summary`
- `phases`

- `required_outputs`
- `constraints`
- `verification_expectations`
- `status`

These are the minimum fields needed for a valid execution plan.

---

6. Allowed optional properties

Optional fields can be:

- `repo_blueprint_ref`
- `required_file_inventory_ref`
- `policy_snapshot_ref`
- `notes`
- `planner_version`

---

7. Property definitions

`plan_id`

Type:

- `string`

Purpose:

- unique identifier for the build plan

Example:

- `BPL-0001`

`run_id`

Type:

- `string`

Purpose:

- originating run ID

`build_request_id`

Type:

- `string`

Purpose:

- reference to the build request

`build_session_id`

Type:

- `string`

Purpose:

- reference to the concrete build attempt

`build_basis_ref`

Type:

- `string`

Purpose:

- reference to the approved Agent Kit

`derived_from_refs`

Type:

- `array` of `string`

Purpose:

- supporting upstream references used to derive the plan

Examples:

- standards snapshot ref
- route spec ref
- entity spec ref

- template selection ref

Constraint:

- at least 1 item

`build_profile_ref`

Type:

- `string`

Purpose:

- locked build profile reference

`repo_blueprint_ref`

Type:

- `string`

Purpose:

- optional reference to expected repo structure artifact

`required_file_inventory_ref`

Type:

- `string`

Purpose:

- optional reference to required file inventory artifact

`policy_snapshot_ref`

Type:

- `string`

Purpose:

- optional reference to effective build/export policy snapshot

---

`plan_created_at`

Type:

- `string`
- `format: "date-time"`

Purpose:

- when the plan was created

`target_repo_type`

Type:

- `string`

Enum:

- `web_app`
- `mobile_app`
- `api_service`
- `fullstack_monorepo`

Purpose:

- high-level repo type

`target_repo_summary`

Type:

- `string`

Purpose:

- human-readable repo target description

Constraint:

- non-empty string

`phases`

Type:

- `array` of phase objects

Purpose:

- ordered build phases/slices

Constraint:

- at least 1 item

Each phase object should require:

- `phase_id`
- `order`

Optional:

- `purpose`
- `target_areas`
- `entry_dependencies`
- `expected_outputs`

`required_outputs`

Type:

- `object`

Purpose:

- summary of what the build must materialize

Minimum properties:

- `repo_root_required`
- `manifests_required`
- `verification_required`

Optional:

- `export_artifact_required`

`constraints`

Type:

- `object`

Purpose:

- generation constraints that prevent drift

Minimum properties:

- `allow_scope_expansion`
- `allow_undeclared_dependencies`
- `block_on_missing_required_inputs`
- `block_on_critical_placeholders`

All booleans.

`verification_expectations`

Type:

- `object`

Purpose:

- what downstream verification must check

Minimum properties:

- `require_structure_check`
- `require_placeholder_scan`
- `require_manifest_check`

Optional:

- `require_command_checks`
- `require_export_readiness_when_requested`

All booleans.

`status`

Type:

- `string`

Enum:

- `draft`
- `active`
- `superseded`

Purpose:

- plan-local lifecycle status

`notes`

Type:

- `string` or `null`

Purpose:

- optional freeform note

`planner_version`

Type:

- `string`

Purpose:

- optional planner version identifier

---

8. Nested object outlines

Phase object schema

Each `phases[]` item should look like:

Required:

- `phase_id`
- `order`

Optional:

- `purpose`
- `target_areas`
- `entry_dependencies`
- `expected_outputs`

Recommended shape:

```
{
 "type": "object",
 "required": ["phase_id", "order"],
 "properties": {
   "phase_id": { "type": "string", "minLength": 1 },
   "order": { "type": "integer", "minimum": 1 },
   "purpose": { "type": "string" },
   "target_areas": {
     "type": "array",
     "items": { "type": "string" }
   },
   "entry_dependencies": {
     "type": "array",
     "items": { "type": "string" }
   },
   "expected_outputs": {
     "type": "array",
     "items": { "type": "string" }
   }
```

```
 },
 "additionalProperties": false
}
```

`required_outputs` object schema

Minimum shape:

```
{
 "type": "object",
 "required": [
   "repo_root_required",
   "manifests_required",
   "verification_required"
 ],
 "properties": {
   "repo_root_required": { "type": "boolean" },
   "manifests_required": { "type": "boolean" },
   "verification_required": { "type": "boolean" },
   "export_artifact_required": { "type": "boolean" }
 },
 "additionalProperties": false
}
```

`constraints` object schema

Minimum shape:

```
{
 "type": "object",
 "required": [
   "allow_scope_expansion",
   "allow_undeclared_dependencies",
   "block_on_missing_required_inputs",
   "block_on_critical_placeholders"
 ],
 "properties": {
   "allow_scope_expansion": { "type": "boolean" },
   "allow_undeclared_dependencies": { "type": "boolean" },
   "block_on_missing_required_inputs": { "type": "boolean" },
   "block_on_critical_placeholders": { "type": "boolean" }
```

```
 },
 "additionalProperties": false
}
```

`verification_expectations` object schema

Minimum shape:

```
{
 "type": "object",
 "required": [
   "require_structure_check",
   "require_placeholder_scan",
   "require_manifest_check"
 ],
 "properties": {
   "require_structure_check": { "type": "boolean" },
   "require_placeholder_scan": { "type": "boolean" },
   "require_manifest_check": { "type": "boolean" },
   "require_command_checks": { "type": "boolean" },
   "require_export_readiness_when_requested": { "type": "boolean" }
 },
 "additionalProperties": false
}
```

---

9. `additionalProperties`

Set top-level:

"additionalProperties": false

Reason:
Plans should be strict. If the planner invents new ad hoc fields, that is usually drift.

---

10. Validation rules beyond basic types

The schema should enforce these consistency rules:

Rule 1

`derived_from_refs` must contain at least 1 item.

Rule 2

`phases` must contain at least 1 phase.

Rule 3

If `required_outputs.export_artifact_required = true`, then `verification_expectations.require_export_readiness_when_requested` should normally be `true`.

Rule 4

If `constraints.allow_scope_expansion = true`, that should likely be rejected for the first version unless you explicitly want that allowed. Safer first version:

- require `allow_scope_expansion = false`

Rule 5

Safer first version:

- require `allow_undeclared_dependencies = false`

That keeps Build Mode locked by default.

You can enforce rules 4 and 5 with `const: false` if that is your desired initial discipline.

---

11. Recommended schema structure

The schema should have these sections:

1. metadata
2. `type: object`

3. `required`
4. `properties`
5. nested definitions for phases/constraints/outputs/verification expectations
6. optional `allOf` consistency rules
7. `additionalProperties: false`

---

12. Example property block

A clean top-level property section would include:

- string IDs/refs
- datetime fields
- enum fields
- nested `phases`
- nested `required_outputs`
- nested `constraints`
- nested `verification_expectations`

---

13. Example valid JSON


```
{
 "plan_id": "BPL-0001",
 "run_id": "RUN-0001",
 "build_request_id": "BRQ-0001",
 "build_session_id": "BSES-0001",
 "build_basis_ref": "KIT-0001",
 "derived_from_refs": ["STD-0001", "ROUTE-SPEC-0001"],
```

"build_profile_ref": "PROFILE-0001",
"repo_blueprint_ref": "RBP-0001",
"required_file_inventory_ref": "RFI-0001",
"policy_snapshot_ref": "BPS-0001",
"plan_created_at": "2026-03-06T12:03:00Z",
"target_repo_type": "fullstack_monorepo",
"target_repo_summary": "Generate a fullstack monorepo with frontend, backend, shared contracts, tests, and supporting config files.",
"phases": [
{ "phase_id": "prepare_workspace", "order": 1 },
{ "phase_id": "create_repo_skeleton", "order": 2 },
{ "phase_id": "materialize_core_config", "order": 3 },
{ "phase_id": "materialize_shared_contracts", "order": 4 },
{ "phase_id": "materialize_backend", "order": 5 },
{ "phase_id": "materialize_frontend", "order": 6 },
{ "phase_id": "materialize_tests_and_supporting_files", "order": 7 },
{ "phase_id": "write_manifests", "order": 8 },
{ "phase_id": "handoff_to_verification", "order": 9 }
],
"required_outputs": {
"repo_root_required": true,
"manifests_required": true,
"verification_required": true,
"export_artifact_required": true
},
"constraints": {
"allow_scope_expansion": false,
"allow_undeclared_dependencies": false,
"block_on_missing_required_inputs": true,
"block_on_critical_placeholders": true
},
"verification_expectations": {
"require_structure_check": true,
"require_placeholder_scan": true,
"require_manifest_check": true,
"require_command_checks": true,
"require_export_readiness_when_requested": true
},
"status": "active",
"notes": null,
"planner_version": "1.0.0"
}

---

14. What this schema should reject

It should reject:

- missing `plan_id`
- missing `phases`
- empty `phases`
- invalid `target_repo_type`
- empty `derived_from_refs`
- missing `constraints`
- missing `verification_expectations`
- extra unknown fields
- phase items with missing `phase_id` or `order`
- top-level drift fields not defined in the schema

---

15. Success criteria

`build_plan.schema.json` is successful when it enforces a strict, execution-ready build plan object that is explicit about repo target, ordered phases, required outputs, constraints, and verification expectations, so generation does not have to improvise the build path.

build_manifest.schema.json

Physical file list

Build runtime directory contract

Implementation order